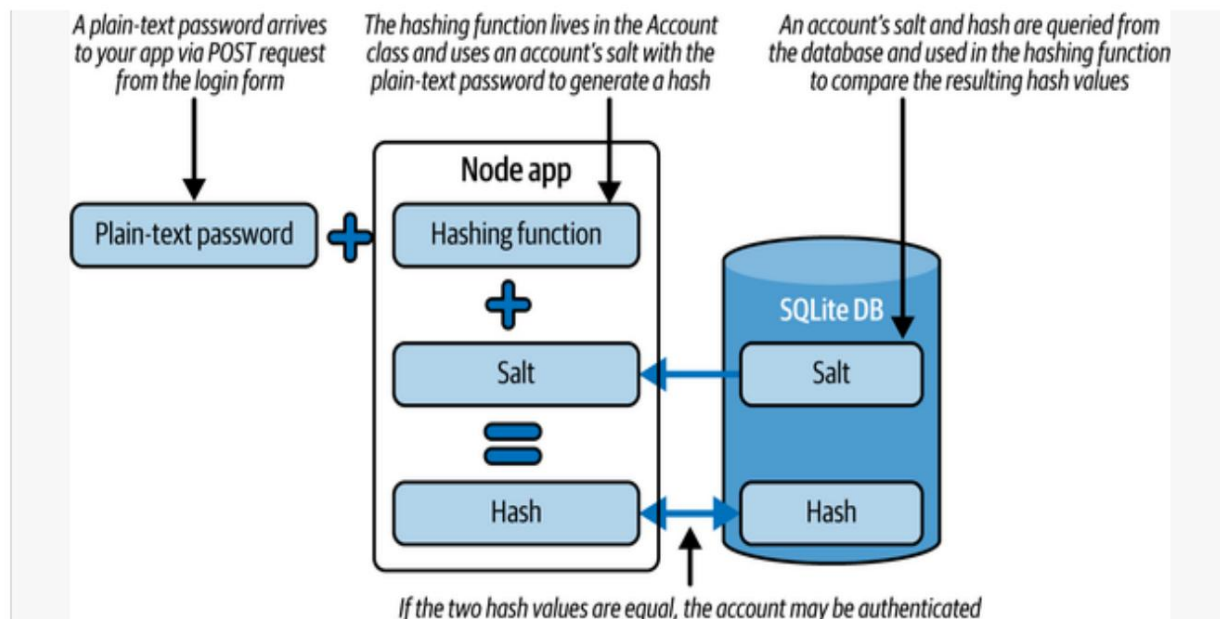




Cifra 106. Flujo de información para el hash de la contraseña de una cuenta

En este proyecto se utiliza el algoritmo hash `crypto.pbkdf2`.

Este algoritmo se considera seguro y rápido. Para obtener más información sobre la API de cifrado, consulte la [documentación de Node](#).

Además de `crypto.pbkdf2`, existen otros algoritmos de hash de contraseñas que se utilizan ampliamente en aplicaciones modernas, y cada uno ofrece diferentes ventajas y desventajas en términos de velocidad, seguridad y resistencia a diversos tipos de ataques:

Argón2

Esta función de derivación de claves fue diseñada para resistir diversos tipos de ataques, como el cracking de GPU y los ataques de diccionario. Se considera una de las funciones hash más seguras disponibles actualmente. Más información en el [sitio web de npm](#).

bcrypt

Esta es una función de hash de contraseña que utiliza una técnica llamada hash adaptativo para hacerla resistente a la fuerza bruta.

Ataques. Se usa ampliamente en aplicaciones web y se considera una opción muy segura. Lea más en el [sitio web de npm](#).

cifrado

Está diseñada para ser una función de derivación de clave de propósito general y generalmente es más lenta que pbkdf2 debido a su resistencia adicional a la aceleración de hardware y a los ataques de diccionario. Lea más en la [documentación de Node](#).

Al final del archivo, se utiliza un gancho de Sequelize , beforeCreate, que se ejecuta antes de guardar la cuenta en la base de datos. Esta es una función integrada de Sequelize . En este caso, se reasigna el nombre de usuario a minúsculas. Por último, se ejecuta Account.sync para sincronizar el modelo con la base de datos creando la tabla "Cuentas" si aún no existe.

Ejemplo 1011. Configuración de su modelo de cuenta Sequelize en Account.js

```
...
Cuenta.init( {
  nombre de
    usuario: { tipo: Sequelize.STRING,
    allowNull: falso, único:
    verdadero, },

  hash:
    { tipo: Sequelize.TEXT,
    allowNull: falso, }, sal:

  { tipo:
    Sequelize.STRING, allowNull:
    falso, }, }, {
```

```
sequelize: db, 5
modelName: "Cuenta", } );
```

```
Cuenta.beforeCreate((cuenta) => { 6
  cuenta["nombre de usuario"] = cuenta["nombre de usuario"].toLowerCase(); });
```

```
Account.sync(); 7
exportar cuenta predeterminada ;
```

- 1** Inicialice la clase Cuenta para trabajar con Sequelize.
- 2** Defina un campo de nombre de usuario de tipo STRING.
- 3** Defina un campo hash de tipo TEXTO.
- 4** Defina un campo de sal de tipo STRING.
- 5** Especifique la instancia de la base de datos y el nombre del modelo.
- 6** Utilice el gancho beforeCreate para convertir todos los valores de nombre de usuario en minúsculas.
- 7** Sincronizar el modelo de cuenta con la base de datos.

Con el modelo de datos configurado, puede incorporar funciones de configuración de contraseñas y autenticación. Primero, importe el módulo passport-local agregando `import { Strategy as LocalStrategy } from "passport-local"` al inicio de JavaScript. Esta biblioteca le ayudará a definir su cuenta de autenticación.

Estrategia. LocalStrategy se refiere a un proceso de autenticación que utiliza únicamente nombre de usuario y contraseña. En [el Ejemplo 10-12](#), se agregan las funciones `register` y `setPassword`, que se utilizan para crear nuevas cuentas con contraseñas cifradas.

La función de registro acepta los parámetros de nombre de usuario y contraseña, y primero comprueba si existe una cuenta mediante el nombre de usuario. Si no existe, puede continuar creando una cuenta. A diferencia del objeto de usuarios en memoria mencionado anteriormente en este capítulo, no puede crear cuentas duplicadas ni anular las existentes. La función `this.build` del modelo `Cuenta` crea una nueva cuenta.

instancia de una `Cuenta` con el campo de nombre de usuario.

NOTA

La función de compilación no realiza una operación de E/S y, por lo tanto, no es una función que necesite las palabras clave `async-await`.

Con su instancia de cuenta, `cuenta`, establece la contraseña de la cuenta llamando a la función `setPassword` en esa nueva instancia. Esta función gestionará todas las operaciones de hash. Por último, se llama a `account.save` para guardar oficialmente la cuenta en la base de datos.

Ejemplo 1012. Definiendo su función de creación de cuenta en `Account.js`

```
...
registro asíncrono estático (nombre de usuario, contraseña) { ❶
  const existingAccount = await
this.findByUsername(nombre de ❷
usuario); if (existingAccount) ❸
  { throw new Error("La cuenta ya existe");

  } const cuenta = this.build({ nombre de usuario }); ❹
  cuenta.setPassword(contraseña); return ❺
  await cuenta.save(); ❻
}
...
```

❶ Define una función de registro que acepte un nombre de usuario y una contraseña.

- ② Verifique su base de datos para encontrar un registro de cuenta existente .
- ③ Genera un error si ya existe una cuenta.
- ④ Cree una nueva instancia de cuenta virtual con los datos proporcionados
nombre de usuario.
- ⑤ Llame a la función setPassword para guardar una versión hash de su contraseña.
- ⑥ Guardar y devolver el objeto de cuenta registrado.

Para que la función de registro funcione, debe crear la función setPassword dentro de la clase Account . Agregue el código del **Ejemplo 10-13** debajo de la función de registro en Account.js. Esta función acepta una contraseña como parámetro. Primero comprueba que la contraseña tenga un valor antes de continuar con la función hash.

CONSEJO

Con la llegada de async-await, la programación se ha vuelto mucho más concisa y fácil de leer. En esencia, todas las funciones que se pueden esperar son solo funciones que devuelven una Promesa. Una Promesa devuelve un valor eventual, lo que permite que la lógica del servidor continúe sin bloquearse hasta que ese valor esté listo. Para funciones con operaciones síncronas o que aún no admiten un valor de retorno esperable, puede crear su propia Promesa. Encapsular la lógica en una Promesa le permite llamar a esa función con la palabra clave async-await.

Dentro del bloque try-catch se inicia la lógica de hash. Recuerde que un algoritmo de hash puede contener diferentes componentes para garantizar un valor hash resultante que no pueda ser modificado mediante ingeniería inversa para obtener la contraseña original. Uno de los criterios para generar el hash es...

Crea un valor de sal personalizado (y aleatorio) . Node incluye la biblioteca crypto para facilitar este tipo de operaciones. Primero, importa esa biblioteca añadiendo "import crypto from 'crypto'" al principio de Account.js. Luego, dentro de la función setPassword , usa crypto.randomBytes para generar una selección aleatoria de bytes.

NOTA

Se pasa 32 para indicar el número de caracteres que se desea que tenga la sal . No hay una respuesta correcta, aunque una longitud de 32 o más es un valor ampliamente aceptado para una cadena de sal segura . Para obtener más información, lea el artículo ["Sal de contraseñas"](#). Tras generar un conjunto aleatorio de bytes, se convierte a una cadena hexadecimal mediante la función bufferBytes.toString("hex") . A continuación, se utiliza la función hash oficial. La función crypto.pbkdf2Sync genera un hash sincrónicamente con los argumentos proporcionados. Se pasa la contraseña en texto plano , la sal generada , 12000 como número de iteraciones de hash, una longitud de bytes de 64 y el algoritmo de resumen de hash.

Tenga en cuenta también que este ejemplo utiliza el resumen SHA512 , que se prefiere al SHA1 para obtener garantías criptográficas más sólidas. Puede consultar la [documentación de Node](#). Para saber más.

El valor hashRaw resultante se convierte a una cadena hexadecimal y se guarda en el campo hash de la cuenta . El valor de sal generado también se guarda en el campo de sal de la cuenta . Dado que todo el proceso es síncrono, no es necesario incluirlo en una promesa ni usar await.

Ejemplo 1013. Definiendo su función de hash de contraseña en Account.js

...

```
setPassword(contraseña)
{ si (!contraseña) {
```



```

    generar nuevo Error("No se proporcionó la contraseña."); }

    try
    { const bufferBytes = crypto.randomBytes(32); const salt =
      bufferBytes.toString("hex"); const hashRaw =
      crypto.pbkdf2Sync(contraseña, salt, 12000, 64, "sha512"); this.set("hash",
      Buffer.from(hashRaw).toString("hex"));
    this.set("salt", salt); } catch (e)
    { throw new
      Error("No se puede generar el hash de la contraseña."); }
  }
  ...

```

- 1 Define tu función setPassword para tomar una contraseña de texto simple .
- 2 Genere una variedad aleatoria de bytes para su sal.
- 3 Asigne la cadena de bytes convertida como su valor de sal .
- 4 Genere un valor hash a partir de su contraseña, sal y parámetros hash.
- 5 Guarde el valor de la cadena de su hash en la instancia de su cuenta .
- 6 Guarde el valor de la cadena de su sal en la instancia de su cuenta .
- 7 Genera un error si ocurre algún problema en el proceso de hash.

Una vez completado el registro de cuenta y la función hash, se agrega la lógica para autenticar el inicio de sesión de un usuario. Agregue la función del **Ejemplo 10-14** a la clase Cuenta . La función de autenticación se proporciona con la contraseña en texto plano que el usuario ha ingresado. La función devuelve verdadero si la contraseña es correcta o falso si es incorrecta. La función comienza extrayendo la sal y el hash.

Valores del objeto `this` , que se supone que es una instancia de una cuenta de usuario. Si el valor de `sal` no está presente, la función lanza un error.

NOTA

Dado que se llama a `authenticate` en la instancia de la cuenta , y no en la clase en sí, la palabra clave `"this"` se refiere a la instancia. De esta manera, se puede acceder a todos los campos de esa instancia.

A continuación, la función utiliza la función `crypto.pbkdf2Sync` para generar un hash a partir de la contraseña y los valores de `sal` proporcionados. A continuación, compara el hash generado con el hash almacenado para la cuenta de usuario. Si ambos hashes coinciden, la función devuelve `"true"`, lo que indica una autenticación exitosa. Si los hashes no coinciden, devuelve `"false"`. Este diseño separa la verificación de contraseña de cualquier flujo de control circundante, como la devolución de llamada `"done"` en `Passport.js`.

Clase de 1014. Añade un método de autenticación a tu cuenta
ejemplo en `Account.js`

...

```
autenticar(contraseña) { const ❶
  { sal, hash } = esto; si (!sal) { lanzar
  nuevo Error("No ❷
    se encontró sal"); }
```

```
const hashRaw = crypto.pbkdf2Sync( contraseña, sal, ❸
  12000, 64,
```

```
"sha512" );
```

```
❹ const currentHash = Buffer.from(hashRaw).toString("hex");
```

```
    devuelve currentHash === hash;    5
  }
  ...
```

- 1 Define la función de autenticación que acepta una contraseña como argumento.
- 2 Verifique si la cuenta tiene una sal válida antes de continuar.
- 3 Genere el valor hashRaw utilizando la función hash crypto.pbkdf2Sync .
- 4 Convierte el valor hashRaw en una cadena llamada currentHash.
- 5 Devuelve verdadero si los hashes coinciden, de lo contrario devuelve falso.

Con este método de autenticación listo, debe hacerlo accesible a la biblioteca Passport.js. Agregue la función de clase del **Ejemplo 10-15** a su clase Account . Este código define un método estático llamado passportAuthenticate en la clase Account . El método passportAuthenticate devuelve una función asíncrona que toma tres argumentos: nombre de usuario, contraseña y "doin". Intenta encontrar una cuenta con el nombre de usuario especificado llamando al método findByUsername . Si se encuentra una cuenta, llama al método de instancia authenticate con la contraseña proporcionada. Si la contraseña es válida, la función llama a "doin" con el objeto account.

De lo contrario, responde con un mensaje de error.

Este diseño desacopla la verificación de contraseña de la invocación de devolución de llamada, lo que hace que el método de autenticación sea más fácil de probar de forma independiente.

su clase 1015. Agregue un método de clase passportAuthenticate al ejemplo de Account en Account.js

```

...
pasaporte estáticoAuthenticate () 1
{ return async (nombre de usuario, contraseña, hecho) => 2
  prueba
    { const cuenta = await this.findByUsername(nombre de usuario);
3
    if (!cuenta) { return
      done(null, false, { message: "Usuario no encontrado"
4
    });
    }

    const isValid = cuenta.authenticate(contraseña); if (isValid) { 5

      return done(null, cuenta); } else 6
    { return
      done(null, false, { mensaje: "Contraseña
incorrecto" }); } } 7

    catch (err) { return
      done(err); 8

    }
  }
}

```

- ...
- 1** Define un método estático que devuelve una devolución de llamada de estrategia.
 - 2** Passport.js utilizará esta función asíncronica para los intentos de inicio de sesión.
 - 3** Busque la cuenta de usuario por nombre de usuario.
 - 4** Si no lo encuentra, responda con un mensaje de "Usuario no encontrado".
 - 5** Llame al método de autenticación de la instancia para validar la contraseña.
 - 6** Si es válido, la llamada se realiza con la cuenta autenticada.
 - 7** Si no es válido, se realiza la llamada con un mensaje de error.
 - 8** Manejar y devolver cualquier error inesperado.

Este código concluye el proceso de autenticación. Si bien Passport.js incluye soporte adicional para mantener el estado entre solicitudes de usuario, en lugar de requerir que los usuarios inicien sesión cada vez que solicitan una nueva página, Passport.js envía datos en cada solicitud que confirman el estado de autenticación de la cuenta.

El ejemplo 10-16 añade dos métodos más a la clase Account . Las funciones `serializeUser` y `deserializeUser` se utilizan para serializar y deserializar cuentas de usuario para su almacenamiento en una sesión. Estas funciones suelen utilizarse junto con un middleware de sesión, como `@fastify/session`. La función `serializeUser` toma como argumentos "account", el objeto de cuenta de usuario que debe serializarse, y "doin", una función de devolución de llamada que se invoca al finalizar el proceso de serialización. La función extrae la propiedad `username` del objeto `account` y la pasa a la devolución de llamada "doin" como segundo argumento.

La función `deserializeUser` toma como argumentos el nombre de usuario, los datos serializados del usuario utilizados para encontrar la cuenta de usuario y la función de devolución de llamada "done", que se invoca al finalizar el proceso de deserialización. Esta función llama al método `findByUsername` para encontrar la cuenta con el nombre de usuario proporcionado y, a continuación, pasa la cuenta encontrada a la función de devolución de llamada "done" como segundo argumento.

UNA BREVE SESIÓN SOBRE LAS SESIONES

En una aplicación web, una sesión almacena datos de usuario entre solicitudes. Cuando un usuario inicia sesión, el servidor crea una nueva sesión y almacena sus datos en ella. Posteriormente, el servidor envía una cookie al navegador del cliente con un identificador único para la sesión.

NOTA

Una cookie de navegador es un pequeño fragmento de datos que el navegador web almacena en el ordenador del usuario. Se utiliza para identificar el navegador del usuario y rastrear sus movimientos dentro de un sitio web. Las cookies se utilizan normalmente para registrar las sesiones de inicio de sesión, almacenar las preferencias del usuario y personalizar su experiencia en un sitio web.

En solicitudes posteriores, el cliente envía la cookie de vuelta al servidor junto con la solicitud. El servidor utiliza el identificador de sesión de la cookie para consultar los datos del usuario en el almacén de sesiones y recuperarlos para la solicitud.

En Node con Fastify, las sesiones permiten monitorizar el estado del usuario mientras navega por la aplicación. Para usar sesiones en Fastify, debe instalar y configurar los plugins `@fastify/session` y `@fastify/cookie`. Estos funcionan en conjunto para permitir la creación de sesiones y la gestión de cookies.

Passport.js se integra con Fastify mediante el plugin

`@fastify/passport`. Cuando un usuario inicia sesión, Passport.js almacena sus datos en la sesión para que se pueda acceder a ellos en futuras solicitudes. Esto permite que el usuario mantenga la sesión iniciada mientras navega por la aplicación.

Ejemplo Agregue los métodos de clase `serializeUser` y `deserializeUser` a su clase `Account` en `Account.js`

...

```
static serializeUser(cuenta, hecho) { const { nombre
  de usuario } = cuenta; hecho(null,
  nombre de usuario); }
```

```
static async deserializeUser(nombre de usuario, hecho) {
  prueba
  { const foundAccount = await
this.findByUsername(nombre de usuario); if (!
  foundAccount) {
    devolver hecho(nuevo Error("Usuario no encontrado")); ❶

    } hecho(null, foundAccount); } ❷
  catch (err)
    { hecho(err); ❸
  }
}
```

...

- ❶ Devuelve un error si no se encuentra una cuenta coincidente.
- ❷ Pase la cuenta encontrada al deserializador de sesión Passport.js.
- ❸ Si ocurre un error en la base de datos, páselo a la devolución de llamada realizada .

El último paso para habilitar Passport.js es crear una nueva estrategia de inicio de sesión. El ejemplo 10-17 define la función `genStrategy` , que se utiliza para crear una nueva instancia de la clase `LocalStrategy` desde la biblioteca `Passport.js`. La función `genStrategy` devuelve una nueva instancia de la clase `LocalStrategy` , con la función `passport Authenticate` como retorno de llamada. Cuando la estrategia se utiliza para autenticar una cuenta, se llamará a la función `passportAuthenticate` para gestionar el proceso de autenticación mediante un nombre de usuario y una contraseña.

NOTA

La clase LocalStrategy toma un objeto de opciones como argumento, que puede incluir una variedad de opciones, como los campos utilizados para el nombre de usuario y contraseña, así como una función de devolución de llamada para manejar el proceso de autenticación.

Clase de 1017. Agregue un método de clase genStrategy a su cuenta ejemplo en Account.js

```
...
genStrategy estático () { 1
  devuelve nueva LocalStrategy(this.passportAuthenticate()); 2
}
```

1 Define el método de clase genStrategy para devolver un nuevo Instancia de LocalStrategy .

2 Devuelve una nueva LocalStrategy usando tu Método de clase passportAuthenticate .

Con esta función de estrategia configurada, puede agregar el resto Configuraciones de index.js. Para usar Passport.js con sesiones en Fastify, Asegúrese de tener instalados los complementos de sesión y pasaporte necesarios:

```
instalación de npm
  @fastify/cookie@^10.0.0 @fastify/sesión@^11.0.0
  @fastify/passport@^3.0.0
```

Luego, importe los módulos necesarios en la parte superior de index.js:

```
importar fastifyCookie desde "@fastify/cookie";
importar fastifySession desde "@fastify/session";
importar fastifyPassport desde "@fastify/passport";
importar Cuenta desde "../models/Account.js";
```

El ejemplo 10-18 configura los plugins de sesión y pasaporte compatibles con Fastify. También registra las funciones de serialización y deserialización, que se utilizan para almacenar y recuperar datos de usuario de la sesión. Las sesiones de Fastify requieren que `@fastify/cookie` y `@fastify/session` se registren primero.

NOTA

El secreto utilizado para la configuración de la sesión debe ser una cadena fuerte y aleatoria y normalmente se almacena en una variable de entorno.

Ejemplo 1018. Agregue funciones de middleware de sesión y Passport a de su aplicación Fastify en `index.js`

...

```
await app.register(fastifyCookie); await 1
app.register(fastifySession, { secreto: "un_valor_muy_secreto_1!
2@3#4$5%6^7&8*9(0)", cookie: { seguro: falso, edad_máxima: 1000 * 60 * 60 * 24 },
guardar sin
  inicializar: falso, volver
  a guardar: falso });
```

²

```
esperar aplicación.registrar(fastifyPassport.initialize()); esperar 3
aplicación.registrar(fastifyPassport.secureSession()); 4
```

```
fastifyPassport.registerUserSerializer(async (usuario, solicitud) => usuario.nombreusuario);
```

⁵

```
fastifyPassport.registerUserDeserializer(async (nombreusuario, solicitud) => { const
account = await
  Account.findByUsername(nombreusuario); if (!account) { throw new Error("Usuario
no encontrado");
} devolver cuenta;
```

}); ❷

`fastifyPassport.use("local", Account.genStrategy());` Registra el ❷

❶ complemento de cookies de Fastify para analizar cookies.

❷ Registre el complemento de sesión de Fastify con un secreto y una configuración, incluida la duración de la sesión.

❸ Registra el middleware de inicialización de Passport.

❹ Registra el middleware de sesión de Passport (a través de `secureSession()`).

❺ Registra una función de serialización asincrónica que devuelva el nombre de usuario directamente.

❻ Registra una función de deserialización asincrónica que genera un error.

❼ Registre la estrategia de inicio de sesión local personalizada.

NOTA

La principal diferencia con Express es que las funciones de serialización de Fastify son asíncronas y devuelven valores directamente, en lugar de usar devoluciones de llamada. Además, reciben el objeto de solicitud como parámetro.

Tu aplicación ya está completamente configurada para usar el modelo de Cuentas y sus métodos para registrar y autenticar nuevas cuentas. El último paso es crear puntos de entrada para que las cuentas nuevas y existentes interactúen con tu servidor Fastify.

El ejemplo 10-19 define dos rutas nuevas. La primera es una ruta POST que recibe solicitudes en el punto final `/cuenta`. Cuando se recibe una solicitud en este punto final, el servidor extrae los campos de nombre de usuario y contraseña del cuerpo de la solicitud y llama al

Método de registro de la clase Cuenta . Este método registra una nueva cuenta con las credenciales proporcionadas. El servidor responde con un mensaje JSON indicando si la operación fue exitosa o no.

La segunda ruta es una ruta POST que recibe solicitudes en el punto final /auth . Esta ruta gestiona la autenticación llamando al método authenticate de Passport dentro de un bloque try-catch. A diferencia de Express, la integración de Passport con Fastify genera errores si falla la autenticación, que deben detectarse y gestionarse adecuadamente.

Ejemplo 1019. Defina nuevos puntos finales /account y /auth con de manejo de errores en index.js

```
...
app.post("/cuenta", async (solicitud, respuesta) => { const { nombre de usuario, ❶
  contraseña } = solicitud.cuerpo; try { await Cuenta.register(nombre de ❷
    usuario,
    contraseña); return respuesta.send({ mensaje: "Cuenta creada." }); } ❸
    catch (e) { return respuesta.código(400).send({ mensaje: "Error en la creación de ❹
      la cuenta.", error:
      e.mensaje });
    }
  });

app.post("/auth", async (solicitud, respuesta) => { ❺
  try
    { await
      fastifyPassport .authenticate("local", { authInfo: false })(solicitud, ❻
        respuesta); if (solicitud.usuario) ❼
        { const { nombre de usuario } = solicitud.usuario;
          return responder.send({ mensaje: "Iniciado sesión.", nombre de usuario
        }); ❽

        } } atrapar (err) {
          devolver respuesta.code(401).send({ ❿
            mensaje: "Error de autenticación",
```

error: "Nombre de usuario o contraseña no válidos"

```
}});
```

...

- ➊ Define la ruta para el registro de la cuenta utilizando la aplicación app.post de Fastify.
- ➋ Extraiga el nombre de usuario y la contraseña del cuerpo de la solicitud.
- ➌ Intente registrar una nueva cuenta con las credenciales proporcionadas.
- ➍ Responda con un mensaje de éxito luego de registrarse exitosamente.
- ➎ Responda con un mensaje de error y el código de estado 400 si falla la creación de la cuenta.
- ➏ Define la ruta de autenticación que maneja los intentos de inicio de sesión.
- ➐ Llame al método de autenticación de Passport, que devuelve una función que procesa la solicitud.
- ➑ Compruebe si la autenticación se realizó correctamente verificando que el objeto de usuario existe.
- ➒ Responda con un mensaje de éxito y el nombre de usuario registrado.
- ➓ Manejar errores de autenticación con un estado 401 No autorizado.

NOTA

A diferencia de Express, la integración de Passport de Fastify requiere llamar a `authenticate` como una función que devuelve otra función.

Los errores de autenticación se manejan a través de bloques `try-catch` en lugar de URL de redireccionamiento.

Tu código ya está completo y listo para probar con Passport.js en una aplicación Fastify. Reinicia tu aplicación y crea una cuenta nueva visitando el formulario de registro o usando el siguiente comando cURL:

```
curl -X POST http://127.0.0.1:3000/cuenta \
  -H "Tipo de contenido: aplicación/x-www-form-urlencoded" \ -d "nombre de
  usuario=usuario de prueba&contraseña=securepass123"
```

Deberías recibir una respuesta como esta:

```
{"message":"Cuenta creada."}
```

Después de crear tu cuenta, puedes probar la autenticación. Primero, intenta iniciar sesión con credenciales incorrectas:

```
curl -X POST http://127.0.0.1:3000/auth \
  -H "Tipo de contenido: aplicación/x-www-form-urlencoded" \ -d "nombre de
  usuario=usuario de prueba&contraseña=contraseña incorrecta"
```

Recibirás una respuesta 401 no autorizada:

```
{"message":" Error de autenticación","error":" Nombre de usuario o
  contraseña no válidos"}
```

Ahora, intenta iniciar sesión con la contraseña correcta:

```
curl -X POST http://127.0.0.1:3000/auth \
  -H "Tipo de contenido: aplicación/x-www-form-urlencoded" \ -d "nombre de
  usuario=usuario de prueba&contraseña=securepass123"
```

Una autenticación exitosa devolverá:

```
{"message":"Iniciado sesión.", "username":"testuser"}
```

Su aplicación ahora puede aceptar y mantener nuevas cuentas de usuario y autenticarlas de forma segura a través del inicio de sesión basado en sesión. El flujo de autenticación en Fastify proporciona códigos de estado HTTP y

Respuestas JSON adecuadas tanto para clientes basados en navegador como para clientes programáticos.

La siguiente sección demuestra cómo agregar una estrategia de autenticación adicional para soportar la autenticación sin estado para clientes que no son navegadores (como aplicaciones móviles e integraciones de terceros) mediante la emisión y verificación de tokens web JSON (JWT).

Uso de JWT para la autenticación de API. Imagina que tu aplicación

de viajes tiene tanto éxito que empiezas a ofrecer compatibilidad con aplicaciones móviles y clientes de terceros. Estas plataformas necesitan autenticar a los usuarios, al igual que tu aplicación web. Sin embargo, la autenticación basada en sesiones con cookies (usada en tu configuración de Fastify + Passport) no funciona bien en entornos que no pueden almacenar cookies ni mantener sesiones.

En cambio, un sistema de autenticación sin estado basado en tokens es más adecuado para las API. Aquí es donde entran en juego los tokens web JSON (JWT).

Los JWT son tokens autónomos que incluyen todos los detalles de autenticación en su carga útil. Se utilizan comúnmente en sistemas sin estado, como clientes móviles, SPA y microservicios, donde el servidor no necesita conservar los datos de la sesión. Cuando un usuario inicia sesión, el servidor genera un token firmado que contiene su identidad. El cliente envía dicho token en el encabezado de autorización para cada solicitud posterior.

Por el contrario, la autenticación basada en sesión almacena el estado de la sesión en el servidor y espera que el navegador guarde una cookie con el ID de la sesión. Este enfoque funciona bien para aplicaciones web tradicionales renderizadas en servidor, pero no tanto para sistemas distribuidos.

Con Passport.js, puedes admitir ambos métodos de autenticación en paralelo:

1. Instale los paquetes necesarios ejecutando el siguiente comando en su terminal:

```
npm instalar pasaporte-jwt@^4.0.1 jsonwebtoken@^9.0.2
```

2. Importe los módulos JWT necesarios en la parte superior de Account.js:

```
importar { Estrategia como JWTStrategy, ExtractJwt } de "passport-jwt";  
importar jwt de "jsonwebtoken";
```

El asistente ExtractJwt proporciona funciones para extraer un token de una solicitud HTTP, generalmente del encabezado de autorización utilizando el esquema Bearer.

3. Agregue la estrategia JWT y la lógica de firma a su clase Cuenta .

El Ejemplo 10-20 muestra los métodos genJWTStrategy y signJWT . La estrategia extrae el nombre de usuario del token decodificado e intenta encontrar una cuenta coincidente en su base de datos. Si tiene éxito, autentica la solicitud.

NOTA

En producción, su clave secreta JWT nunca debe estar codificada. Use process.env.JWT_SECRET y almacene los secretos de forma segura en su entorno o administrador de secretos.

Ejemplo 1020. Defina los métodos genJWTStrategy y signJWT en Account.js

```
static genJWTStrategy() { return ❶  
  new JWTStrategy( ❷  
    {  
      jwtDeSolicitud:
```

```

    ExtraerJwt.deAuthHeaderAsBearerToken(),
    secretOrKey: proceso.env.JWT_SECRET || "CLAVE_SECRETA", }, async

    (jwtPayload, hecho) => { intentar { const cuenta
    =
    esperar
    esto.findByUsername(jwtPayload.nombreusuario);
    si (cuenta) {
        retorna hecho(null, cuenta);
    }
    return done(null, false, { mensaje: "Usuario no encontrado"
    });
    } catch (e) { return
    hecho(e);
    }

    });
}

signJWT()
{ const { nombre_de_usuario } =
  this; return jwt.sign({ nombre_de_usuario }, process.env.JWT_SECRET ||
  "CLAVE_SECRETA"); }

```

- 1 Defina el método genJWTStrategy para devolver una nueva estrategia JWT de Passport.
- 2 Devuelve una nueva instancia de JWTStrategy para autenticar tokens portadores.
- 3 Extraiga el token JWT del encabezado de autorización de la solicitud .
- 4 Utilice una clave secreta segura para verificar los tokens.
- 5 Busque en la base de datos una cuenta que coincida con el nombre de usuario en la carga útil del token.
- 6 Si se encuentra una coincidencia, autentique la solicitud.

- 7 Si no se encuentra ninguna coincidencia, deniegue la autenticación.
- 8 En caso de error, se realiza la llamada con el error.
- 9 Defina signJWT para generar un token firmado.
- 10 Devuelve un JWT firmado que codifica el nombre de usuario de la cuenta.

Una vez implementada esta lógica, actualice su servidor Fastify para registrar la nueva estrategia JWT junto con la local. En su archivo index.js , agregue:

```
fastifyPassport.use("jwt", Cuenta.genJWTStrategy());
```

Esto registra la estrategia "jwt" con Passport.js. Ahora puede crear rutas de API que usen passport.authenticate("jwt", ...) para proteger el acceso mediante autenticación basada en tokens.

En la siguiente sección, definirá los puntos finales de la API para emitir y verificar JWT para clientes móviles o basados en API. Por último, defina nuevos puntos finales para clientes no web. Agregue el código del **Ejemplo 10-21** después de las rutas existentes en index.js. Este código define dos rutas de Fastify: una para la autenticación y otra para probar el acceso a recursos protegidos. Ambas utilizan el middleware de autenticación Passport.js registrado con Fastify.

La primera ruta, /api/auth, es una ruta POST que autentica una cuenta mediante la estrategia local. Si la autenticación es correcta, la ruta utiliza el método signJWT en la cuenta autenticada para devolver un JWT firmado al cliente.

La segunda ruta, /api/test, es una ruta GET protegida por la estrategia JWT. El cliente debe enviar un token JWT válido en el encabezado de autorización . Si el token es válido y corresponde a un usuario del sistema, la ruta confirma que el usuario está...

Autenticada. De lo contrario, la solicitud se rechaza con una respuesta 401

No autorizada .

Ejemplo 1021. Definir rutas API adicionales en index.js

```
app.post( "/"
  api/auth",
  { preValidation: fastifyPassport.authenticate("local", { session: false }) }, async
(solicitud, respuesta) =>
  { const { usuario: cuenta } = solicitud;
    const token = cuenta.signJWT(); return
    reply.send({ token });

});

app.get( "/"
  api/test",
  { preValidation: fastifyPassport.authenticate("jwt", { session: false }) }, async
(solicitud, respuesta) =>
  { return reply.send({ estado:
    "Autenticado." });

});
```

- ➊ Define la ruta API /api/auth para el inicio de sesión de la cuenta y la emisión de JWT.
- ➋ Utilice la estrategia local para autenticar el nombre de usuario/contraseña de la solicitud.
- ➌ Utilice el método signJWT de la cuenta para generar un JWT.
- ➍ Envíe el token en una respuesta JSON.
- ➎ Defina la ruta /api/test para probar el acceso protegido por JWT.
- ➏ Utilice la estrategia jwt para validar el token enviado en el encabezado de la solicitud.
- ➐ Envíe un mensaje de confirmación si la autenticación es exitosa.

Ahora puedes probar esto directamente desde la línea de comandos.

Primero, autentícate con un usuario existente:

```
curl -X POST http://localhost:3000/api/auth \
  -H "Tipo de contenido: aplicación/json" \ -d
  '{"nombre de usuario":"<nombre de usuario>","contraseña":"<contraseña>"}
```

NOTA

Reemplace <nombre de usuario> y <contraseña> con sus credenciales reales.

Si tiene éxito, la respuesta incluirá un token:

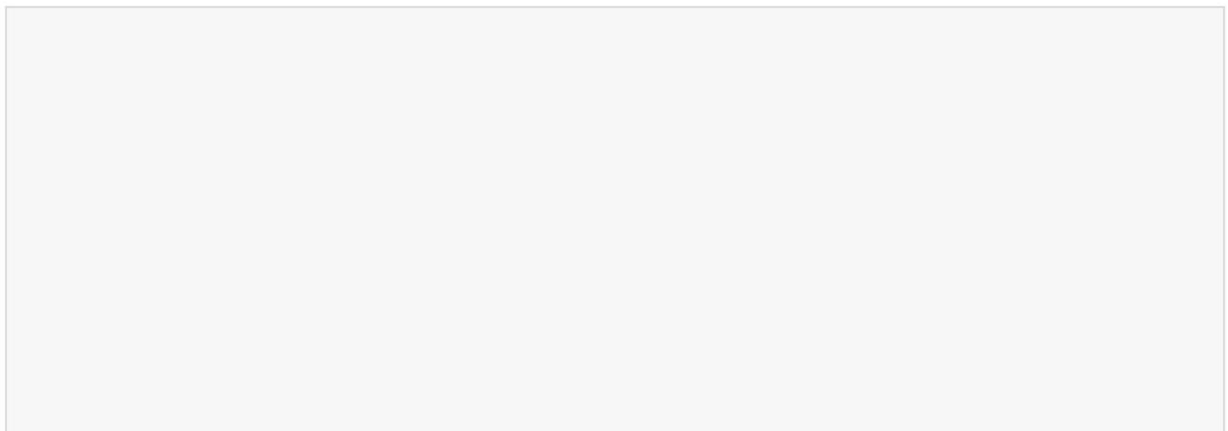
```
{ "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..." }
```

A continuación, utilice ese token para acceder a la ruta protegida:

```
curl http://localhost:3000/api/test \
  -H "Aceptar: aplicación/json" \
  -H "Autorización: Portador <su_token_aquí>"
```

Si el token es válido, recibirás { "status": "Authenticated." }.

Ahora ha ampliado con éxito su aplicación para admitir la autenticación de API sin estado con JWT.



EJERCICIOS DEL CAPÍTULO

1. Agregar funcionalidad de cierre de sesión para usuarios que inician sesión según la sesión:

- a. Defina una ruta GET/logout .
- b. En el controlador de ruta, llame a `req.logout()` y `reply.redirect("/")`.
- c. Actualice su página (o vista) de inicio de sesión exitoso para mostrar un enlace de cierre de sesión cuando el usuario inicie sesión.
- d. Pruébalo iniciando sesión, haciendo clic en “Cerrar sesión” y asegurándose de que ya no pueda acceder a páginas protegidas.

CONSEJO

Un buen flujo de autenticación siempre incluye una forma para que los usuarios finalicen manualmente su sesión.

2. Restringir el acceso a la página a usuarios autenticados:

- a. Cree una vista llamada `dashboard.hbs` con un mensaje de bienvenida.
- b. En el `index.js`, defina una ruta GET/dashboard .
- c. En el controlador de ruta, utilice `req.isAuthenticated()` y, condicionalmente, responda `redirect("/?page=login")` o visualice el panel.
- d. Actualice su lógica de inicio de sesión exitoso para redirigir a los usuarios a /tablero de mando.

Esto agrega control de acceso basado en vista, esencial para aplicaciones web con sesión protegida.

Resumen

En este capítulo, usted:

- Se exploraron vulnerabilidades de seguridad comunes en las aplicaciones Node y cómo implementar una autenticación adecuada es responsabilidad del desarrollador.
- Se utilizó el módulo criptográfico integrado de Node para codificar de forma segura las contraseñas mediante sales y algoritmos sólidos.
- Passport.js integrado con Fastify para gestionar estrategias de autenticación en una arquitectura modular basada en complementos
- Se implementó la autenticación basada en sesión usando cookies y `@fastify/session` para mantener el estado de inicio de sesión para los usuarios del navegador.
- Se agregó autenticación basada en token sin estado mediante JWT para admitir aplicaciones móviles y clientes API que no usan cookies.

Capítulo 11. Administrador de pedidos de café

Este capítulo cubre lo siguiente:

- Diseño de un sistema de colas
- Uso de Redis como sistema de gestión de colas en memoria
- Uso de RabbitMQ para colas más avanzadas y escaladas

En este capítulo, creará un par de aplicaciones Node que utilizan colas para gestionar cargas de trabajo y mejorar la escalabilidad. Si bien Node se ha adoptado ampliamente en diversos sectores por su eficiencia, presenta dificultades al gestionar aplicaciones a gran escala. El bucle de eventos de Node le permite procesar las solicitudes entrantes de forma asíncrona, delegando tareas a otros procesos para garantizar un funcionamiento fluido. Sin embargo, a medida que aumenta el número de solicitudes, o si las tareas consumen mucha CPU o son síncronas, el bucle de eventos puede convertirse en un cuello de botella, retrasando las solicitudes. Las colas abordan este desafío descargando tareas, organizándolas para su procesamiento secuencial o paralelo y garantizando que los procesos críticos no se saturen. Al usar colas, puede equilibrar las cargas de trabajo, mejorar los tiempos de respuesta y mantener la confiabilidad de sus aplicaciones Node, incluso con alta demanda.

Una solución arquitectónica a este problema es introducir un sistema de colas para gestionar las solicitudes y las tareas. Así como el bucle de eventos utiliza colas primitivas para añadir nuevas tareas y detectar eventos de finalización, las colas más complejas ayudan a organizar el flujo de tareas en una aplicación, garantizando que ningún segmento del proceso impida que los datos lleguen a su destino final.

En este capítulo, explorarás cómo usar una cola primitiva para gestionar tareas en una aplicación Node. Luego, usarás Redis como...

Un sistema de mensajería de publicación/suscripción para descargar tu lógica a su estructura probada. Por último, usarás un sistema de colas popular, RabbitMQ, para respaldar tu aplicación y lograr una escala aún mayor. Al final de este capítulo, comprenderás mejor cuándo considerar las colas de mensajería y cuándo te resultarían engorrosas.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en [el Capítulo 1](#), mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en [el Apéndice A](#). Las instrucciones para instalar y usar Postman se encuentran en [el Apéndice B](#). Para una explicación más detallada de SQLite, así como los pasos de instalación de Redis y RabbitMQ, consulte [el Apéndice C](#). Una vez completado, vuelva aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, lo que le brinda mayor control y flexibilidad a medida que avanza.

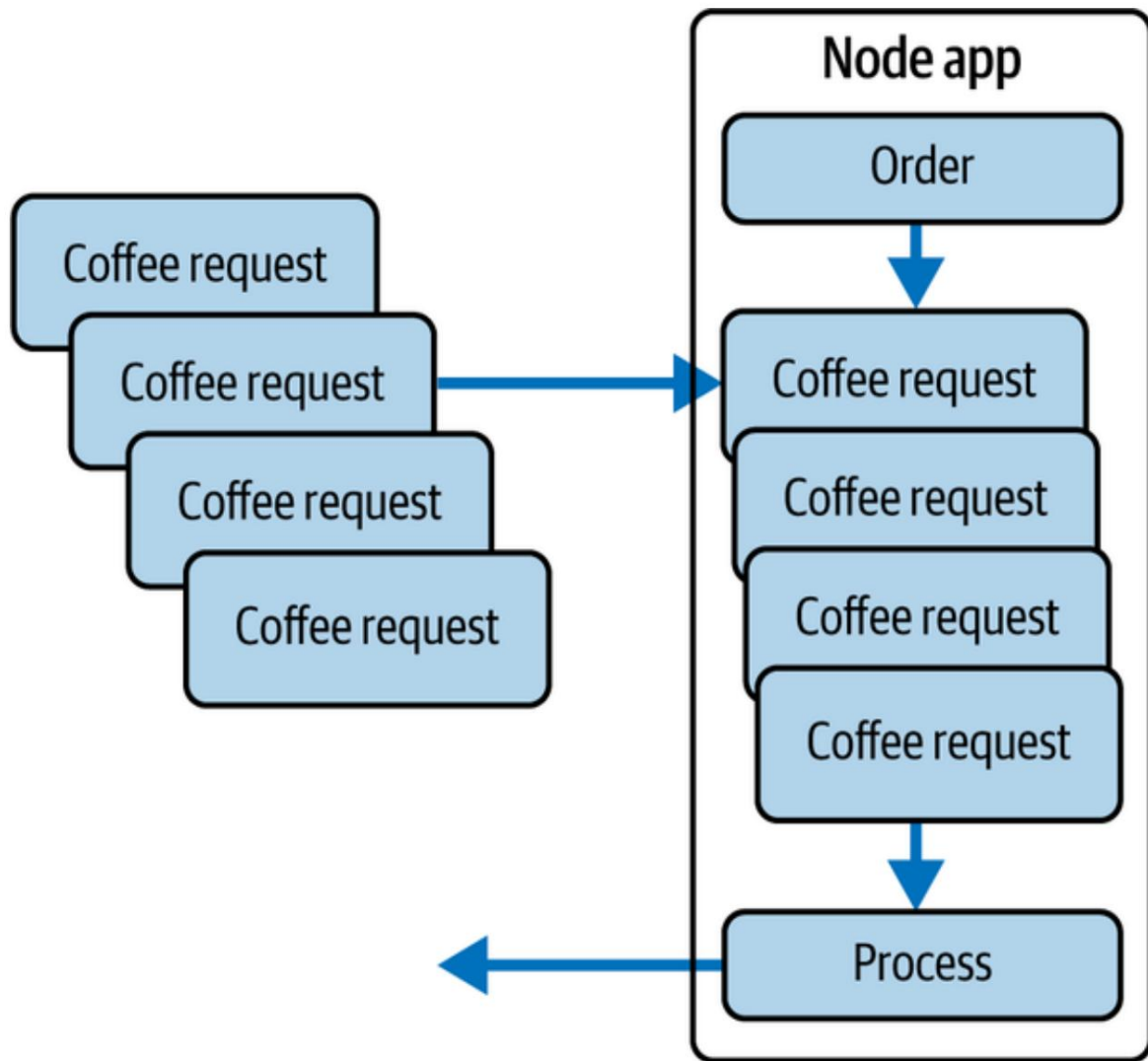
Tu mensaje: Una

empresa de entrega de café en línea llamada JavaShipped ha ganado popularidad, con una afluencia masiva de pedidos de café en línea que saturan los servidores centrales. El equipo de JavaShipped necesita tu ayuda para implementar un sistema que gestione el aumento de escala. Sugieren un sistema de mensajería para gestionar las solicitudes de pedidos entrantes y te han encomendado la tarea de construir dicho sistema con Node.

Obtener planificación

El equipo de JavaShipped está satisfecho con su plataforma de pedidos de café y su proceso de cumplimiento. Sin embargo, su aplicación Node actual presenta un cuello de botella que impide nuevos pedidos después de cierto límite. También desean mejorar su aplicación para gestionar mejor la comprobación del inventario y asegurarse de que haya suficiente café para entregar antes de aceptar pedidos, así como para procesar análisis internos del producto en su conjunto.

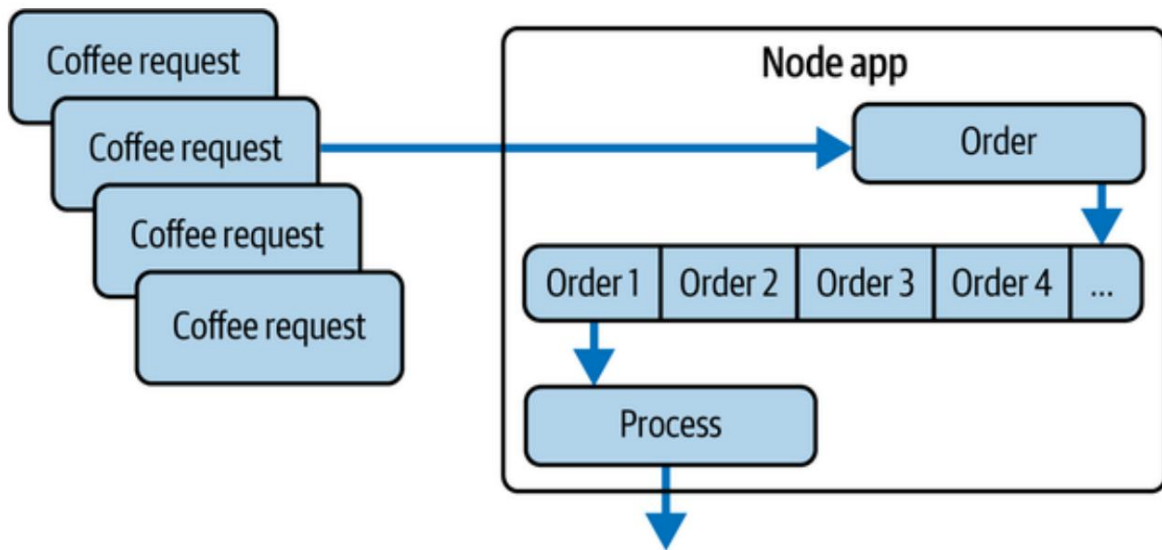
El problema se visualiza en **la Figura 11-1** mostrando las solicitudes entrantes que llegan a un único punto final del servidor de aplicaciones. En esta figura, el punto final `/order` es el principal punto de entrada para los nuevos pedidos de café. Si bien Node suele ser rápido al procesar millones de solicitudes web por minuto, la lógica subyacente de la aplicación puede ser demasiado lenta para procesar suficientes pedidos entrantes de forma oportuna. Debido a esto, la respuesta de la aplicación podría agotar el tiempo de espera o, peor aún, podría bloquearse por completo.



Cifra 111. Estructura de aplicación JavaShipped existente con un cuello de botella de solicitud

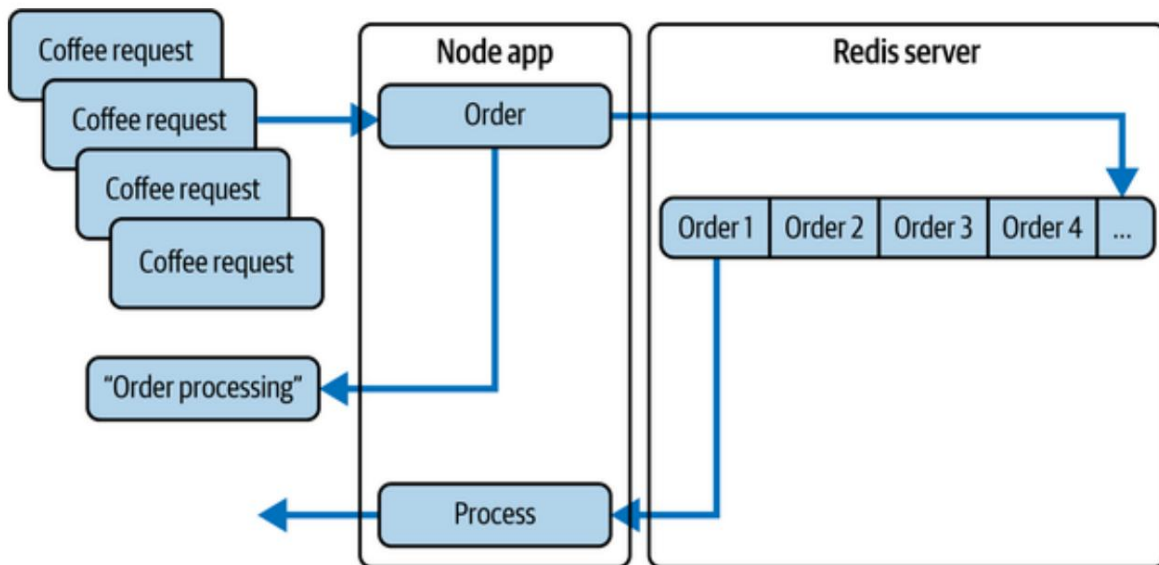
Para mejorar este diseño, comience diagramando una estructura similar que depende de una cola interna para manejar el procesamiento de pedidos de bebidas.

La figura 11-2 demuestra cómo se puede usar una cola para almacenar en búfer solicitudes entrantes, dando a la lógica de procesamiento de pedidos un lugar desde donde extraer nuevas solicitudes de pedidos. Si bien Node tiene su propio sistema interno de cola que se comporta de manera similar, esta cola explícita podría ayudar. Los ingenieros determinan una nueva lógica para gestionar mejor los pedidos, o incluso Notificar a los usuarios finales cuando el sistema está sobrecargado o cuando hay nuevos. Ya no se aceptan pedidos.



Cifra 112. Agregar una cola interna para gestionar nuevas solicitudes de pedidos

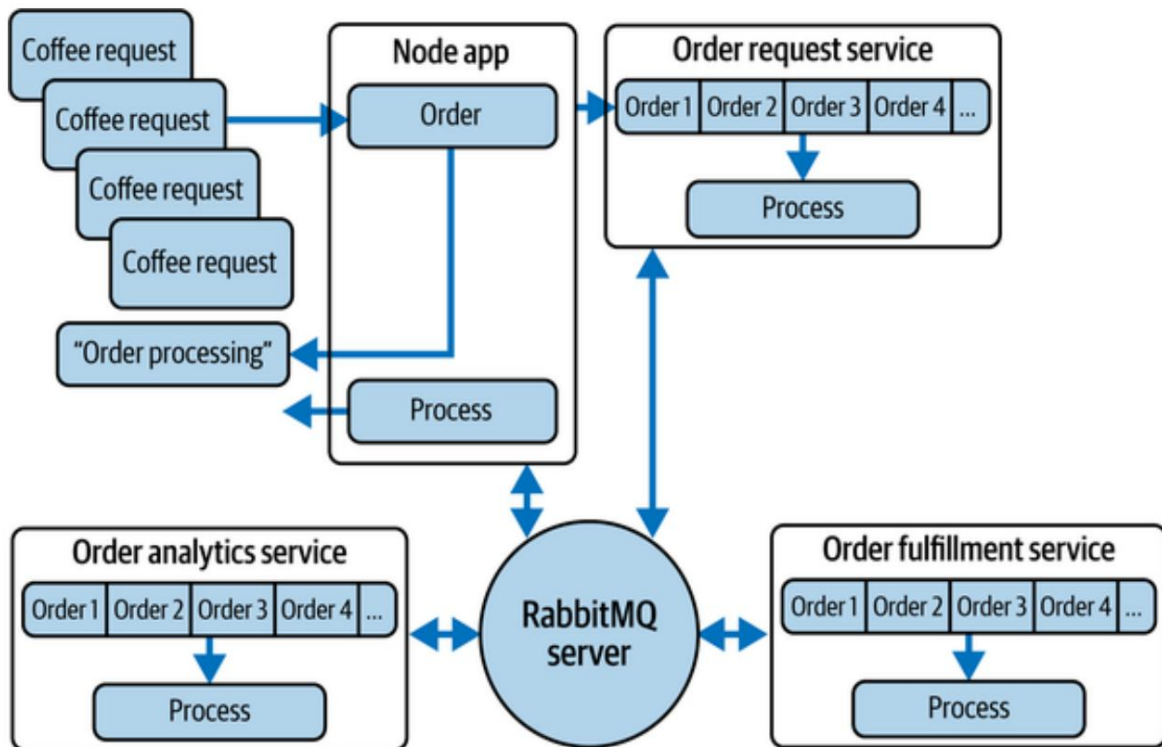
Este enfoque es suficiente para proyectos más pequeños, pero pronto...
 descubre que no escala bien. Peor aún, tu cola todavía está en el
 la memoria interna de la aplicación. Esto significa que si la aplicación falla
 Antes de agregar un pedido al almacenamiento persistente, todos sus pedidos más recientes
 Las solicitudes de pedidos desaparecerán. Por eso, diagrama un
 Sistema mejorado con su cola almacenada en un servidor separado.
 La Figura 11-3 muestra su aplicación junto con un servidor Redis.
 El papel de Redis aquí es gestionar el almacenamiento temporal de su pedido.
 solicitudes a medida que se procesan. Si su aplicación falla, su
 Los pedidos permanecen intactos en su almacenamiento Redis (más sobre esto más adelante en el
 capítulo).



Cifra 113. Utilizando un servidor Redis para gestionar solicitudes de pedidos

Presenta su solución a JavaShipped y les recuerda que la escalabilidad es especialmente importante a medida que crece su imperio de entrega de café. Explica que la solución Redis es excelente, pero que existen formas más robustas de gestionar las solicitudes de pedidos a nivel global. Presenta la arquitectura de colas de mensajes de RabbitMQ en la Figura 11-4 , que separa parte de la lógica de la aplicación en servicios independientes. Estos servicios dependen de un servidor RabbitMQ para escuchar las tareas relevantes y solo las emiten al servicio cuando están disponibles. De esta forma, su aplicación puede descargar tanto el sistema de colas como parte de la lógica de procesamiento de la aplicación a servicios independientes. Dado que estos servicios también son aplicaciones Node, pueden alojarse en entornos aislados y seguir ejecutándose incluso si el servidor Node principal falla.

Una vez completada la demostración, recibirás luz verde de JavaShipped para comenzar a desarrollar una aplicación Node mejorada para manejar el problema de escalabilidad de la empresa.



Cifra 114. Actualización de la estructura de la aplicación con una cola de mensajes de RabbitMQ servidor entre servicios

Obtenga programación

Para comenzar a desarrollar su aplicación, cree una nueva carpeta de proyecto llamada `coffee_queue`. Navegue a la carpeta de su proyecto con su comando `cd` y ejecute `npm init`. Este comando inicializa su aplicación Node. Puede presionar Enter durante los pasos de inicialización para aceptar el valores predeterminados. El resultado de estos pasos es la creación de un archivo llamado `package.json`. Este archivo le indicará a su aplicación Node cualquier configuraciones o scripts necesarios para ejecutarse correctamente.

A continuación, crea un archivo llamado `index.js` dentro de la carpeta de tu proyecto. Este archivo es El punto de entrada para su aplicación.

Con el archivo `index.js` listo, puedes instalar el servidor principal Marco utilizado en este proyecto. Vaya a la raíz de su proyecto. carpeta en su línea de comandos y ejecute `npm install fastify@^5.4.0 @fastify/formbody@^8.0.2` para instalar

Fastify como framework para tu aplicación web. Luego, agrega el código del **Ejemplo 11-1**, que configura un servidor HTTP simple con Fastify. Primero, importas Fastify y creas una instancia del servidor Fastify, asignándola a la variable `app`. Defines la variable constante `PORT` con el valor 3000. Este es el número de puerto en el que el servidor escuchará las solicitudes entrantes.

NOTA

La variable de la aplicación se utiliza para configurar el servidor y definir las rutas.

Registra un complemento que analiza las solicitudes entrantes con cargas útiles codificadas en URL. Este complemento se utiliza para gestionar los datos de formulario enviados en solicitudes POST. Fastify gestiona las solicitudes JSON de forma nativa, por lo que no requiere middleware adicional. Por último, escribe el código que inicia el servidor a la escucha en el puerto 3000 y registra un mensaje en la consola una vez que el servidor está en ejecución.

NOTA

El método `listen()` utiliza una función de devolución de llamada que se ejecuta cuando el servidor empieza a escuchar las solicitudes entrantes. En este caso, registra un mensaje en la consola que indica que el servidor se está ejecutando en el puerto especificado.

Ejemplo Configuración de su aplicación en `index.js` `import Fastify from "fastify";`

`import formbody from "@fastify/formbody";`

`const aplicación = Fastify();`

`const PUERTO = 3000;`

`esperar aplicación.register(formbody);`

1

2

3

4

5

```
app.post("/slow-order", async (request, reply) => { const { drinkOrder }  
  = request.body; for (let i = 0; i < 10000000000; i+  
  +) {} console.log("PEDIDO REALIZADO"); reply.send(  
  Pedido de bebida agregado a la cola:  
  ${drinkOrder}`); });
```

```
await app.listen({ port: PORT });  
console.log(`Servidor escuchando en http://localhost:${PORT}`);
```

12

- 1 Importa Fastify a tu aplicación.
- 2 Imprime el complemento para manejar cuerpos codificados por formulario.
- 3 Cree una instancia de servidor Fastify.
- 4 Defina un valor de puerto para las solicitudes entrantes.
- 5 Registre el complemento formbody para gestionar envíos de formularios codificados en URL.
- 6 Agregue una ruta POST para gestionar los pedidos de bebidas entrantes.
- 7 Desestructurar drinkOrder del cuerpo de la solicitud entrante.
- 8 Simular una operación de bloqueo con un bucle largo.
- 9 Registra cuando se realiza un pedido.
- 10 Enviar una respuesta con la confirmación del pedido.
- 11 Espere a que el servidor Fastify comience a escuchar (el nodo 18+ admite espera de nivel superior).
- 12 Registre la dirección del servidor para confirmar que está funcionando.

Antes de ejecutar esta aplicación, sería útil agregar un punto final al que se puedan enviar solicitudes para su procesamiento. Porque esto...

El proyecto está lidiando con problemas de escalabilidad debido a los largos tiempos de procesamiento de pedidos, usted crea una ruta para imitar el tiempo de espera de un pedido realizado. Este código define un controlador de ruta para el método HTTP POST en la ruta / slow-order. De la solicitud, se extrae la propiedad drinkOrder del objeto request.body , que contiene los datos enviados en el cuerpo de la solicitud.

CONSEJO

El objeto request.body se rellena mediante la compatibilidad integrada de Fastify con JSON y el complemento formbody registrado. Asegúrese de que este complemento esté registrado antes de la ruta para garantizar que el contenido del cuerpo esté disponible.

La siguiente línea crea un bucle que se itera 10 mil millones de veces. Este bucle simula un proceso de larga duración que podría ralentizar el tiempo de respuesta del servidor. En este caso, el bucle simplemente desperdicia ciclos de CPU e impide el procesamiento de otras solicitudes para simular un pedido procesado por JavaShipped. Registra un mensaje en la consola indicando que se ha realizado un pedido, una vez finalizada la ejecución del bucle. Por último, responde al cliente con un mensaje indicando que su pedido de bebida se ha añadido a la cola interna.

Puede ejecutar esta aplicación accediendo al directorio raíz de su proyecto en la línea de comandos y ejecutando node index. Al iniciar la aplicación, verá un mensaje de registro que indica que el servidor está escuchando en <http://localhost:3000> . Ahora puede publicar contenido en su aplicación. Sin una interfaz web, puede publicar un pedido de bebidas abriendo una nueva ventana de línea de comandos junto a su servidor en ejecución y ejecutando el siguiente comando cURL:

```
curl -X POST -H "Tipo de contenido: aplicación/json" \
  -d '{"drinkOrder":"café con leche"}' http://localhost:3000/slow-order
```

Tenga en cuenta que ejecutar este comando tarda aproximadamente 10 segundos antes de que se registre en su consola la respuesta " Pedido de bebida añadido a la cola: latte" . Si un pedido de bebida normal tarda tanto tiempo, puede ser un problema para los nuevos pedidos entrantes. Para abordar este problema, utilice un enfoque sencillo: cree una cola simple para gestionar las nuevas solicitudes de pedido.

Debido a que JavaScript no admite de forma nativa la estructura de datos de cola, puede simular una utilizando los métodos Array integrados, como `push()` y `shift()`, para agregar y eliminar elementos en el orden de entrada, salida (FIFO). (JavaScript, colas en Este proyecto utiliza una matriz simple para administrar la cola de pedidos de bebidas. Para necesidades más complejas, como tiempos de espera de trabajos, reintentos o simultaneidad, puede considerar una biblioteca dedicada como el paquete `queue` en npm, que proporciona una implementación de cola más sólida.

COLAS EN JAVASCRIPT

Aunque JavaScript no cuenta con una implementación de cola integrada, es posible implementar una estructura de datos de cola mediante matrices o listas enlazadas. JavaScript se diseñó originalmente como lenguaje de scripting para la web, donde las colas podrían no ser necesarias con tanta frecuencia como en otros tipos de aplicaciones. JavaScript también se diseñó para ser ligero y fácil de usar, y la inclusión de un conjunto completo de estructuras de datos puede haberlo hecho más complejo. Métodos de matriz como `push`, `shift` y `slice` permiten implementar una estructura de datos de cola mediante matrices. Ha surgido una demanda de estructuras de datos más robustas, como las colas, en JavaScript, lo que ha impulsado el desarrollo de bibliotecas y paquetes de terceros.

El paquete npm de colas es una implementación sencilla de una estructura de datos de colas para JavaScript. Una cola es una colección de elementos que se pueden añadir al final y eliminar del frente según el orden FIFO (primero en entrar, primero en salir). Puede usar el paquete de colas como una API sencilla para crear y manipular colas.

Los siguientes son algunos métodos que puede utilizar con una instancia de Queue :

`poner en cola(elemento)`

Agrega un elemento al final de la cola.

`desencolar()`

Elimina el elemento que está al frente de la cola y lo devuelve.

`ojeada()`

Devuelve el elemento al principio de la cola sin eliminarlo.

`estáVacío()`

Devuelve verdadero si la cola está vacía, falso en caso contrario

`longitud()`

Devuelve el número de elementos en la cola.

`aArray()`

Devuelve una matriz que contiene los elementos en la cola, en el orden en que se agregaron

Para obtener más información sobre el paquete de cola , visite el sitio web [de npm](#) .

A continuación, se añaden dos nuevas rutas del **Ejemplo 11-2** después de la ruta `/slow-order` en `index.js`. Estos puntos finales gestionan la adición de nuevos pedidos de bebidas y el procesamiento de pedidos desde `coffeeQueue`. La primera ruta recibe solicitudes HTTP POST a la ruta URL `/order` . Cuando se recibe una solicitud, los datos de `drinkOrder` se extraen del cuerpo de la solicitud mediante una asignación de desestructuración y se insertan en una cola `coffeeQueue` . La longitud de la cola se registra en la consola y se envía una respuesta al cliente con el mensaje "Pedido de bebida añadido a la cola".

El segundo punto final recibe solicitudes HTTP GET a la ruta URL `/process-order` . Al recibir una solicitud, el siguiente pedido de bebida se retira de la cola de `coffeeQueue` mediante el método `shift` . Si hay un pedido de bebida en la cola, se devuelve al cliente como un objeto JSON mediante el método `reply.send()` de Fastify . Si no hay pedidos de bebida en la cola, se envía al cliente el mensaje "No hay pedidos de bebida en la cola" .

NOTA

Puede agregar el bucle for de la ruta /slow-order a su ruta /process-order . Este bucle seguirá bloqueando nuevas solicitudes a la ruta /order . Sin embargo, la ventaja de la cola es que la lógica de procesamiento se puede extraer a un servicio independiente, por lo que su aplicación principal de Node no necesita dedicar tiempo a las solicitudes entrantes ni al procesamiento de pedidos.

Ejemplo 112. Agregar rutas para realizar pedidos en su cola en index.js

```
app.post("/order", async (request, reply) => { const { drinkOrder } ❶
  = request.body; coffeeQueue.push(drinkOrder);
  console.log(coffeeQueue.length); ❷
  reply.send(" Pedido de bebida agregado a la ❸
    cola"); }); ❹

app.get("/procesar-pedido", async (solicitud, respuesta) => { const nextOrder ❺
  = coffeeQueue.shift(); if (nextOrder) { ❻
    responder.enviar({ order: nextOrder }); } else ❼
    { responder.enviar("No hay pedidos de bebidas en la cola"); ❽
  } });
```

- ❶ Define una ruta POST para la ruta /order usando Fastify.
- ❷ Añade drinkOrder a tu objeto de cola coffeeQueue .
- ❸ Registra el número de elementos en la cola.
- ❹ Responder con un mensaje al cliente indicando que se ha añadido el pedido.
- ❺ Defina una ruta GET para la ruta /process-order .
- ❻ Extraiga un artículo de la cola y asígnelo a nextOrder.

- 7 Si hay un elemento en la cola, devuelve ese valor usando `reply.send()`.
- 8 Si no hay ningún elemento en la cola, devuelve un mensaje al cliente.

Puede ejecutar esta aplicación volviendo a ejecutar el índice del nodo en la línea de comandos. `curl -X POST -H "Content-Type: application/json" -d '{"drinkOrder":"latte"}' http://localhost:3000/order` añade un nuevo pedido de café con leche a su cola. Ahora, al ejecutar el comando `curl http://localhost:3000/process-order`, verá el primer pedido de bebida devuelto desde su cola registrado en su consola.

A medida que llegan los pedidos, se añaden a la cola y, a medida que se procesan, se retiran de ella. Este enfoque es útil para gestionar grandes volúmenes de solicitudes de forma sistemática y eficiente, garantizando que los pedidos se procesen en el orden en que se recibieron.

Puede ampliar este enfoque añadiendo nuevas rutas, como el punto final `/order-count` del [Ejemplo 11-3](#). Esta ruta simplemente comprueba el número de elementos en la cola y registra ese valor en un mensaje en la consola.

Ejemplo: ¹¹³ Agregar un punto final de conteo de pedidos de cola en `index.js`

```
app.get("/order-count", async (request, reply) => {
  reply.send(`${coffeeQueue.length} pedidos de bebidas en cola`);
});
```

- 1 Defina una ruta GET para la ruta `/order-count` usando Fastify.
- 2 Devuelve un mensaje con la longitud de la cola al cliente usando `reply.send()`.

Intenta ejecutar tu aplicación con esta nueva ruta, agregando dos pedidos de bebidas y ejecutando `curl http://localhost:3000/order-count` en otra ventana de línea de comandos. Verás dos pedidos de bebidas en cola. Ejecuta `curl http://`

`localhost:3000/process-order` y vuelve

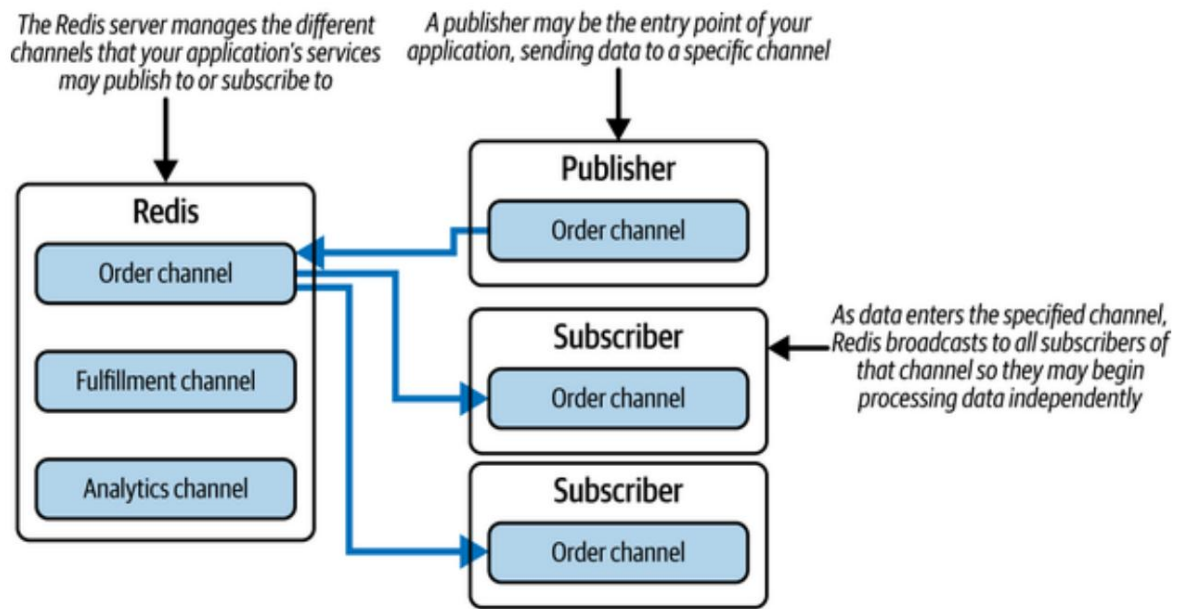
a comprobar el número de pedidos para ver si ha cambiado. De esta forma, tu sistema de colas ya ayuda a mantener el estado de los pedidos de tu proyecto.

Sin embargo, a pesar de la mejora, aún debe procesar manualmente los pedidos de su cola a través de un endpoint. Además, si su aplicación falla, su cola, con todos sus pedidos, desaparece. En la siguiente sección, solucionará este problema añadiendo un servidor Redis para dar soporte a su aplicación.

Añadiendo un servidor Redis. Su

aplicación está empezando a tomar forma estructural para gestionar los pedidos de bebidas a un ritmo cómodo para su negocio. Sin embargo, una cola por sí sola aún requiere una función para procesar manualmente los pedidos de bebidas uno a uno. Para un negocio con un número creciente de solicitudes, su aplicación se beneficiaría de mensajería en tiempo real, una cola que gestione la escalabilidad y una lógica desacoplada de su servidor Node principal.

Una solución es implementar Redis como un patrón de mensajería de publicación/suscripción (pub/sub) entre su aplicación y los datos de pedidos de bebidas. Redis Pub/Sub permite la mensajería en tiempo real entre clientes y servidores. Esto significa que cualquier actualización o cambio realizado en el servidor se puede enviar inmediatamente a los clientes suscritos a canales específicos, sin necesidad de que los clientes consulten constantemente el servidor para obtener actualizaciones. Estos canales pueden tener nombres personalizados. **La Figura 11-5** muestra cómo una parte de su aplicación puede publicar datos en Redis y, a su vez, Redis los transmitirá en ese canal específico a los servicios suscritos.



Cifra 115. Redis transmite a los suscriptores de su canal

Esta solución está diseñada para manejar grandes volúmenes de mensajes y Se puede escalar fácilmente para adaptarse a más clientes y canales. Esto puede ayudar a mejorar el rendimiento y la confiabilidad de su aplicación, especialmente a medida que crece el número de clientes y canales. Por último, la lógica de la aplicación permite que diferentes partes del sistema... comunicarse entre sí a través de canales sin necesidad de conexiones directas o dependencias en su aplicación principal Fastify.

MÁS SOBRE REDIS

Redis es un almacén de datos en memoria que se utiliza comúnmente para el almacenamiento en caché y el procesamiento de datos en tiempo real. Al ejecutarse en su propio servidor, permite almacenar diversas estructuras de datos, como cadenas, hashes, listas y conjuntos, fuera de la aplicación principal. En una aplicación Node, Redis se utiliza a menudo para almacenar en caché datos solicitados previamente, claves de autenticación de usuario para la actividad reciente de la cuenta y, en definitiva, cualquier otro dato al que se acceda con frecuencia. Su sistema de gestión de datos es una opción popular en prácticamente todas las plataformas técnicas actuales.

Usar Redis como almacén de datos ayuda a reducir la carga de la base de datos y a mejorar el rendimiento y la velocidad de la aplicación. También permite implementar la limitación de velocidad en la aplicación, lo que ayuda a prevenir el abuso y a mejorar la fiabilidad del servicio. El sistema de mensajería Redis Pub/Sub admite mensajería en tiempo real y arquitecturas basadas en eventos en la aplicación.

Con pub/sub, puedes publicar mensajes en canales y los suscriptores pueden recibir esos mensajes en tiempo real.

Para su proyecto, utilizará la funcionalidad de mensajería en tiempo real a través de los siguientes tres métodos:

`publicar(nombreCanal, mensaje)`

Publica un mensaje en un canal de Redis. El argumento "channelName" es una cadena que representa el nombre del canal y el argumento "message" es el mensaje que se publicará.

Cuando se publica un mensaje en un canal, Redis distribuirá el mensaje a todos los clientes que estén suscritos a ese canal.

`suscribirse(nombreCanal)`

Se suscribe a un canal de Redis. El argumento channelName es una cadena que representa el nombre del canal. Cuando un

Cuando un cliente se suscribe a un canal, Redis lo añade a la lista de suscriptores de ese canal. El cliente recibirá los mensajes publicados en ese canal.

`on(evento, devolución de llamada)`

Registra un detector de eventos para un cliente Redis. El argumento "evento" es una cadena que representa el nombre del evento que se va a detectar, y el argumento "devolución de llamada" es una función que se llamará cuando se produzca el evento. El evento "mensaje" es un evento común para los clientes Redis y se activa al recibir un mensaje en un canal suscrito.

Cuando ocurre un evento de mensaje, se llamará a la función de devolución de llamada con dos argumentos: el nombre del canal donde se recibió el mensaje y el mensaje en sí.

Después de instalar Redis, puede crear un nuevo cliente de Redis y usar estos métodos para enviar datos rápidamente a diferentes partes de su aplicación. En Node 18+, con módulos ES, puede importar Redis de la siguiente manera:

```
importar { createClient } desde 'redis';

const pub = crearCliente(); const sub
= crearCliente();

esperar pub.connect();
esperar sub.connect();
```

Luego usa:

```
await pub.publish('pedidos', 'latte'); await
sub.subscribe('pedidos', (mensaje) => { console.log(` Pedido
recibido: ${message}`); });
```

Para obtener más información sobre Redis Pub/Sub, consulte la documentación [de Redis](#).

Para integrar Redis en su aplicación, instale el paquete npm de Redis ejecutando `npm install redis@^5.6.1` en la raíz del directorio de su proyecto desde la línea de comandos. A continuación, importe ese paquete añadiendo `"import { createClient } from 'redis'"` al principio de su archivo `index.js`. A continuación, deberá crear dos clientes de Redis (uno para suscribirse y otro para publicar) mediante el método `createClient`. Después de crear sus clientes de Redis en `index.js`, añada el código del [Ejemplo 11-4](#) para configurar el suscriptor y el publicador de su sistema Redis Pub/Sub.

NOTA

Recuerda que Redis se ejecuta como un servidor independiente y debe ejecutarse junto con tu aplicación Fastify. Sigue las instrucciones del [Apéndice C](#) para iniciar tu servidor Redis.

Este código configura un cliente Redis para suscribirse y publicar, y luego se suscribe al canal de pedidos de bebidas mediante el cliente suscriptor. Para ambos clientes, se utiliza el método `connect()` para establecer una conexión con el servidor Redis.

`await subscriber.subscribe("drink-order", (drinkOrder) => {...})` se suscribe al canal `drink-order` y configura un receptor para los mensajes entrantes en ese canal. La función de devolución de llamada se activa al recibir un mensaje, y el argumento `drinkOrder` contendrá la carga útil del mensaje. En este ejemplo, la devolución de llamada registra un mensaje en la consola.

NOTA

Debido a que await se utiliza antes de los métodos connect() y subscribe() , el código debe escribirse utilizando la sintaxis async/await (compatible con Node 18+ con await de nivel superior).

Ejemplo 114. Agregar un Redis Pub / Subsistema en index.js

```

de importación { createClient } desde "redis";           ❶

const subscriber = createClient(); await                 ❷
subscriber.connect();

const editor = createClient(); await                    ❸
editor.connect();

esperar suscriptor.subscribe("pedido-de-bebida", (pedidoDeBebida) => {
❹
  console.log(` Se recibió un nuevo pedido ${drinkOrder} .`); }); Importe      ❺
el
❶ método de creación del cliente Redis.

❷ Cree un cliente Redis como suscriptor y conéctelo a su Redis
servidor.

❸ Cree un cliente Redis como editor y conéctelo a su Redis
servidor.

❹ Suscríbete al canal de pedidos de bebidas y escucha los mensajes.

❺ Registra la carga útil del mensaje cuando se recibe un nuevo pedido de bebida.

```

Luego, en su ruta /order , agregue la línea await

publisher.publish("drink-order", drinkOrder) debajo de la línea donde desestructura drinkOrder del cuerpo de la solicitud.

Esto publica el valor del pedido de bebida en el canal de pedido de bebida y notifica inmediatamente a todos los suscriptores.

Abra una nueva ventana de línea de comandos junto a su servidor Fastify en ejecución y ejecute este comando cURL:

```
curl -X POST -H "Tipo de contenido: aplicación/json" \  
  -d '{"bebidaOrden":"café con leche"}' http://localhost:3000/orden
```

Inmediatamente verás que se recibió un nuevo pedido de café con leche. Se registró en tu consola.

Dado que su servidor Redis se ejecuta en su propio puerto, puede crear tantas aplicaciones Node como necesite para comunicarse con él a través de canales. Por ejemplo, dentro del bloque `subscriber.subscribe`, podría agregar ``await publisher.publish("fulfill-order", "latte")`` para activar otra acción.

También puedes publicar datos estructurados utilizando JSON convertido en cadena:

```
esperar editor.publish("orden-de-bebida", JSON.stringify({ bebida: "latte", costo: 450,  
  cliente: "Jon Wexler" }));
```

El suscriptor entonces tendría que analizarlo:

```
await subscriber.subscribe("orden-de-bebida", (mensaje) => { const parsed =  
  JSON.parse(mensaje); console.log(`Se recibió $  
    {parsed.drink} para  
    ${parsed.customer}`); });
```

Con los cambios implementados, podrá crear un sistema distribuido de servicios que se comuniquen a través de Redis. Sin embargo, Redis tiene algunas limitaciones. Los mensajes no se almacenan de forma persistente por defecto; si un cliente se desconecta, perderá los mensajes. Redis también solo admite cargas útiles simples basadas en cadenas, y Pub/Sub opera en un solo hilo, lo que puede limitar el rendimiento.

En la siguiente sección, adaptará patrones de mensajería más sólidos para superar estas limitaciones.

Integración de un sistema de mensajería robusto. Su aplicación ahora puede gestionar una gran cantidad de mensajes a través de diversos canales personalizados. Redis Pub/Sub es un potente mecanismo para crear sistemas de comunicación en tiempo real. En su caso, se pueden añadir más canales para comunicarse con otros servicios sobre el procesamiento de datos a lo largo de una cadena de dependencias de tareas. Por ejemplo, se puede realizar un pedido, lo que activa un servicio de cumplimiento, que a su vez publica en otro canal, lo que activa un servicio de análisis. Dado que busca la escalabilidad y la fiabilidad para una empresa que no puede perder pedidos de clientes, decide utilizar un sistema de mensajería más avanzado.

Similar a su sistema de colas y Redis Pub/Sub, RabbitMQ ofrece un conjunto de funciones de mensajería que le permiten crear estructuras de mensajes más complejas, incluyendo enrutamiento, confirmación, durabilidad y colas persistentes. Estas funciones permiten que los mensajes se entreguen de forma fiable, se enruten según reglas y se confirmen o reintenten si fallan. Con RabbitMQ, puede mantener servicios aislados y enfocados que se comunican a través de una canalización interna, sin sobrecargar ningún proceso de Fastify ni ninguna aplicación de Node.

Para demostrar esto, comenzará a integrar RabbitMQ en su proyecto organizando su aplicación Node en tres servicios: el punto de entrada principal (que acepta pedidos), un servicio de cumplimiento (que los procesa) y un servicio de análisis (que los rastrea y registra).

NOTA

Asegúrese de que RabbitMQ esté instalado y en ejecución antes de ejecutar cualquier código.

Al igual que Redis, RabbitMQ se ejecuta como un servidor independiente con su propio puerto.

Consulte [el Apéndice C](#) para obtener instrucciones de configuración.

¿POR QUÉ RABBITMQ?

Una vez que haya decidido usar un sistema de colas robusto, es importante considerar las opciones disponibles y las alternativas que se utilizan en la industria. El sistema que elija dependerá de los requisitos y limitaciones específicos de su aplicación, como la escalabilidad, la durabilidad, las garantías de los mensajes y la complejidad operativa.

Las siguientes son opciones comunes:

RabbitMQ

Un popular agente de mensajes de código abierto compatible con múltiples protocolos de mensajería (como AMQP). Es fiable, está bien documentado y admite enrutamiento, colas, persistencia y acuses de recibo.

ZeroMQ

Una biblioteca de mensajería ligera diseñada para aplicaciones de alto rendimiento. A diferencia de RabbitMQ, no requiere un bróker y se centra en la velocidad y la simplicidad en aplicaciones distribuidas.

Kafka

Una plataforma distribuida de transmisión de eventos de alto rendimiento. Kafka es ideal para casos de uso que implican la ingesta y el análisis en tiempo real de flujos de datos a gran escala.

Amazon SQS

Una cola de mensajes escalable y totalmente administrada por AWS. Es duradera y fácil de escalar, pero implica dependencia de un proveedor de nube y posibles desventajas en cuanto a latencia.

Utilizamos RabbitMQ para su proyecto porque es de código abierto, fácil de configurar localmente y es ideal para escalar sistemas de producción.

RabbitMQ le permite desacoplar servicios, descargar el procesamiento de su servidor Fastify y establecer una comunicación asíncrona entre servicios, todo lo cual admite tolerancia a fallas y escalabilidad a largo plazo.

Al evaluar estas herramientas, tenga en cuenta sus objetivos, presupuesto y tolerancia a la complejidad. RabbitMQ ofrece un equilibrio perfecto para aplicaciones de mediana a gran escala que necesitan fiabilidad sin grandes dependencias de la nube. Para más información, visite el [sitio web de RabbitMQ](#).

Para comenzar a usar RabbitMQ, instale el paquete necesario ejecutando `npm install amqplib@^0.10.8` en la raíz de su proyecto desde la línea de comandos. Dado que está compilando varios servicios, puede comenzar inicializando las carpetas de aplicación independientes para cada uno de ellos. Estas carpetas de proyecto pueden coexistir con la carpeta principal de su proyecto. Inicialice dos nuevas aplicaciones de Node llamadas `fulfillment_service` y `analytics_service`, instale los paquetes `npm fastify` y `amqplib`, y asegúrese de que el código del [Ejemplo 11-5](#) se encuentre en la parte superior del archivo `index.js` de cada aplicación.

En esta lista, importa el marco Fastify y la biblioteca `amqplib`, que se utiliza para interactuar con el agente de mensajes RabbitMQ.

Después de crear una nueva instancia de Fastify, registra los analizadores de cuerpo integrados de Fastify para manejar las solicitudes entrantes codificadas en URL y JSON. También define el número de puerto en el que se ejecutará el servidor mediante `process.env.PORT || 3000`. La última línea declara dos variables, `canal` y `conexión`, que se usarán para interactuar con el agente de mensajes de RabbitMQ. Finalmente, la aplicación usa la función `"await"` de nivel superior para empezar a escuchar en el puerto designado.

Ejemplo 115. Agregar configuraciones de proyectos de Fastify y RabbitMQ en `index.js`

```
importar Fastify desde "fastify"; importar
amqp desde "amqplib";
```

❶

```
constante aplicación = Fastify();
```

❷

```
const PORT = process.env.PORT || 3000; let canal, conexión;
```

```
await app.listen({ port: PORT });
console.log("Servidor ejecutándose en http://localhost:" + PORT);
```

- ➊ Importe las bibliotecas fastify y amqpplib a su proyecto.
- ➋ Cree una instancia de aplicación Fastify.
- ➌ Defina el número de PUERTO en el que tu servidor escuchará las solicitudes.
- ➍ Declare las variables de canal y conexión para RabbitMQ.
- ➎ Inicie su servidor Fastify utilizando await de nivel superior.

Como este código se aplica a la aplicación principal y a los servicios adicionales, es necesario cambiar el valor de PORT a valores diferentes para cada proyecto para que puedan ejecutarse simultáneamente. Actualice el valor de PORT predeterminado para fulfillment_service a 3001 y a 3002 para analytics_service. En este punto, debería tener tres carpetas de proyecto, cada una con su propio archivo indexjs y valores de PORT definidos por separado .

NOTA

RabbitMQ no requiere un framework de servidor web como Fastify, pero usar Fastify permite que cada servicio exponga sus propias rutas y endpoints, lo que puede ser útil para depuración, comprobaciones de estado o ampliación de funcionalidades. Con el servidor RabbitMQ en funcionamiento, cada aplicación debe conectarse a él para comunicarse a través de la red de aplicaciones. Agregue el código del [Ejemplo 11-6](#) al archivo index.js de cada una de sus tres aplicaciones Node basadas en Fastify. Este código describe los pasos a seguir:

1. Cree una función asincrónica denominada `connect`.
2. Utilice `amqp.connect("amqp://localhost:5672")` para Establezca una conexión con un servidor RabbitMQ que se ejecuta localmente en el puerto predeterminado (5672). Asigne la conexión a la variable `connection` . El método `amqp.connect()` devuelve una promesa, así que use `await` para esperar a que se establezca la conexión.
3. Llame a `connection.createChannel()` para crear un nuevo Canal en el servidor RabbitMQ. Asigne el canal a la variable `"canal"` . Use `"await"` para asegurarse de que el canal esté listo antes de continuar.

NOTA

Los canales se utilizan para enviar y recibir mensajes entre clientes y servidores. Dado que la conexión a otro servidor es un proceso que tarda un tiempo indeterminado, cada una de estas líneas utiliza `"await"` para garantizar que no se ejecuten más tareas antes de establecer la conexión con RabbitMQ.

4. Llame a `channel.assertQueue("drink-order")` para verificar que existe una cola llamada `drink-order` . Si la cola no existe, RabbitMQ la creará automáticamente.
5. Envuelva la lógica de creación de conexión y canal en un intento. Bloque `catch` . Registra o gestiona cualquier error que ocurra durante el proceso.
6. Usa la cola de pedidos de bebidas para registrar nuevos pedidos desde el punto de entrada principal de tu aplicación. En un servicio de cumplimiento independiente, lee desde la cola de pedidos de bebidas.

cola y también conectarse y escribir en la cola de análisis para fines de seguimiento.

NOTA

RabbitMQ se ejecuta en el puerto 5672 de forma predeterminada. Este número de puerto se seleccionó para el protocolo AMQP (Protocolo de Cola de Mensajes Avanzado), utilizado por RabbitMQ para la comunicación entre canales.

Ejemplo 116. Conectarse a su servidor RabbitMQ en index.js

```
...
función asíncrona connect() { try ❶

  { connection = await
    amqp.connect("amqp://localhost:5672"); ❷
      canal = await conexión.createChannel(); await ❸
      canal.assertQueue("orden-de-bebida"); } catch (err) ❹
    { console.error(err); ❺
  }
}
```

- ❶** Defina una función para configurar una conexión a RabbitMQ y la cola seleccionada.
- ❷** Conéctese al servidor RabbitMQ y asigne la conexión a la conexión.
- ❸** Cree un nuevo canal RabbitMQ y asígnelo al canal.
- ❹** Asegúrese de que exista la cola de pedidos de bebidas (o créela si no existe).
- ❺** Detecta y registra cualquier error durante la conexión o la creación del canal.

Con cada servicio conectado a su servidor RabbitMQ, puede empezar a transferir datos de una cola a otra. Para ello, agregue el

Función del **Ejemplo 11-7** para index.js. Esta función utiliza `channel.sendToQueue` para enviar un objeto JavaScript como datos binarios a la cola de pedidos de bebidas. El método `Buffer.from` convierte el objeto en un búfer binario para que pueda transferirse de manera eficiente.

NOTA

Los sistemas de mensajería como RabbitMQ esperan datos binarios. Serialización mediante `Buffer.from(JSON.stringify(...))` garantiza que los datos estructurados pueden ser reconstruidos de forma fiable por el receptor.

Esta función se utiliza en el servidor principal de Fastify y solo `fulfillment_service`, ya que estos son los sistemas que producen datos para colas. En `fulfillment_service`, cambie el nombre de la cola desde el pedido de bebida hasta el análisis.

Ejemplo 117. Agregar una función `sendOrderData` en index.js

```
...
función asíncrona sendOrderData(datos) {
  esperar canal.sendToQueue("pedido de bebida",
  Buffer.from(JSON.stringify(datos)));
}
```

...

1 Define una función para publicar datos en RabbitMQ.

2

Serializar los datos y enviarlos a la cola de destino.

También necesitarás llamar a `connect()` para inicializar la conexión cuando se inicia el servicio. Agregue esta línea después de la definición de la función.

Para actualizar su ruta para usar RabbitMQ para enviar pedidos de bebidas, modifique su controlador de ruta Fastify como se muestra en el **Ejemplo 11-8**:

Ejemplo de 118. Actualizar tu ruta para enviar datos del pedido a la bebida.

...

```
app.post("/order", async (request, reply) => { const { drinkOrder: order,
  cost, customer } = request.body; const data = { order, customer, };
await
  sendOrderData(data); ❶
```

```
console.log(`Drink: ${order} se está ❷
procesando para ${customer}`); reply.send("Procesando pedido "); });
```

❸

❹

...

- ❶ Crea un objeto que represente el pedido de bebida entrante.
- ❷ Envíe el objeto al canal de mensajería a través de RabbitMQ.
- ❸ Registra la transacción en la consola.
- ❹ Responder al cliente con una confirmación.

NOTA

En fulfillment_service, asegúrese de actualizar sendOrderData para usar channel.sendToQueue("analytics", ...) de forma que los datos sigan fluyendo por la canalización del sistema. Dentro de la función connectQueue , en la carpeta del proyecto fulfillment_service , agregue el código del **Ejemplo 11-9** para procesar los datos entrantes. Primero, establezca conexiones con la cola de lectura (drink-order) y la cola de escritura (analytics) mediante channel.assertQueue.

Al igual que channel.sendToQueue envía datos a una cola, channel.consume se usa para leer datos de una cola específica. Este método configura un consumidor que escucha los mensajes en la cola drink-order . A medida que los mensajes estén disponibles, este receptor consumirá los datos del mensaje.

A continuación, se extrae la propiedad de contenido (el cuerpo del mensaje) del objeto de datos . El contenido se convierte de binario a cadena y se analiza como JSON, de donde se extraen los valores del pedido y del cliente . Se registra un mensaje en la consola para indicar que el pedido se está procesando. `channel.ack(data)` envía una confirmación a RabbitMQ, informándole de que el mensaje se gestionó correctamente y debe eliminarse de la cola. Finalmente, los datos del pedido se pasan a `sendOrderData` para su reenvío a la siguiente cola.

NOTA

En RabbitMQ, un consumidor debe confirmar que ha procesado correctamente un mensaje. Si no lo confirma, RabbitMQ asume que el mensaje no se ha procesado y lo deja en cola, lo que puede provocar un procesamiento duplicado.

Ejemplo 119. Conectando a su consumidor de `fulfillment_service` en `index.js`

```
...
await channel.assertQueue("analítica"); await          ❶
channel.assertQueue("orden-de-bebida");

canal.consume("pedido-de-bebida", async (datos) => {    ❷
  const { content } = data; const                      ❸
  { order, customer } =
  JSON.parse(content.toString());                      ❹
  console.log(`${order} se está cumpliendo para ${customer}`); channel.ack(data); ❺
  await                                                ❻
  sendOrderData({ order, customer });                ❼
}
```

❶ Afirmar que existen las colas de análisis y pedidos de bebidas .

❷

Configure un consumidor para escuchar los mensajes entrantes en la cola de pedidos de bebidas .

- ③ Extraer el contenido binario del mensaje.
- ④ Analizar el mensaje en un objeto JavaScript.
- ⑤ Registrar el pedido entrante que se está cumpliendo.
- ⑥ Reconocer el mensaje para eliminarlo de la cola.
- ⑦ Reenviar el mensaje a la siguiente cola de procesamiento.

El paso final es consumir mensajes de la cola de análisis .

Para comenzar, define un objeto en tu proyecto `analytics_service` que registre el número de pedidos por bebida: `const drinkMap = { latte: 0, coffee: 0, cappuccino: 0 }`. Este objeto puede usarse para analizar cómo realizan los pedidos los clientes. Luego, para leer la cola de análisis y procesar esos datos, agrega el código del **Ejemplo 11-10** al final de la función `connectQueue` en `index.js`.

Esta configuración refleja la lógica de `fulfillment_service` : escuchar en una cola, extraer el mensaje, analizarlo y ejecutar una acción. En este caso, se incrementa el recuento de una bebida en `drinkMap`, se registra el paso de análisis y se confirma el mensaje.

Ejemplo 11-10. Conectando su consumidor de `analytics_service` en `index.js`

```
...
const drinkMap = { café con leche: 0, café: 0, capuchino: 0 };

esperar canal.assertQueue("analítica");           ❶

canal.consume("analytics", (datos) => { const      ❷
  { contenido } = datos; const                      ❸
  { pedido, cliente } =
```



```

JSON.parse(content.toString());
    si (drinkMap[orden] !== indefinido) {
        drinkMap[orden]++; }

    console.log(`${orden} siendo analizado para ${customer}`); channel.ack(data); }

```

- ...
- 1 Asegúrese de que exista la cola de análisis .
 - 2 Inicie un consumidor para la cola de análisis .
 - 3 Extraer el contenido del mensaje binario.
 - 4 Analizar el mensaje convertido en cadena en variables utilizables.
 - 5 Verifique si el tipo de pedido existe en el mapa de seguimiento e increméntelo.
 - 6 Registra el paso de análisis en la consola.
 - 7 Acusar recibo del mensaje a RabbitMQ.

Tu servidor principal y los dos servicios de soporte ya están completamente conectados a través de RabbitMQ. Para probarlos, abre tres ventanas de terminal, una para cada carpeta del proyecto: el servidor principal, fulfillment_service y analytics_service. En cada ventana, ejecuta:

índice de nodo

Luego, en una cuarta terminal, ejecute el siguiente comando curl para simular un nuevo pedido de bebida:

```

curl -X POST -H "Tipo de contenido: aplicación/json" \
  -d '{"bebidaOrden":"café con leche", "costo":"4.50","cliente":"Jon Wexler"}' \
  http://
  localhost:3000/orden

```

Su salida debería verse así:

- Servidor principal: Bebida: se está procesando un café con leche para Jon Wexler
- Servicio de cumplimiento: se prepara un café con leche para Jon Wexler
- Servicio de análisis: Latte analizado para Jon Wexler

Esto demuestra un flujo eficaz de gestión de mensajes asíncrona y distribuida de un servicio a otro mediante colas de RabbitMQ. Con esta estructura implementada, puede optimizar el tipo de procesamiento de datos que desea que se realice en cada servicio. Por ejemplo, puede verificar que tenga inventario antes de aceptar y procesar una solicitud de bebida en `fulfillment_service`.

También puede optimizar su lógica de análisis para mostrar métricas significativas en la salida de la consola de `analytics_service`. Añada el código del archivo **Example 11- js** en función de índice `analytics_service` para introducir un **11** en su que resume los porcentajes de pedidos de bebidas en tiempo real.

Este código define una función llamada `processDrinkAnalytics` que calcula el porcentaje de cada pedido de bebida del total de pedidos recibidos cada 10 segundos. Utiliza `setInterval` para programar la ejecución repetida e itera sobre las claves de `drinkMap`, que registra el recuento de cada bebida. Calcula el recuento total de pedidos, calcula los porcentajes, formatea los resultados y los registra en la consola. Además, `setTimeout` restablece el recuento de bebidas a cero cada 5 minutos para mantener las métricas actualizadas. Llame a `processDrinkAnalytics()` después de definirlo para que el seguimiento comience en cuanto se ejecute el servicio.

Función de 1111. Presentación de un servicio de análisis analítico
ejemplo en index.js

```
...
función processDrinkAnalytics() {
  constante CINCO_MINUTOS_EN_MILISEGUNDOS = 5 * 60 * 1000;
  constante DIEZ_SEGUNDOS_EN_MILISEGUNDOS = 10000;

  establecerIntervalo(() => {
    const drinkNames = Object.keys(drinkMap);

    const totalDrinkCount = drinkNames.reduce((total, drinkName) =>
{ return total +
    drinkMap[drinkName]; }, 0);

    const porcentajes de bebidas = nombres de bebidas.map((nombre de la bebida) => {
      porcentaje constante =
        Math.floor((drinkMap[nombreDeBebida] / totalDrinkCount) * 100) || 0; return });

      ` ${drinkName}: ${porcentaje}%`;

      console.log(` Pedidos de bebidas: ${drinkPercentages}`);

      establecerTiempo de espera(() => {
        drinkNames.forEach((nombreDeBebida) =>
          { mapaDeBebidas[nombreDeBebida]
            = 0; });
        }, CINCO_MINUTOS_EN_MILISEGUNDOS);
      }, DIEZ_SEGUNDOS EN MILISEGUNDOS);
    }

    procesoDrinkAnalytics();
  }
}
```

- 1 Define una función que maneje el análisis de pedidos de bebidas.
- 2 Establezca dos constantes de tiempo: una para reiniciar y otra para registrar.
- 3 Programe los registros de porcentaje de bebidas para que se impriman cada 10 segundos.

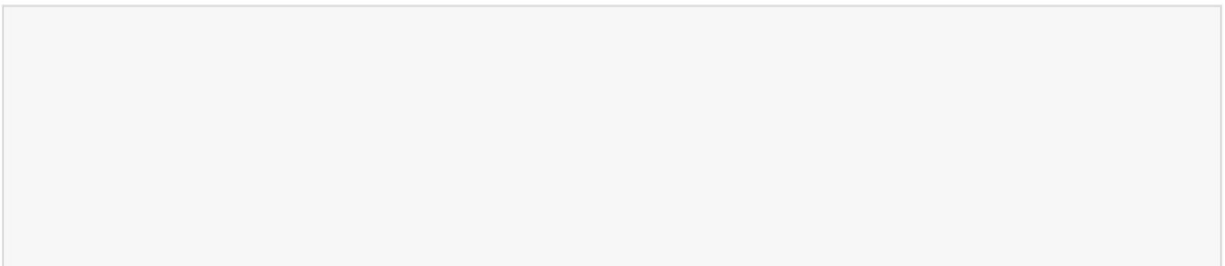
- ④ Recopila la lista de nombres de bebidas que se están rastreando.
- ⑤ Calcular el recuento total de bebidas.
- ⑥ Convierte el consumo de bebidas en porcentajes.
- ⑦ Formatear la cadena de porcentaje por bebida.
- ⑧ Registre el desglose del porcentaje de bebida.
- ⑨ Restablecer el conteo de bebidas cada 5 minutos.
- ⑩ Inicie el registro analítico cuando se ejecute el servicio.

Ejecute `analytics_service` nuevamente y envíe una variedad de pedidos de bebidas usando el comando `curl` :

```
curl -X POST -H "Tipo de contenido: aplicación/json" \  
  -d '{"bebidaOrden":"café con leche", "costo":"4.50","cliente":"Jon Wexler"}' \  
  http://localhost:3000/orden
```

Observa los desgloses porcentuales en tiempo real en la consola. Por ejemplo, después de enviar solo pedidos de café con leche, verás algo como:
Pedidos de bebidas: café con leche: 100%, café: 0%, capuchino: 0%.

Desde aquí, puede ampliar su sistema con servicios adicionales, puntos finales, bases de datos o una lógica de gestión de pedidos más detallada. Este capítulo demostró cómo diseñar un sistema de mensajería basado en colas que puede escalar según la demanda y mejorar la fiabilidad.



EJERCICIOS DEL CAPÍTULO

1. Agregue un servicio de seguimiento de inventario usando Redis:
 - a. Cree una nueva aplicación Node llamada `inventory_service` e instale el paquete `redis`.
 - b. En `index.js`, conéctese a Redis y suscríbase al canal de pedidos de bebidas.
 - c. Defina un objeto de inventario (por ejemplo, `{ latte: 10, café: 10, capuchino: 10 }`) que rastrea el stock disponible para cada bebida.
 - d. En cada pedido entrante, disminuya el inventario de la bebida correspondiente.
 - e. Registre una advertencia si el stock cae por debajo de 3 (por ejemplo, "Stock bajo para café con leche: quedan 2").
 - f. Utilice `setInterval` para restablecer el inventario a 10 cada 5 minutos.

CONSEJO

Utilice `JSON.parse(drinkOrder)` para extraer datos de los mensajes de Redis.

2. Vuelva a poner en cola los mensajes fallidos en RabbitMQ:
 - a. En su `fulfillment_service`, modifique el consumidor para la cola de pedidos de bebidas.
 - b. Simular una falla por cada tercer pedido (por ejemplo, `orderCount % 3 === 0`).

- c. Utilice `channel.nack(data, false, true)` en lugar de `ack()` para volver a poner en cola los mensajes fallidos.
- d. Registre un mensaje de reintento cada vez que esto sucede.
- e. De manera opcional, realice un seguimiento del número de reintentos con un campo de reintentos en la carga útil del mensaje.

CONSEJO

La puesta en cola evita la pérdida de pedidos, pero tenga cuidado de evitar bucles de reintentos infinitos.

Resumen

En este capítulo, usted:

- Se exploró cómo las aplicaciones Node manejan la alta demanda y el alto rendimiento mediante patrones y colas asincrónicas.
- Se crearon sistemas basados en colas para administrar las tareas entrantes y mejorar el manejo de solicitudes.
- Se utilizó Redis como una cola en memoria para almacenar en búfer y transmitir mensajes entre servicios.
- Se aprovechó Redis Pub/Sub para permitir la comunicación de eventos en tiempo real entre partes desacopladas de la aplicación
- RabbitMQ integrado para implementar colas de mensajes robustas y escalables con una arquitectura duradera orientada a servicios

Capítulo 12. Mercado Blockchain de Sellos Musicales

Este capítulo cubre lo siguiente:

- Construyendo su propia arquitectura blockchain
- Diseño de un mercado con un libro de contabilidad tokenizado
- Integración de Web3 para transacciones escalables y seguras

En este capítulo, creará una aplicación Node que explora la arquitectura de blockchain y cómo se puede adaptar con JavaScript.

A través de una implementación básica, aprenderá a conceptualizar una cadena de bloques e incorporar sus métodos fundamentales.

Construirás un prototipo de mercado que admita transacciones en blockchain y aprenderás cómo tu aplicación Node facilita una red descentralizada de intercambios mediante un libro de contabilidad distribuido. Al construir los elementos fundamentales de una blockchain, como el contrato inteligente, el rompecabezas criptográfico, la lógica de consenso y el libro de contabilidad general, comprenderás mejor cómo Node y JavaScript hacen posible esta arquitectura.

En la segunda mitad del capítulo, utilizará herramientas estándar de la industria como Web3, la red de cadena de bloques Ethereum y Solidity para crear contratos inteligentes e integrar su aplicación Node a un público más amplio.

Comprenderás mejor cómo se usa blockchain hoy en día y cómo su uso en el mercado se diferencia de su uso predominante en el intercambio de criptomonedas. Si ya estás familiarizado con blockchain, este capítulo reforzará tus conocimientos técnicos. De lo contrario, esta breve introducción te ayudará a familiarizarte con blockchain y, con el tiempo, a "explotar tu propio negocio".

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en **el Capítulo 1**, mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en **el Apéndice A**. Una vez completado, regrese aquí para continuar. Desarrollar un proyecto desde cero le ayuda a comprender mejor cada componente, brindándole mayor control y flexibilidad a medida que avanza.

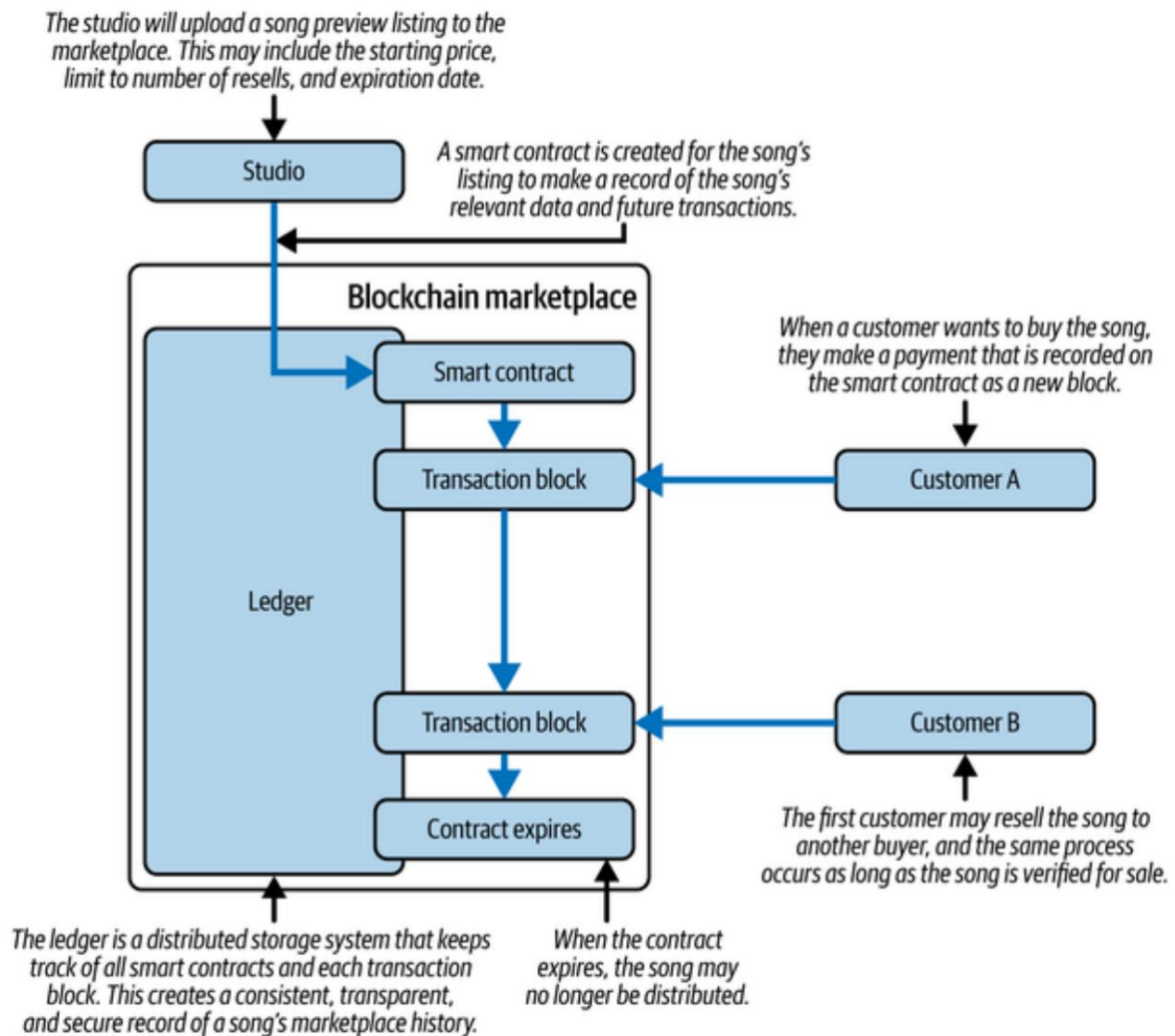
Un estudio musical,

DeSoundTralized Studios, **busca** asociarse con artistas importantes para vender avances de canciones y álbumes antes de su lanzamiento oficial. Quieren desarrollar un mercado donde los clientes puedan adquirir los derechos para escuchar archivos de audio de alta calidad de la colección limitada y autenticada. Quieren permitir que los compradores revendan las canciones después de escucharlas y limitar el número de reventas antes de que los derechos expiren cuando la música se lance al público. Además, no quieren gestionar la serie continua de ventas; solo la compra inicial y dejar que el mercado decida los precios a partir de entonces. La entrega del archivo de audio y cómo lo escucha el comprador se gestionará mediante otra plataforma y queda fuera del alcance de este proyecto.

Este problema

requiere un sistema que pueda ofrecer de forma segura y eficiente un mercado en tiempo real para la venta de archivos de audio, donde la venta depende de la autenticidad de la canción y su historial de ventas. Esto significa que cuando una canción está disponible durante un mes y hasta cinco reventas, su aplicación debe verificar la transferencia de propiedad de un comprador.

Para el siguiente, realizar un seguimiento del número de transferencias y eliminar propiedad del comprador final al momento del vencimiento. Además, El estudio de música no quiere gestionar las transacciones y distribución más allá de la venta inicial. En conjunto, estos criterios exigen una Aplicación que firma transacciones de forma segura y criptográfica. garantizar la autenticidad, con una base de datos descentralizada y distribuida. Estas condiciones son la prueba de una buena receta para blockchain. Ver **Figura 12-1** para visualizar mejor esta arquitectura.



Cifra 121. Visualizando el mercado blockchain para la distribución de música

BLOCKCHAIN EN POCAS PALABRAS

Blockchain es un registro digital y transparente de transacciones sin administración central. Para explicarlo con más detalle, considere el sistema actual de préstamo de libros de una biblioteca pública. A medida que los usuarios piden prestados libros, la biblioteca mantiene un registro interno e intenta notificar a los lectores cuándo deben devolver su libro. Se siguen ciertos criterios para garantizar que un libro pueda ser prestado, como el período de préstamo, su disponibilidad, el número de reservas realizadas e incluso su ubicación actual. Toda esta información suele ser gestionada por la biblioteca y es propensa a errores humanos, errores del sistema y pérdida de información. Esto, a su vez, puede provocar que los libros nunca se devuelvan o que no se puedan prestar para siempre.

Blockchain resuelve este problema porque está descentralizada y tiene un registro transparente del historial de cada libro en la biblioteca.

Imagine el sistema de préstamos de una biblioteca estándar, pero que comparte el historial de préstamos de cada libro (como una larga lista de fechas, nombres y ubicaciones) con todos los usuarios, y que automatiza el proceso de préstamo verificando este historial público y si un libro es elegible para préstamo o si es necesario devolverlo.

Si alguien desea tomar prestados Proyectos de Nodo (transacción) , el sistema revisa los registros de la cadena de bloques (libro mayor). Este proceso escanea la colección cronológica de transacciones (bloque) y verifica los criterios de préstamo (contrato inteligente) antes de crear una nueva transacción. Esta transacción se publica para que otros usuarios de la red de la biblioteca la aprueben. Los usuarios de la biblioteca se apresuran a validar la transacción resolviendo un acertijo (rompecabezas criptográfico) y acordando la respuesta (consenso), de modo que la transacción pueda codificarse en un bloque y añadirse al libro mayor transparente. Este proceso depende de una red colectiva.

NOTA

El rompecabezas criptográfico puede variar en complejidad y, a menudo, requiere un gran poder computacional para resolverlo. Una razón para este requisito en la validación de transacciones es evitar que ataques maliciosos o transacciones falsas accedan al libro mayor. En el ejemplo de la biblioteca, los usuarios que resuelvan el acertijo podrían recibir recompensas a cambio de su esfuerzo. Asimismo, un sistema de cadena de bloques digital podría recompensar a quienes resuelvan el rompecabezas y a quienes verifiquen la solución. La resolución y validación del rompecabezas generalmente está automatizada y es realizada por participantes en la red con computadoras dedicadas conectadas a la red.

Los siguientes son parte de la terminología central de blockchain:

Cadena de bloques

La cadena de bloques en este sistema sería una tecnología de libro mayor distribuido que permite el registro seguro, transparente e inmutable de todas las transacciones de préstamo de libros en la red.

Transacción

Una transacción en este sistema sería el acto de tomar prestado un libro de la biblioteca. Esto implicaría que el prestatario iniciaría la transacción enviando una solicitud al contrato inteligente en la red blockchain, que la validaría y verificaría antes de registrar los detalles en el libro contable. La transacción también implicaría que el prestatario devolviera el libro a la biblioteca, lo que activaría una nueva transacción que se registraría en la red blockchain.

Contrato inteligente

Un contrato inteligente en este sistema sería un programa autoejecutable que hace cumplir las reglas y condiciones del proceso de préstamo de libros. Estaría escrito en código e implementado en la red blockchain para automatizar el proceso de préstamo.

Consenso

En este sistema, el consenso se refiere al proceso de verificación y validación de las transacciones de préstamo de libros en la red blockchain. Esto se realiza generalmente mediante un algoritmo de consenso, como la prueba de trabajo o la prueba de participación, que implica que una red de nodos acuerda la validez de las transacciones.

Bloquear

Un bloque en este sistema sería un conjunto de transacciones de préstamo de libros verificadas, que se agregan a la red blockchain en orden cronológico.

Libro

mayor. El libro mayor de este sistema sería un registro digital de todas las transacciones de préstamo de libros en la red blockchain, incluyendo detalles como el título del libro, el nombre del prestatario, el período de préstamo y la fecha de devolución. El libro mayor sería inmutable y transparente, lo que significa que ninguna transacción registrada en él podría ser alterada ni eliminada.

Debido a que este sistema está descentralizado, depende de una red activa de participantes que realizan transacciones y las verifican.

Por esta razón, las plataformas blockchain exitosas ofrecen incentivos valiosos para alentar la participación en el mantenimiento del contrato inteligente y los elementos de consenso para aprobar y persistir una nueva transacción.

Para comprender mejor el funcionamiento de la cadena de bloques, se desarrollan las piezas fundamentales en código, incluyendo las clases Bloque, Cadena de Bloques, Transacción y Nodo . La clase Nodo se utiliza para registrar un nuevo nodo, o servidor participante, en la red de la cadena de bloques.

Obtenga programación

Para comenzar a desarrollar su aplicación, cree una nueva carpeta de proyecto llamada `blockchain_marketplace`. Vaya a la carpeta de su proyecto en su Línea de comandos y ejecute `npm init`. Este comando inicializa su Aplicación de nodo. Puede presionar Enter durante los pasos de inicialización para Acepte los valores predeterminados. El resultado de estos pasos es la creación de un archivo. llamadas `package.json`. Este archivo le indicará a su aplicación Node cualquier configuraciones de paquetes o scripts necesarios para ejecutarse correctamente.

A continuación, crea un archivo llamado `index.js` dentro de la carpeta de tu proyecto. Este archivo es El punto de entrada para su aplicación.

Primero, crea las clases fundamentales para tu aplicación:

`MarketplaceNode`, `Blockchain`, `Bloque` y `Transacción`.

Crea una carpeta llamada `src` dentro del directorio de tu proyecto. Aquí es donde El código de tus clases estará activo. Crea el `marketplaceNode.js`.

Archivos `blockchain.js`, `block.js` y `transaction.js` dentro de la carpeta `src`.

Luego instala el paquete `fastify` ejecutando `npm install fastify@^5.4.0`, y dentro de su archivo `index.js`, inicializa su servidor web, como se muestra en [el Ejemplo 12-1](#).

En esta lista, configura una aplicación `Fastify` y la configura para que tome una Valor de puerto personalizado. El puerto se puede asignar mediante una línea de comandos. argumento que se pasa después de la línea `index.js` del nodo. `process.argv.slice(2)` devuelve los valores después del ejecutable Solo la ruta y asigna ese valor a `PORT`. Si no se pasa ningún valor, El puerto predeterminado es 0.

CONSEJO

Establecer el `PUERTO` en 0 le indica a `fastify.listen()` que elija un valor de puerto para ti. Esto resulta útil al asignar valores de masa para varios aplicaciones.

Configura la instancia de Fastify para que gestione y analice contenido JSON mediante el analizador de cuerpo integrado. Por último, inicia el servidor haciendo que escuche en el valor de puerto especificado . Si no existe ningún valor de puerto , Fastify le asigna uno. Reasigna el puerto a ese número mediante `server.address().port` y cierra la sesión de la ubicación del servidor al iniciar.

Ejemplo 121. Configura tu servidor web en `index.js`

```
import Fastify from "fastify"; const 1
fastify = Fastify(); let [PORT] = 2
process.argv.slice(2); PORT = PORT || 0; 3

fastify.get("/", async (solicitud, respuesta) => {
  responder.enviar({ mensaje: "Mercado en ejecución" }); }); 4
```

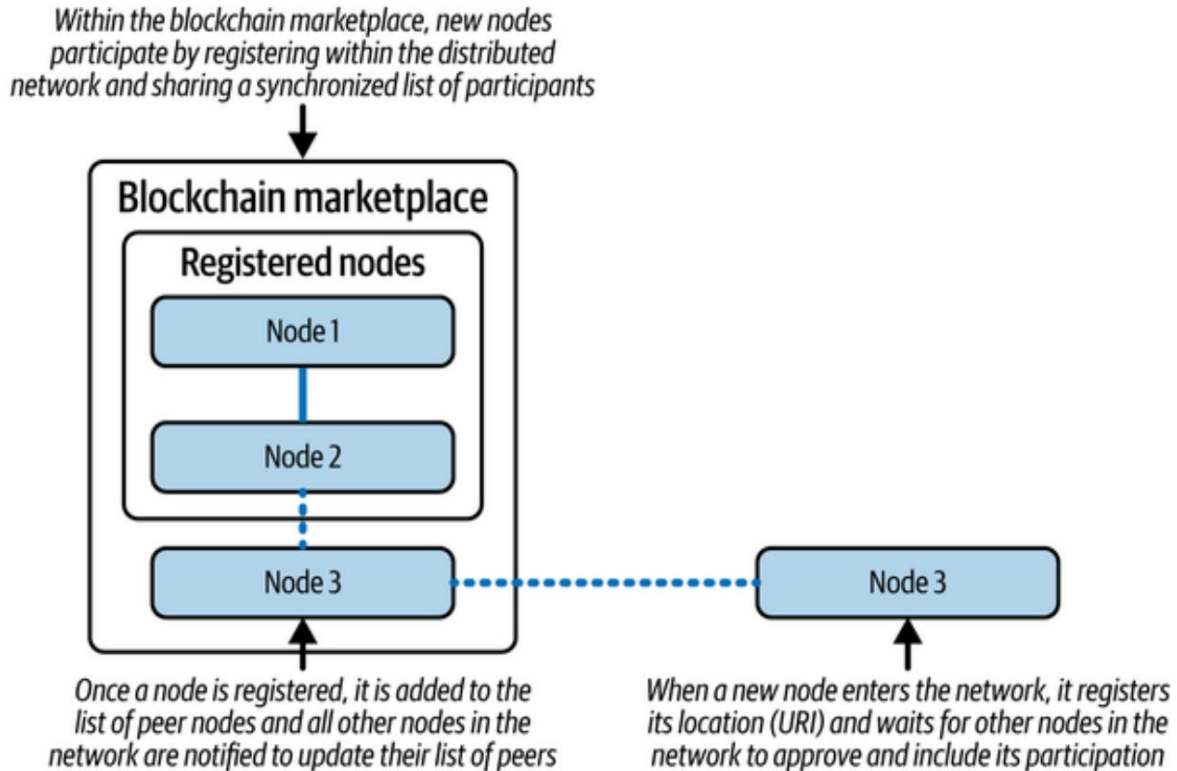
```
await fastify.listen({ port: PORT }); PORT = 5
fastify.server.address().port; 6
console.log(`Ejecutando en http://localhost:${PORT}`); 7
```

- 1 Importa la biblioteca fastify a tu proyecto.
- 2 Crear una instancia de Fastify.
- 3 Recupere argumentos de la línea de comandos y desestructura la matriz para asignar el primer argumento a la variable PORT .
- 4 Ruta de ejemplo para verificar que el servidor está funcionando correctamente.
- 5 Inicie el servidor y escuche en el puerto asignado.
- 6 Actualice la variable PORT al número de puerto asignado real.
- 7 Registra la ubicación del servidor en tu consola.

Pruebe este servidor ejecutando `node index.js` en la línea de comandos en el nivel raíz del proyecto. Tenga en cuenta que cada vez que inicie su...

Aplicación, se asigna un nuevo número de puerto. Luego, ejecute el nodo index.js 3000 y observe que su aplicación solo se ejecuta en el puerto 3000. De ahora en adelante, el puerto 3000 representará su mercado principal. Ubicación del servidor del nodo. Esto significa que `http://localhost:3000` actuará como El punto de partida para que nuevos nodos participen en la cadena de bloques mercado. A medida que nuevos nodos ingresen al mercado, necesitarán Transmiten la ubicación de su servidor a todos los demás nodos para que La comunidad de nodos es conocida por todos los participantes. Para lograrlo, Comienza a construir la clase MarketplaceNode y proporcionándole API. puntos finales para interactuar con otros nodos.

La figura 12-2 demuestra cómo los nuevos nodos del mercado ingresan al mercado. red. Cuando un nuevo nodo quiera participar, registrará su URI. con la red. A partir de ahí, la red de pares existente evaluará El nodo del mercado entrante y agregarlo a la lista de pares. Esta lista es luego se sincroniza en todos los nodos pares para que la lista de La cantidad de participantes es consistente en toda la red.



Cifra 122. Agregar un nuevo nodo de mercado a la red

Para lograr esto, agregue el código del **Ejemplo 12-2** a su archivo `MarketplaceNode.js` en `src`. Este archivo contendrá toda la lógica de la clase `MarketplaceNode`. En este código, defina la clase `MarketplaceNode` con un constructor que acepta tres parámetros: URL, pares y blockchain. Estos parámetros representan la URL del nodo del mercado (ubicación de ese nodo de red), una matriz de pares (otros nodos en la red del mercado) y una referencia al objeto blockchain, respectivamente.

El constructor también inicializa otras propiedades, como establecer el saldo del nodo en 50 000 (esto equivale a 50 \$ en centavos). Por ahora, cada nodo de la red comenzará con un fondo inicial para mostrar las transacciones dentro de la red. `this.songs = {}` inicializa un hash vacío asignado a la propiedad "songs", que se utiliza para registrar las canciones compradas en la red. Por último, `this.broadcastSelf()` invoca la función "broadcastSelf" para informar a otros nodos de la red del mercado sobre la existencia de este nodo.

ejemplo 122. Definición de la clase `MarketplaceNode` en el `marketplaceNode.js`

```
class MarketplaceNode ❶
{ constructor(url, peers = [], blockchain) { ❷
  este.url = url;
  este.peers = peers;
  este.blockchain = blockchain;
  este.balance = 50000;
  este.songs = {};
  este.broadcastSelf(); }
}
```

```
❸ export default MarketplaceNode; Define
```

❶ la clase `MarketplaceNode` para representar un solo nodo en la red del mercado.

❷

Inicializar las propiedades del nodo, incluida su URL, lista de pares, referencia de blockchain, saldo de cuenta y almacén de canciones local. Transmite inmediatamente el nodo a sus pares.



Exporte la clase MarketplaceNode como el módulo de exportación predeterminado para que pueda importarse en otras partes de la aplicación.

En este momento, tienes una clase con una referencia a la función `broadcastSelf`. La difusión es esencial para el funcionamiento de una red blockchain, ya que informa a todos los demás nodos de la red sobre los cambios en el mercado. La transparencia y la entrega consistente de cambios en la red distribuida es lo que distingue a blockchain de otros sistemas transaccionales. Por ello, primero crearás una función de difusión que otras funciones puedan usar para difundir información a través de la red. Agrega la función de difusión que se muestra en [el Ejemplo 12-3](#) a tu clase

`MarketplaceNode`. Esta función usa `axios` para realizar una solicitud HTTP a la URI de cada nodo par y toma una ruta y datos específicos como argumentos. La ruta especificará a qué punto final se envían los datos, y los datos pueden representar cualquier cambio o actualización en la red, como el registro de un nuevo `MarketplaceNode`.

Dentro de la función, itera sobre los pares de `MarketplaceNode`, omitiendo la URL de su propio nodo y realizando una solicitud POST a la ruta especificada para la ubicación de ese par. Esto realiza una solicitud HTTP saliente al mismo punto final para cada par en la red. Para que esto funcione, necesita instalar el paquete `axios` ejecutando `npm install axios@^1.11.0` en la raíz de la carpeta de su proyecto en la línea de comandos e importando la biblioteca `axios` (`import axios from 'axios'`) a la parte superior de `MarketplaceNode.js`.

NOTA

La función de mapa se utiliza para crear una matriz de promesas, y Promise.all garantiza que todas las promesas se resuelvan antes de moverlas en.

Ejemplo 123. Agregar una función de transmisión en marketplaceNode.js

```
...
async broadcast(ruta, datos) { await ❶

  Promise.all( this.peers.map(async (peer) => { if ❷
    (peer === this.url) return; try { await ❸

    axios.post(`${peer}/${ruta}`, datos); } catch (error) ❹
    { console.error(error.message);

  } }));
}
```

...

- ❶** Define la función de transmisión asíncrona para tomar una ruta y un argumento de datos .
- ❷** Iterar sobre la lista de URL de pares de MarketplaceNode .
- ❸** Omite la URL del par que coincide con la URL del nodo de transmisión.
- ❹** Realice una solicitud POST de axios a la ruta proporcionada, pasando datos en el cuerpo de la solicitud.

Con la función de difusión activada, se puede reutilizar para transmitir información a todos los demás nodos pares de la red. Dos ejemplos son broadcastSelf, para avisar a otros nodos pares cuando te has unido a la red, y registerNode para añadir otros nuevos.

Nodos participantes a su lista de pares. Agregue las funciones del **Ejemplo 12-4** a su clase MarketplaceNode para usarlas. La función broadcastSelf es una función asíncrona que llama a los puntos finales de registro de otros nodos pares y pasa la URI de su propio nodo de mercado como datos. La función registerNode recibe la información de la URI del nuevo nodo de mercado y la añade a su lista de pares. A continuación, ese nodo llama a la función broadcast para el punto final sync-peers, pasando todos sus pares como datos. El punto final sync-peers notificará a todos los demás nodos de la red para que se alineen con los datos del nodo par, de modo que todos los participantes estén sincronizados.

Ejemplo: Cómo agregar las funciones broadcastSelf y registerNode en marketplaceNode.js

```
...
difusión asíncrona Self() { ❶
  this.broadcast("nodo-de-registro", { url: this.url }); } ❷

async registerNode(newNodeUrl) ❸
{ this.peers.push(newNodeUrl); ❹
  await this.broadcast("sync-peers", { peers: this.peers });
} ❺
...
```

- ❶ Define la función broadcastSelf.
- ❷ Llame a la función de transmisión en el objeto MarketplaceNode utilizando la ruta del nodo de registro y la URL del nodo como datos.
- ❸ Define la función registerNode que toma un argumento newNodeUrl.
- ❹ Agregue el valor newNodeUrl a la lista de pares de su nodo.
- ❺

Agregue la función de transmisión en el objeto MarketplaceNode utilizando la ruta sync-peers y la lista de pares del nodo como datos.

Con estas funciones implementadas, ahora puede inicializar un nuevo Instancia de MarketplaceNode . En la parte superior del archivo indexjs , declara la variable marketplaceNode agregando let marketplaceNode. Luego, agregue la función initializeNode desde **Ejemplo 12-5** de indexjs. Esta función se utiliza para establecer el valor actual. URI del nodo, inicializa la lista de nodos pares (para comenzar, sus nuevos nodos) siempre apuntará al mismo nodo antes de sincronizar su lista para incluir todos participantes en la red). Por último, asigna marketplaceNode a una nueva instancia de MarketplaceNode usando la URL del nodo y Array initialPeers . Luego, puedes llamar a initializeNode(). inmediatamente después de que Fastify termine de escuchar y se asigne un puerto.

NOTA

Es importante llamar a esta función solo después de asignar el valor PORT , de lo contrario, su nodo inicializado no tendrá una URI establecida.

Ejemplo 125. Agregar una función initializeNode en index.js

```
...
const inicializarNodo = () => {
  constante URL = `http://localhost:${PORT}`;
  constante initialPeers = ["http://localhost:3000"];
  marketplaceNode = nuevo MarketplaceNode(URL, initialPeers);
};
...
```

1 Define la función initializeNode .

2 Asigna una URL a tu URL de desarrollo y se asigna dinámicamente Número de PUERTO .

- ③ Establezca la matriz `initialPeers` para incluir el URI `http://localhost:3000`.
- ④ Asignar `marketplaceNode` a un nuevo `MarketplaceNode` instancia que utiliza la URL definida y la lista `initialPeers`.

Con su `MarketplaceNode` inicializado en `index.js`, el paso final es Para definir los puntos finales `/register-node` y `/sync-peers`. Agregar el código del **Ejemplo 12-6** hasta la mitad de `index.js`, después de inicializar el variable `fastify`.

Ejemplo de ¹²⁶adición de API de registro-nodo y sincronización de pares puntos finales en `index.js` con Fastify

```
fastify.post("/register-node", async (solicitud, respuesta) => { ①
  const { url: newNodeUrl } = solicitud.cuerpo; ②
  esperar marketplaceNode.registerNode(newNodeUrl); ③
  responder.enviar({ mensaje: "Nodo registrado" }); ④
});

fastify.post("/sync-peers", async (solicitud, respuesta) => { ⑤
  const { peers } = solicitud.cuerpo; ⑥
  marketplaceNode.peers = pares; ⑦
  console.log(`${marketplaceNode.url} sincronizado
  ${marketplaceNode.peers}`); ⑧
  reply.send({ message: "Pares sincronizados" }); ⑨
});
```

- ① Define la ruta POST `/register-node` usando Fastify.
- ② Extraiga la URL del cuerpo de la solicitud.
- ③ Registra el nodo en el `marketplaceNode` actual.
- ④ Devolver un mensaje confirmando el registro.
- ⑤ Define la ruta POST `/sync-peers`.
- ⑥ Extraiga la lista entrante de URL de pares.

- 7 Reemplazar la lista de pares locales.
- 8 Registra la sincronización entre pares con fines de depuración.
- 9 Confirme la sincronización con un mensaje JSON.

Su código ahora está listo para configurar una red de mercado y asumir el reto.

Nuevos participantes del nodo. Para probar esto, abra una ventana de línea de comandos en el directorio raíz de su proyecto y ejecute el nodo índice 3000 para iniciar su servidor. Esto iniciará el código en `index.js`, que crea un nuevo

`MarketplaceNode`. El primer nodo se establecerá en `http://localhost:3000`. Su línea de comandos debe indicar que este nodo está en línea (Figura 12-

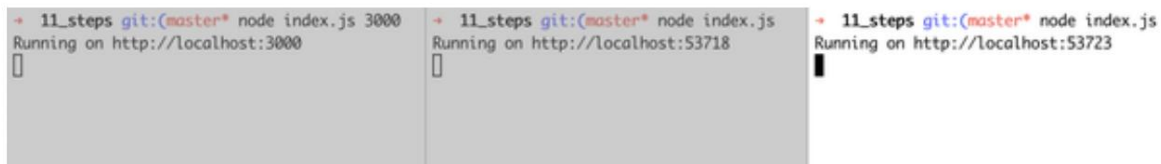
3) Ahora abra una segunda y tercera ventana desde el mismo directorio.

Ejecutar el índice del nodo e iniciar su aplicación una segunda y tercera vez

Mientras el primer servidor esté en funcionamiento, los otros dos servidores se iniciarán.

y registrarse inmediatamente en la red a través del nodo en

`http://localhost:3000`.



```

+ 11_steps git:(master*) node index.js 3000
Running on http://localhost:3000

+ 11_steps git:(master*) node index.js
Running on http://localhost:53718

+ 11_steps git:(master*) node index.js
Running on http://localhost:53723

```

Cifra 123. Salida de la línea de comandos para iniciar tres servidores de nodo

Puede registrar la lista de pares agregando

`console.log(`${marketplaceNode.url} sincronizado`

`${marketplaceNode.peers}`)` en su ruta `/sync-peers` en

`index.js`. Esto te ayudará a ver la sincronización en acción. El último servidor que...

Las empresas emergentes se agregarán al final de la lista de pares para cada participante.

nodo. El mensaje de registro debería ser similar a

`http://localhost:53896 sincronizado`

`http://localhost:3000,http://localhost:53884,http:`

`//localhost:53896`.

Con los nodos del mercado conectados, es hora de construir la estructura central del mercado en sí, la cadena de bloques.

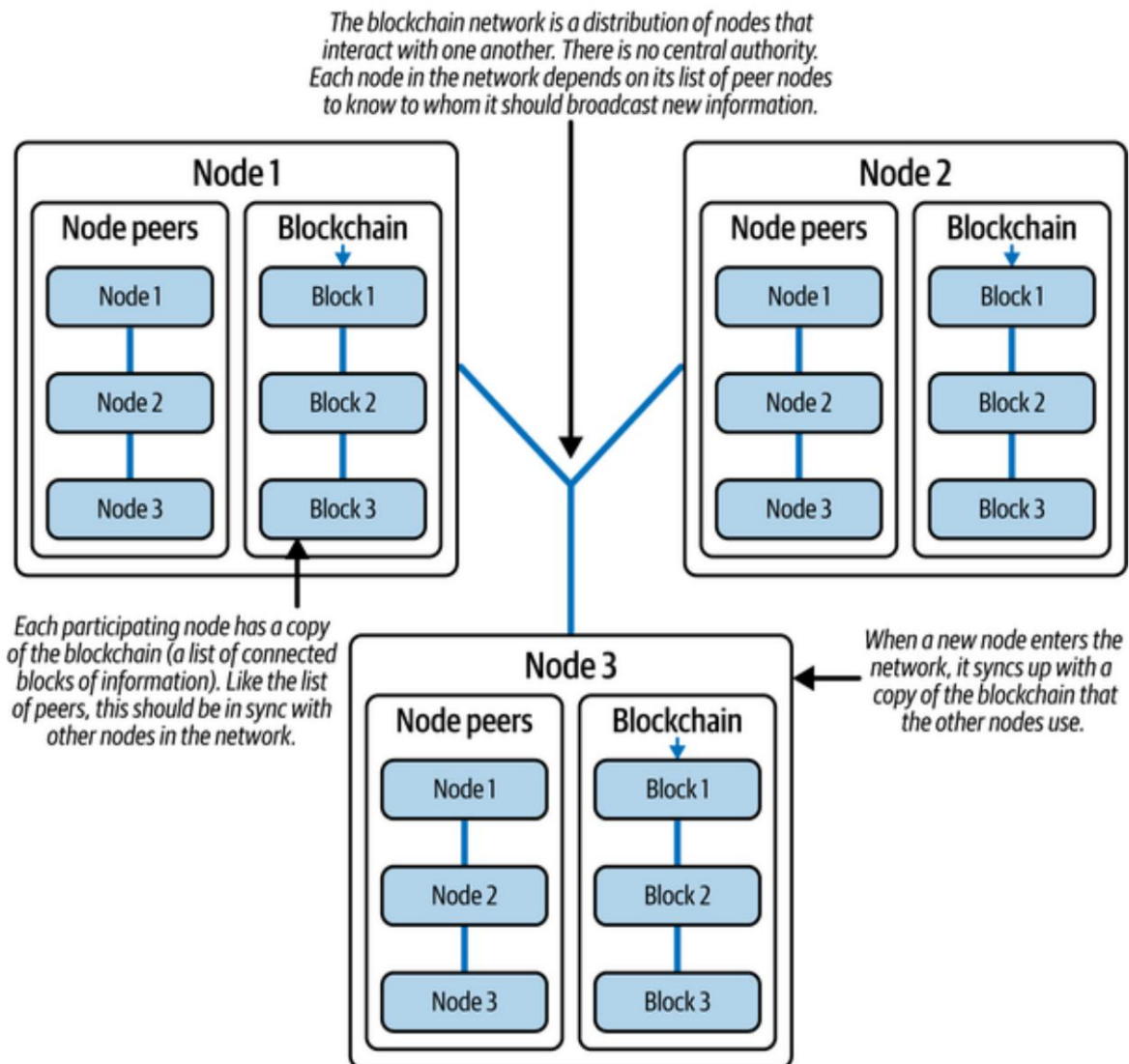
Codificación de la cadena de

bloques. El mercado depende en gran medida de la capacidad de los nodos participantes para intercambiar dinero e información a través de la red.

Esto es posible gracias a la estructura de la cadena de bloques. A medida que los participantes participan en el mercado, se realizan más transacciones y el registro de los cambios se guarda en la cadena de bloques, sincronizada en todos los nodos de la red.

La Figura 12-4 muestra cómo la red existe sin una autoridad central.

Cada nodo de la red contiene una copia de todos los cambios y una lista de todos los nodos participantes. De esta forma, cualquier nodo puede iniciar una transacción y todos los demás nodos pueden consultar la misma copia de la cadena de bloques para verificar su validez.



Cifra 124. Visualizando la red blockchain de nodos

Desglosando aún más esta estructura, la Figura 12-5 detalla el contenido.

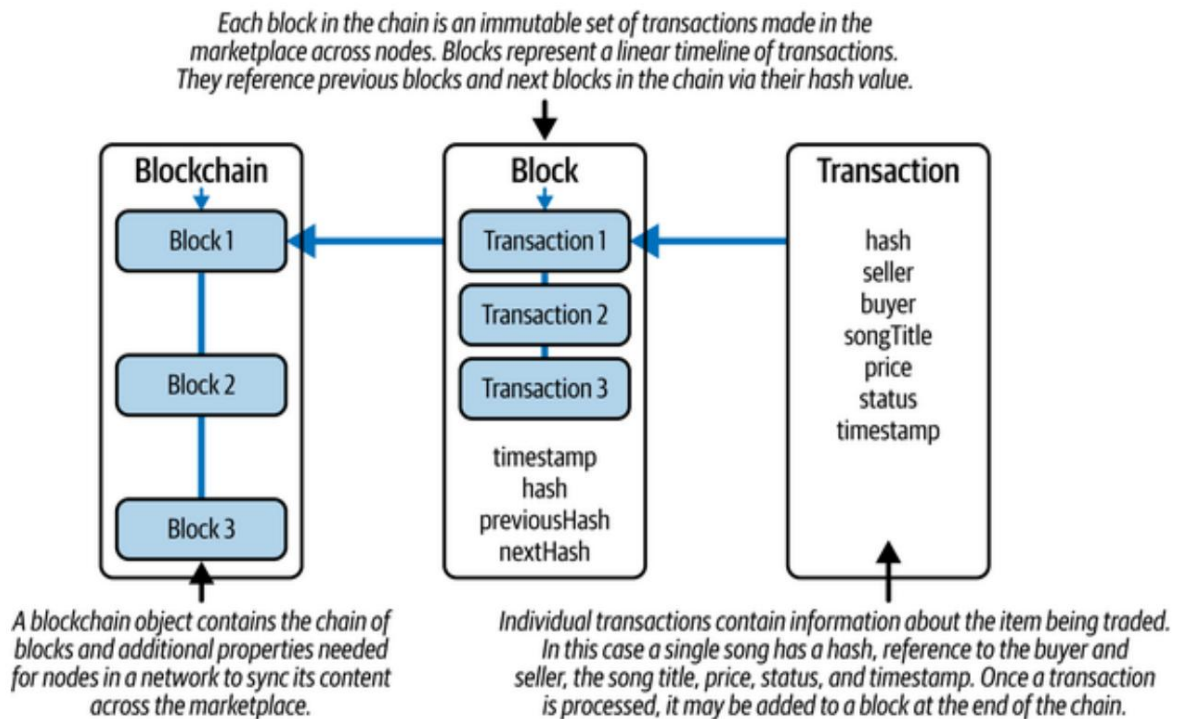
de las clases Blockchain, Bloque y Transacción . El

La clase Blockchain contiene una colección de bloques. Una instancia de la

La clase de bloque representa una colección de transacciones y contiene propiedades que lo distinguen de otros bloques de la cadena. Individual

Las transacciones de la clase Transacción realizan intercambios de canciones

Para la moneda una posibilidad.



Cifra 125. Visualización de la relación entre modelos de datos

Debido a que la cadena de bloques depende de bloques, se comienza construyendo la clase `Block`. Agregue el código del [Ejemplo 12-7](#) a `blockjs` en su carpeta `src` del proyecto. Este código define una clase llamada `Block` que representa un bloque en una cadena de bloques, preparándolo para vincularse con otros bloques para formar una cadena. Un bloque contiene transacciones y tiene propiedades como una marca de tiempo, `previousHash`, `hash` (el hash actual) y `nextHash` (una referencia al siguiente bloque).

Para inicializar un nuevo objeto `Bloque` se requiere una matriz de transacciones y un valor `previousHash` opcional. La marca de tiempo se establece en el hora actual utilizando el método `Date.now()`. Las transacciones son asignado a la propiedad de transacciones del objeto `Bloque`. El hash anterior se almacena en la propiedad `previousHash`. El hash actual se calcula llamando al método `calculateHash`, que se añadirá en breve. El hash calculado se almacena en la propiedad `hash` y la propiedad `nextHash` se establece inicialmente en `nula`, lo que indica que todavía no hay ninguna referencia al siguiente bloque.

Ejemplo 127. Definición de la clase Block en block.js

clase Block

```

{ constructor(transactions, previousHash) { this.timestamp ❶
  = Date.now(); this.transactions = ❷
  transaction; this.previousHash = previousHash; ❸
  this.hash = this.calculateHash(); this.nextHash ❹
  = null; } ❺
} ❻

```

```

} export default Block; Acepta ❼

```

- ❶ transacciones y un valor previousHash al crear una instancia de un nuevo Bloque.
- ❷ Asignar una propiedad de marca de tiempo a la fecha y hora actual.
- ❸ Asigna una propiedad de transacciones a la matriz de argumentos de transacciones.
- ❹ Asigne una propiedad previousHash al valor del argumento que apunta al último bloque.
- ❺ Asignar una propiedad hash a un nuevo valor hash calculado.
- ❻ Asigne una propiedad nextHash a un valor nulo para comenzar.
- ❼ Exporte la clase Block para usarla en otras partes de la aplicación.

A continuación, agregue la función calculateHash a su clase Block mediante el código del **Ejemplo 12-8**. Este código define un método para calcular y devolver el valor hash del bloque mediante el algoritmo hash SHA-256. El método comienza llamando a la función createHash , que forma parte de la biblioteca de cifrado incluida con Node.

Necesitará agregar import { createHash } desde "crypto" js para utilizar esta función.

createHash en la parte superior del bloque

crea un objeto hash, utilizando el algoritmo SHA-256 que se utilizará para calcular el valor hash.

NOTA

SHA-256 es un algoritmo hash ampliamente adoptado y estandarizado, conocido por su eficiencia computacional y la casi imposibilidad de revertir el valor hash a su entrada original. Más importante aún, es extremadamente improbable que dos entradas diferentes produzcan el mismo valor hash, lo que lo convierte en un buen candidato para generar un identificador hash aleatorio para sus bloques.

A continuación, se invoca el método de actualización en el objeto hash, utilizando el hash anterior del bloque, la marca de tiempo y una representación JSON de las transacciones del bloque como datos de entrada para el hash. Se invoca el método de resumen en el objeto hash, pasando hex para generar el valor hash final en formato hexadecimal. De esta forma, se genera un valor hash único para cada bloque. Este valor hash es un componente importante de la tecnología blockchain, ya que garantiza la integridad y seguridad de los datos almacenados en cada bloque.

Ejemplo 128. Creando la función calculateHash en block.js

...

calculateHash() { return ❶

❷

❸

createHash("sha256").update(este.previousHash + este.timestamp + JSON.stringify(este.transact

) .digest("hexadecimal"); ❹

}

...

❶

Define la función calculateHash para generar un valor hash cifrado.

- ② Utilice el método `createHash` para crear un objeto hash utilizando el algoritmo `sha256`.
- ③ Genere un valor hash basado en el valor hash anterior de la entrada, la marca de tiempo y la matriz de transacciones.
- ④ Convierte el valor a un valor hexadecimal.

Con la clase `Block` configurada, puede empezar a trabajar en la clase `Blockchain`. Cree un archivo llamado `blockchain.js` en `src` y agregue el código del [Ejemplo 12-9](#). Este código importa la clase `Block` desde `block.js`. La clase `Blockchain` inicializa nuevos objetos `Blockchain` con los parámetros opcionales `"chain"` y `"pendienteTransacciones"`. Si se proporciona una cadena, se asigna a la propiedad `"chain"`. De lo contrario, se crea una nueva cadena con un bloque génesis mediante el método `createGenesisBlock`. De igual forma, `"pendienteTransacciones"` tomará como valor predeterminado una matriz vacía, a menos que se pase una matriz de valores. El constructor también inicializa otras propiedades como el nivel de dificultad, una lista de transacciones pendientes y la recompensa de minería.

MINERÍA DE UN BLOQUE

La minería es el proceso de añadir nuevos bloques a la blockchain mediante la resolución de un problema computacional. El objetivo de la minería es encontrar un valor hash que cumpla ciertos criterios para que la red de nodos pueda añadir un bloque a la blockchain de forma segura y descentralizada. La clase Blockchain funciona simultáneamente como almacenamiento de bloques y transacciones, así como modelo para codificar nuevos bloques en la cadena. Los siguientes son los pasos clave del proceso de minería en relación con la clase Blockchain :

Transacciones pendientes

Un nodo minero recopila un conjunto de transacciones pendientes que aún no se han incluido en ningún bloque.

Formación de bloques

El minero reúne estas transacciones pendientes en un nuevo bloque, junto con otra información como el hash del bloque anterior y una marca de tiempo.

Cálculo de hash

El minero calcula el hash del bloque aplicando la función `calculateHash` a los datos del bloque. La función hash considera todo el contenido del bloque, incluyendo las transacciones, el hash del bloque anterior y la marca de tiempo.

Prueba de trabajo

El minero ajusta repetidamente un valor numérico aleatorio y recalcula el hash del bloque hasta encontrar un valor que satisfaga el problema computacional. Los criterios específicos dependen del nivel de dificultad de la red, que suele estar representado por el número de ceros iniciales requeridos en el hash.

Validando la prueba

Una vez que el minero encuentra un valor hash adecuado, transmite el bloque a la red. Otros participantes de la red pueden verificar la prueba de trabajo rehaciendo los datos del bloque y confirmando que el hash resultante cumple con los criterios de dificultad.

Para este proyecto, puede omitir este paso para reducir la complejidad del código.

Adición de bloques

Si la prueba de trabajo es válida, el bloque se añade a la blockchain y el minero recibe una recompensa predefinida por su esfuerzo. El bloque se vincula al bloque anterior mediante su hash, formando una cadena de bloques.

Actualización de la red

Después de agregar un bloque, el minero inicia el proceso nuevamente, recopilando nuevas transacciones pendientes y creando un nuevo bloque para minar.

Mediante este proceso, los nodos del mercado en la red tienen un incentivo para intercambiar bienes y divisas, a la vez que contribuyen a la distribución de registros y la fiabilidad de los datos de la blockchain. Por ello, la clase Blockchain define el nivel de dificultad y la recompensa de minería .

El método `createGenesisBlock` crea un nuevo bloque génesis, que es el primer bloque de la cadena de bloques si no se proporciona ninguna cadena . Toma una matriz vacía como parámetro de transacciones y establece el hash anterior como nulo. El método `getLatestBlock` recupera el bloque más reciente de la cadena accediendo al último elemento de la matriz . Por último, la clase Blockchain se exporta para su uso en otros archivos.

import 129. Definición de la clase Blockchain en blockchain.js Ejemplo

```

Block from "./block.js"; class Blockchain ❶
{ constructor(chain, ❷
  pendingTransactions = [] ❸ {
    esta.cadena = cadena || [esta.createGenesisBlock()]; esta.dificultad ❹
    = 4; ❺
    esta.transaccionespendientes = transaccionespendientes; ❻
    esta.recompensamineria = 100; } ❼

  createGenesisBlock() ❽
  { devolver nuevo Bloque([], null);
  }

  getLatestBlock() ❾
  { devolver esta.cadena[esta.cadena.longitud - 1];
  }

} exportar Blockchain predeterminado;

```

- ❶ Importe la clase Block , que se utiliza para crear nuevos bloques en la cadena.
- ❷ Define la clase Blockchain , que representa un libro de contabilidad distribuido de bloques.
- ❸ Cree una nueva instancia de blockchain con una cadena inicial opcional y una lista de transacciones pendientes.
- ❹ Si no se proporciona ninguna cadena, inicialice la cadena con un bloque de génesis.
- ❺ Establezca la dificultad de minería, que determina qué tan difícil es minar un nuevo bloque.
- ❻ Almacene cualquier transacción que esté esperando a ser agregada al siguiente bloque minado.
- ❼ Establece la recompensa minera otorgada a un nodo por minar con éxito un bloque.

- 8 Define el método `createGenesisBlock` , que devuelve el bloque inicial en la cadena de bloques.
- 9 Defina el método `getLatestBlock` para recuperar el bloque más reciente. bloque en la cadena.

NOTA

El bloque génesis es creado o generado manualmente por la cadena de bloques. La red como punto de partida cuando una red blockchain se pone en marcha inicialmente y funcionando. Cuando nuevos nodos entran a la red, también crean una nueva blockchain con un bloque génesis hasta que otros nodos de la red se sincronicen Su blockchain más actualizada.

Ahora que su clase `Blockchain` está configurada, puede acceder a ella en Otros archivos. Agregar importación `Blockchain` desde `"/src/blockchain.js"` en la parte superior de `index.js` para importar el Clase `Blockchain` . Luego, agrega `const blockchain = new`.

`Blockchain()` justo encima de donde asignas `marketplaceNode` en la función `initializeNode` y agregue `blockchain` como el tercero parámetro al crear una instancia de su objeto `MarketplaceNode` . Esto... Inicializar un nuevo objeto `Blockchain` junto con su `marketplaceNode` cuando te conectas a la red.

Los siguientes pasos para utilizar estas nuevas clases son configurar el Funciones de transmisión y rutas para sincronizar su blockchain con otras nodos en la red. Agregue el código del **Ejemplo 12-10** a su Clase `MarketplaceNode` en `marketplaceNode.js`. Este código define `broadcastBlockchain`, que utiliza la función de transmisión para sincronizar la cadena de bloques de un nodo determinado en todos los demás nodos.

Ejemplo 1210. Creando la función `broadcastBlockchain` en `marketplaceNode.js`

```
...
transmisión asíncronaBlockchain() 1
{ this.broadcast("sync-blockchain", { cadena:
this.blockchain.chain }); } 2
```

...

- ¹ Define la función broadcastBlockchain .
- ² Llame a la función de transmisión con la ruta y la cadena de bloques de sincronización como parámetros de datos.

A continuación, define la ruta para aceptar la llamada a la API de sincronización de blockchain agregando el código del **Ejemplo 12-11** a indexjs. Este controlador de ruta de Fastify recibe la cadena de bloques de una blockchain mediante el cuerpo de la solicitud. Extraes el valor de esa cadena y actualizas la blockchain de tu marketplaceNode con ella. De esta forma, cada nodo de la red tendrá cadenas sincronizadas y actualizadas.

Ejemplo 1211. Definición de la ruta /sync-blockchain en index.js

...

```
fastify.post("/sync-blockchain", async (solicitud, respuesta) => {
1  const { cadena } = solicitud.cuerpo; 2
  marketplaceNode.blockchain = nueva Blockchain( 3
    cadena,
    marketplaceNode.blockchain.pendingTransactions );

  responder.enviar({ mensaje: "Blockchain sincronizado", blockCount:
cadena.longitud }); 4
});
```

...

- ¹ Define la ruta POST /sync-blockchain usando Fastify.
- ² Desestructurar el valor de la cadena del cuerpo de la solicitud.
- ³ Actualice la cadena de bloques del nodo del mercado pasando la cadena recibida y las transacciones pendientes existentes.

- ④ Envíe una respuesta JSON confirmando que la cadena de bloques se sincronizó.

Para probar esto, coloca `this.broadcastBlockchain()` en `registerNode` de tu clase `MarketplaceNode` y cierra la sesión con `console.log(`Syncing Blocks ${JSON.stringify(chain)}`)` en tu ruta `/sync-blockchain` de `indexjs` para ver el resultado de las cadenas sincronizadas. El resultado debería ser similar a `[{"timestamp":1686164334757,"transactions": [{"previousHash":null,"hash":"fd4b...", "nextHash" :null}]]`. Esto confirma que el bloque génesis se está sincronizando en todos los nodos recién conectados.

Con la cadena de bloques configurada, es hora de introducir transacciones y habilitar los nodos para que las usen en toda la red. Una vez verificada la nueva transacción, el código comprueba si la longitud del array `this.blockchain.pendingTransactions` es mayor que 4. De ser así, llama al método `"mine"` del nodo (este método aún no se ha definido) y registra el resultado en la consola. Una vez completado, se llama al método `"broadcastBlockchain"` para difundir la cadena de bloques actualizada a otros nodos, y la función devuelve la cadena `"Transacción procesada"` como resultado de la función `"processTransaction"`.

1212. Definición del método `processTransaction` en el ejemplo `marketplaceNode.js`

```
...
proceso asíncrono Transacción(transacciónHash) { ①
  transacción constante = nueva Transacción(HashTransacción); id ②
  constante = HashTransacción.id || idTransacción; si
  (tipoTransacción === "COMPRAR") { si ③
    (este.saldo < precioTransacción) devolver
    "Saldo insuficiente"; this.balance ④
    -= transaction.price; try { await axios.post(`${
      {transaction.sender}/payment`, { ⑤
```

```

        precio: transacción.precio, }); }

    catch (e)
    { console.log(e.message);

    } console.log({ bal: this.balance });

    } this.songs[id] = transacción;
    this.blockchain.pendingTransactions.push(transacción); if
    (this.blockchain.pendingTransactions.length > 4) {
        console.log(espera esto.mío());

    } await this.broadcastBlockchain(); return
    "Transacción procesada";
}
...

```

- 1 Define el método asíncrono processTransaction para manejar nuevas transacciones.
- 2 Crear una instancia de un nuevo objeto Transacción a partir del hash proporcionado.
- 3 Proceda sólo si el tipo de transacción es "COMPRAR".
- 4 Verifique si el saldo es suficiente para completar la compra.
- 5 Envía una solicitud de pago al nodo vendedor con el monto de la transacción.
- 6 Registra el saldo actualizado después de deducir el precio.
- 7 Almacene la transacción en el registro de canciones local usando su ID.
- 8 Añade la transacción a la lista de transacciones pendientes de blockchain.
- 9 Extraer un nuevo bloque si hay más de 4 transacciones pendientes.
- 10 Transmitir la cadena de bloques actualizada a los nodos pares.
- 11 Devuelve un mensaje indicando el resultado de la transacción.

En la aplicación de mercado blockchain, las transacciones se gestionan según la lógica de enrutamiento definida en el archivo `index.js`. El sistema admite las rutas `/sell` y `/buy`. Cuando un usuario inicia una transacción `/sell`, debe especificar el título de la canción y su precio. Por el contrario, cuando se activa una transacción `/buy`, el usuario proporciona el ID de la canción, que también sirve como ID de la transacción. Ambas rutas invocan la función `processTransaction` en la aplicación Node. Esta función añade la transacción a la lista de transacciones pendientes de la blockchain, donde espera su validación antes de ser confirmada en la cadena. El ejemplo 12-13 lo muestra con más detalle.

Ejemplo 12-13. Definición de la ruta `/pago` en `index.js` usando Fastify

```
...
fastify.post("/pago", async (solicitud, respuesta) => { const { precio } ❶
  = solicitud.cuerpo; ❷
  marketplaceNode.balance += precio; ❸
  responder.enviar({ mensaje: `Nuevo saldo
  ${marketplaceNode.balance} ` }); }); ❹
...

```

- ❶ Define la ruta POST `/pago` usando Fastify.
- ❷ Extraiga el valor del precio del cuerpo de la solicitud.
- ❸ Aumenta el saldo del nodo por la cantidad recibida.
- ❹ Enviar respuesta indicando el saldo actualizado.

Si bien la implementación mostrada en el Ejemplo 12-14 está optimizada intencionalmente y carece de medidas de seguridad a nivel de producción, ilustra claramente el mecanismo central de intercambio que impulsa el mercado blockchain. El método `mineBlock` encapsula el proceso de minería, convirtiendo un conjunto de transacciones pendientes en un bloque validado que se añade permanentemente a la cadena.

Esto no solo simula el concepto de prueba de trabajo utilizado en las blockchains del mundo

pero también refuerza la idea de que cada transacción (ya sea de venta, compra o pago) debe ser minada formalmente y anexada al libro contable para completar el ciclo de intercambio y asegurar la consistencia en toda la red distribuida.

Ejemplo 1214. Definición del método mineBlock en blockchain.js

```
...
async mineBlock() { const ❶
  targetPrefix = "0".repeat(this.dificultad); deja que nonce = 0; deja ❷
  que hash = "",

  mientras (hash.substring(0, this.dificultad) !==
targetPrefix) { nonce++; ❸
  hash =
    createHash("sha256").update(JSON.stringify(this.chain)
      + nonce).digest("hex");

}

const previousBlock = this.getLatestBlock(); const ❹
newBlock = new Block(this.pendingTransactions,
previousBlock.hash); ❺
previousBlock.nextHash = newBlock.hash;
this.chain.push(newBlock); ❻
this.pendingTransactions = []; } ❼
```

- ...
- ❶ Define el método mineBlock para convertir las transacciones pendientes en un nuevo bloque.
 - ❷ Establezca el prefijo objetivo en función de la dificultad.
 - ❸ Realizar un bucle hasta encontrar un hash que comience con el prefijo correcto.
 - ❹ Recuperar el bloque más reciente.
 - ❺ Crea un nuevo bloque a partir de transacciones pendientes y vincúlalo al bloque anterior.

- ⑥ Empujar el nuevo bloque hacia la cadena de bloques.
- ⑦ Borrar las transacciones pendientes una vez finalizada la minería.

Ejemplo 12f5. Definición del método mine en marketplaceNode.js

```
...
asíncrono mío() { ①
  precio constante = este.blockchain.miningReward; ②
  const recompensa = nueva Transacción({ ③
    remitente: this.peers[0],
    destinatario: this.url,
    precio,
    tipo de transacción: "MÍO",
  });

  esta.blockchain.pendingTransactions.push(recompensa);
  esperar esto.blockchain.mineBlock(); ④
  este.balance += precio; ⑤
  devolver "Minería completada"; ⑥
}
```

- ① Define el método de mina para recompensar un nodo.
- ② Almacene el monto de la recompensa minera.
- ③ Crear una transacción de recompensa minera.
- ④ Desencadenar el proceso de minería.
- ⑤ Añade la recompensa al saldo del minero.
- ⑥ Devolver un mensaje de finalización.

Ejemplo 12f6. Definición de la ruta /sell en index.js usando Fastify

```
...
fastify.post("/sell", async (solicitud, respuesta) => { ①
  const { precio, tituloDeCancion } = request.body; ②
  esperar marketplaceNode.processTransaction({ ③
```

```

    precio,
    título de la
    canción, remitente:
    marketplaceNode.url, tipo de

```

```

    transacción: "VENDER", }); responder.enviar({ mensaje:
    "Canción en lista" }); });

```

...

- 1 Define la ruta POST /sell usando Fastify.
- 2 Extraiga el precio y el título de la canción de la solicitud.
- 3 Procesar la transacción como tipo "VENTA" .
- 4 Responder al cliente confirmando el listado.

Ahora su aplicación está lista para gestionar solicitudes de venta de canciones en los nodos participantes en la red blockchain. Cuando se alcanza el punto final de un nodo, su URL se utiliza como remitente en la transacción.

Ejemplo 1217. Definiendo la ruta /buy en index.js usando Fastify

...

```

fastify.post("/comprar", async (solicitud, respuesta) => { const { id }
    = solicitud.cuerpo; const transacción
    = marketplaceNode.canciones[id]; si (!transacción) {
    devolver respuesta.send({ mensaje: "No existe ninguna canción con ese ID"
    }); }

```

```

5 const resultado = await marketplaceNode.processTransaction({
    id: transaction.id, precio:
    transaction.price, songTitle:
    transaction.songTitle, vencimiento:
    transaction.expiration, destinatario:
    transaction.sender, remitente:
    marketplaceNode.url, transactionType:
    "BUY", });

```

```
responder.enviar({ mensaje: resultado }); }); ❸
```

```
...
```

```
❶
```

Define la ruta POST /buy para comprar canciones.

```
❷
```

Extraiga el ID de la canción del cuerpo de la solicitud.

```
❸
```

Busque la canción en el registro local.

```
❹
```

Responder con un error si la canción no existe.

```
❺
```

Procesar una transacción de "COMPRA" con detalles de canción y nodo.

```
❻
```

Devuelve el resultado del procesamiento de la transacción.

Para probar esto, use una herramienta como cURL o Postman para enviar una solicitud /sell seguida de una /buy . Intente enviar un ID no válido en /buy para probar la gestión de errores y observe que las nuevas canciones se añaden solo después de ser extraídas en un bloque. El método availableSongs de la clase MarketplaceNode (Ejemplo 12-18) admite esta funcionalidad al devolver una matriz filtrada de las canciones actualmente en venta.

Escanea el objeto de canciones del nodo y extrae únicamente las transacciones marcadas con el tipo "VENTA" , devolviendo su ID, título y precio. Esto garantiza que los clientes y las interfaces puedan consultar qué artículos están realmente disponibles en el mercado, evitando confusiones o transacciones no válidas.

1218. Definición del método availableSongs en Example

marketplaceNode.js

```
...
```

```
availableSongs() { return ❶
```

```
  Object.values(this.songs) .filter((transaction) ❷
```

```
    => transaction.type === "VENDER") .map(({ id, titulocancion, precio }) =>
      [id, titulocancion, precio]); }
```

```
...
```


- ❶ Define el método `availableSongs` en tu clase `MarketplaceNode`.
- ❷ Devuelve una matriz de canciones cuyo tipo de transacción más reciente es VENTA.

Para que la lista de canciones disponibles sea accesible en toda la red, se introduce una ruta `/songs` que expone estos datos como un punto final de API simple (Ejemplo 12-19). Cuando se realiza una solicitud GET a esta ruta, se invoca el método `availableSongs` desde la instancia `marketplaceNode` y se devuelve el resultado como una matriz JSON. Esto permite que los clientes o interfaces externas recuperen una lista actualizada de todas las canciones actualmente en venta, cada una representada por su ID, título y precio.

Ejemplo 12-19. Definición de la ruta `/songs` en `index.js`

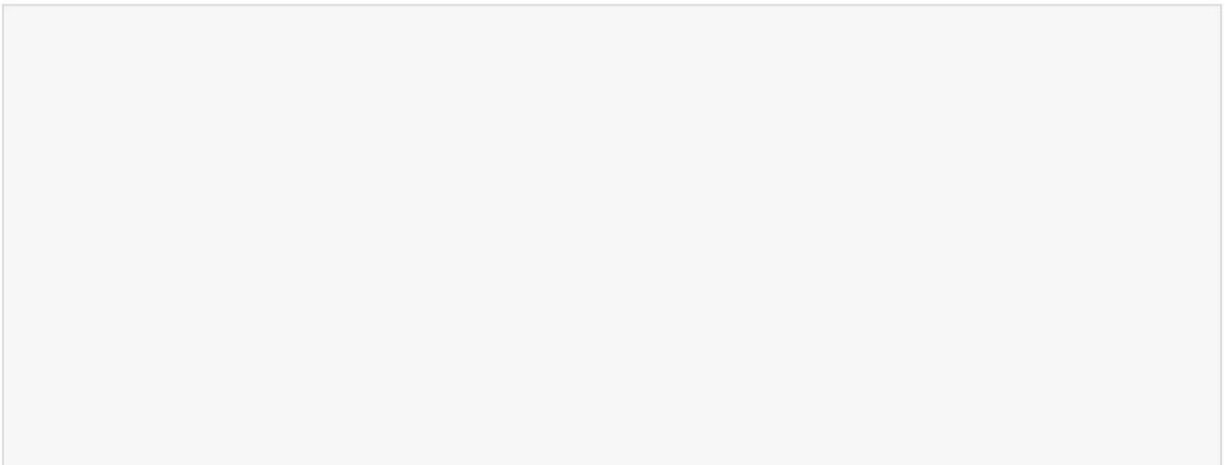
```
...
fastify.get("/canciones", async (solicitud, respuesta) => {
  responder.enviar({ canciones: marketplaceNode.availableSongs() });
});
...
```

- ❶ Define la ruta GET `/songs` para devolver todas las canciones a la venta.
- ❷ Devuelve JSON con la lista de canciones disponibles grabadas en tu `marketplaceNode`.

Puedes probar el endpoint creando primero una canción con la ruta `/sell` y luego consultando `/songs` para confirmar que aparece la lista. Si no hay canciones disponibles, la respuesta será un array vacío. Esta ruta es especialmente adecuada para la integración con una interfaz de usuario, donde las listas de canciones se pueden obtener y mostrar dinámicamente, lo que mejora la visibilidad y la usabilidad en el mercado descentralizado.

Ejecución del ejemplo real. Para ejecutar este ejemplo en un contexto distribuido real, deberá asegurarse de que los nodos participantes permanezcan sincronizados. Esto se gestiona parcialmente mediante la difusión del estado de la cadena de bloques y las listas de canciones compartidas entre pares. Por ejemplo, al llamar a `await this.broadcast("sync-blockchain", { chain: this.blockchain.chain, songs: this.songs })`; se comparten los datos más recientes de la cadena y las canciones a través de la red. Los nodos receptores pueden entonces fusionar los nuevos listados con su tienda local usando una lógica como `marketplaceNode.songs = { ...marketplaceNode.songs, ...songs }`;

Sin embargo, la implementación actual es simplificada y permite mejoras significativas. Las iteraciones futuras deberían centrarse en asegurar la red mediante la autenticación de los nodos antes de permitirles unirse o contribuir a los listados. La introducción de compatibilidad con transacciones reversibles también podría ser útil en casos de disputa o fraude, ofreciendo mecanismos de reversión. Una lógica de minería más robusta debería reflejar los algoritmos de consenso utilizados en las cadenas de bloques de producción (por ejemplo, prueba de trabajo o prueba de participación), en lugar de validar las transacciones instantáneamente con una llamada a una función local. Además, reemplazar la comunicación HTTP simple con protocolos cifrados como HTTPS o WebSockets puede mejorar la privacidad y la fiabilidad. Finalmente, la gestión de las fechas de vencimiento de los listados ayudaría a aplicar ofertas realistas con plazos definidos, haciendo que el mercado sea más dinámico y práctico.



EJERCICIOS DEL CAPÍTULO

1. Agregar lógica de expiración de la canción:

- a. Dentro de la clase Transacción , defina un método isExpired() que compare la hora actual con la marca de tiempo de expiración .
- b. En el controlador de ruta /songs , filtre cualquier canción cuya transacción asociada haya expirado utilizando isExpired().
- c. Opcional: agregue un registro para mostrar las canciones vencidas que se están eliminando del mercado activo.
- d. Prueba creando una canción con una expiración corta. y confirmar que ya no aparece una vez transcurrido el tiempo de caducidad.

CONSEJO

La lógica de expiración es esencial para gestionar activos sensibles al tiempo, como el audio de acceso anticipado en los mercados digitales.

2. Transmitir listados de canciones con la blockchain:

- a. Actualice el método broadcastBlockchain en MarketplaceNode para incluir también this.songs en la carga de transmisión:

```
await this.broadcast("sync-blockchain", { chain: this.blockchain.chain, songs: this.songs }).
```
- b. En su ruta /sync-blockchain , fusione el
Las canciones entrantes se convierten en canciones del nodo local

```
objeto que utiliza: marketplaceNode.songs =  
{ ...marketplaceNode.songs, ...songs };
```

- c. Pruebe enumerando una canción de un nodo y confirmando que aparece en la lista de canciones de otro nodo después de la sincronización.

Este ejercicio mejora la consistencia entre los nodos al sincronizar los datos de canciones compartidas junto con el estado de la cadena.

Resumen

En este capítulo, usted:

- Creó un mercado de música descentralizado usando Node
- Componentes centrales definidos como MarketplaceNode, Clases de blockchain, bloques y transacciones
- Transacciones gestionadas con lógica de validación y hash criptográfico
- Se implementaron mecanismos de minería y consenso entre nodos distribuidos.
- Estado de la cadena sincronizado y listados de canciones en toda la red
- Se exploraron mejoras como el vencimiento, la transmisión de listados y la integración de contratos inteligentes con herramientas Web3.

Capítulo 13. Creación de un asistente de aprendizaje impulsado por IA con la API Gemini de Google

Este capítulo cubre lo siguiente:

- Configuración de una aplicación Node para interactuar con Google API de Géminis
- Implementación de asistencia de aprendizaje impulsada por IA para el aprendizaje técnico
- Integración de una base de datos de perfiles de usuario para realizar un seguimiento del progreso del aprendizaje

Los Modelos de Lenguaje Grandes (LLM), como Gemini de Google o ChatGPT de OpenAI, están transformando rápidamente la forma en que las aplicaciones interactúan con los usuarios, permitiendo experiencias más contextuales, inteligentes y adaptativas. Los sistemas basados en IA ya no se limitan a responder preguntas; ahora pueden analizar el comportamiento del usuario, monitorizar su progreso de aprendizaje y ofrecer recomendaciones personalizadas, lo que los convierte en una herramienta invaluable para la formación técnica y el desarrollo de habilidades. Al integrar los LLM en las aplicaciones, los desarrolladores pueden crear herramientas dinámicas que mejoran la interacción del usuario, ofrecen una formación más eficaz y se adaptan a los estilos individuales de aprendizaje y comunicación.

En este capítulo, crearás una aplicación de agente de IA diseñada para ayudar a los usuarios a prepararse interactivamente para entrevistas técnicas y a aprender conceptos de programación. El asistente, impulsado por IA, guiará a los usuarios en su proceso de aprendizaje, adaptándose a sus fortalezas, debilidades y estilos de aprendizaje. Aprenderás sobre integraciones de API, interacciones de IA contextuales y el uso de una base de datos para almacenar perfiles de aprendizaje de los usuarios.

HERRAMIENTAS Y APLICACIONES UTILIZADAS EN ESTE CAPÍTULO

Antes de comenzar, asegúrese de instalar y configurar las herramientas y aplicaciones necesarias para este proyecto. Las instrucciones de instalación de Node.js, Fastify y VS Code se encuentran en [el Capítulo 1](#), mientras que los pasos de inicialización del proyecto, como la configuración de la estructura de directorios, la configuración de package.json y el uso de sintaxis moderna, se describen en [el Apéndice A](#). Las instrucciones para instalar y usar Postman se encuentran en [el Apéndice B](#). Para una explicación más detallada de SQLite, consulte [el Apéndice C](#). Para obtener instrucciones sobre cómo configurar su cuenta de Google Gemini y obtener una clave API, consulte [el Apéndice E](#). Una vez completado, vuelva aquí para continuar. Desarrollar un proyecto desde cero ayuda a profundizar su comprensión de cada componente, lo que le brinda mayor control y flexibilidad a medida que avanza.

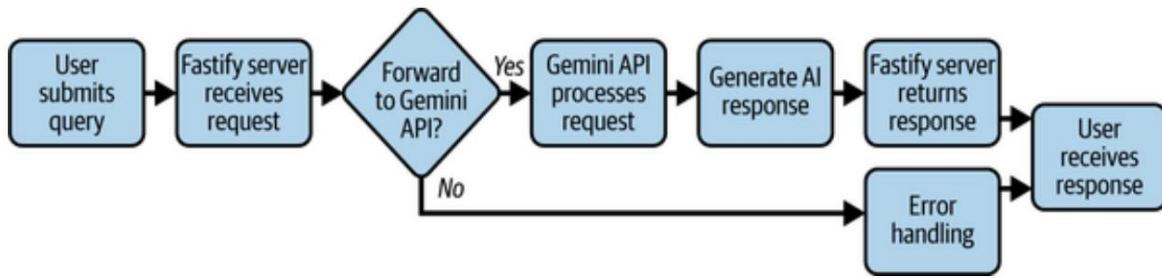
Tu mensaje: Un

servicio en línea llamado Interview Atlas, que ayuda a los ingenieros a dominar las habilidades técnicas de programación y a prepararse para las entrevistas, desea integrar un asistente inteligente. Te han contratado para diseñar una herramienta de IA con la ayuda de las API de IA existentes para ayudar a los usuarios a navegar por las entrevistas técnicas y aprender conceptos de programación. A medida que los usuarios progresan, la IA los guiará, ofreciéndoles explicaciones, preguntas de práctica y perspectivas sobre sus patrones de aprendizaje. La IA también debe adaptarse a las necesidades de los usuarios, identificando las áreas donde tienen dificultades y haciendo recomendaciones para mejorar.

El equipo de Interview

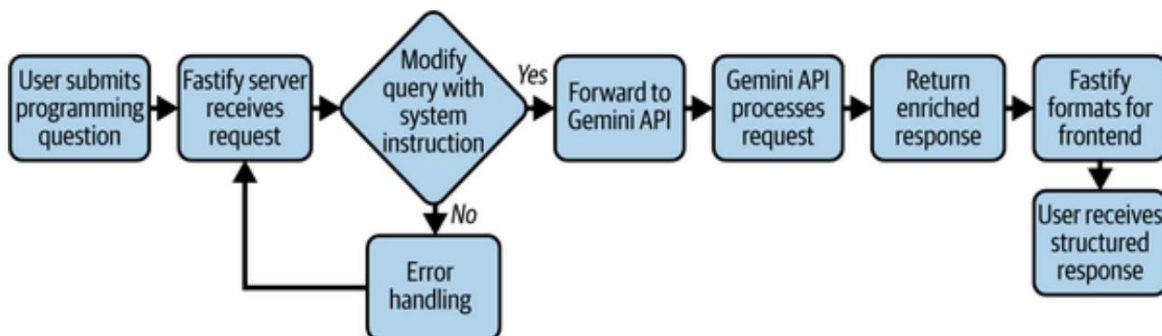
Atlas **quiere** que diseñes un sistema que utilice un LLM externo existente. Te sugieren integrar una aplicación de Node con la API de Google Gemini para crear este asistente de IA. Al hacerlo,

Su aplicación se comunicará con la API de Gemini durante cada usuario Consultar y mostrar la respuesta de IA al usuario. Para empezar, Plan para crear una aplicación Node simple que se conecte a la API de Gemini y devuelve una respuesta. La aplicación se creará con Fastify, framework, que es un marco web ligero y eficiente para Nodo (Figura 13-1).



Cifra 131. Arquitectura del sistema para la primera fase del asistente de IA

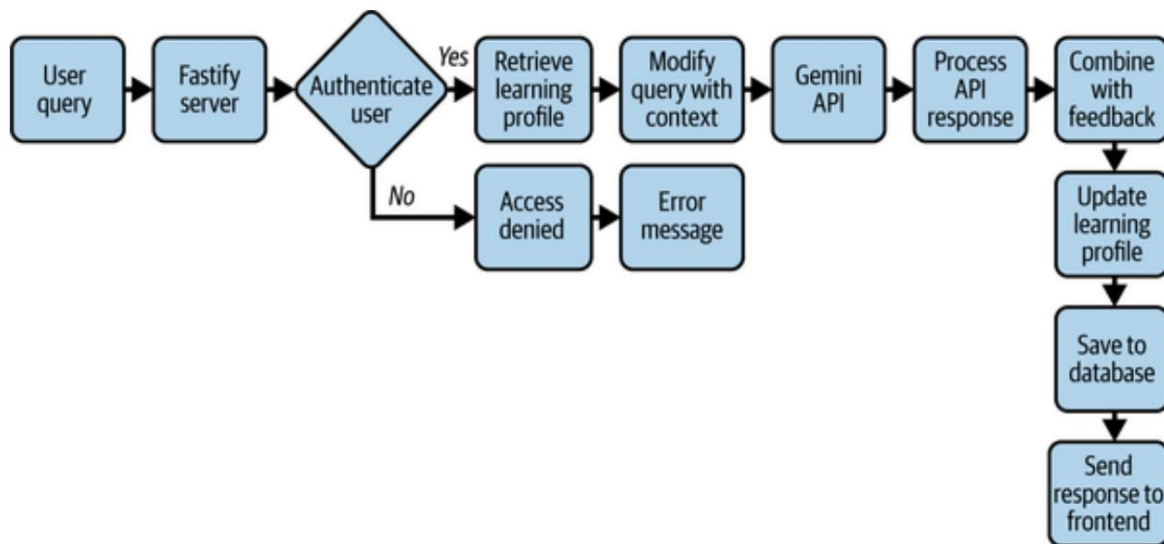
Una vez que se complete la primera fase, planea curar la IA. respuestas para hacerlas más relevantes para el proceso de aprendizaje. Esto Implica diseñar un mensaje que indique a la IA que actúe como un sistema de aprendizaje. Asistente, proporcionando explicaciones, ejemplos y comentarios sobre el usuario. Consultas (Figura 13-2). Cada solicitud enviada a la IA será diseñada para que coincida con el contexto de un asistente de aprendizaje técnico.



Cifra 132. Arquitectura del sistema para la segunda fase del asistente de IA

Para completar la configuración, incluya una base de datos para almacenar perfiles de usuario. y hacer un seguimiento de su progreso de aprendizaje a lo largo del tiempo. También integra un básico medida de autenticación para que los usuarios puedan registrarse, iniciar sesión y guardar sus datos Historial de interacciones. De esta manera, los usuarios podrán estudiar ingeniería.

conceptos, prepararse para entrevistas técnicas y recibir capacitación personalizada retroalimentación sobre su recorrido de aprendizaje (Figura 13-3).



Cifra 133. Arquitectura del sistema para la fase tres del asistente de IA

Con el plan de proyecto en marcha, comienza a implementar la IA.

Asistente como una aplicación Node.

Obtenga programación

Para comenzar a desarrollar su aplicación, cree una nueva carpeta de proyecto llamada `interview_atlas_ai`. Navega a la carpeta de tu proyecto en tu Línea de comandos y ejecute `npm init`. Este comando inicializa su Aplicación de nodo. Puede presionar Enter durante los pasos de inicialización para Acepte los valores predeterminados. El resultado de estos pasos es la creación de un archivo llamado `package.json`. Este archivo le indicará a su aplicación Node cualquier configuraciones o scripts necesarios para ejecutarse correctamente.

A continuación, crea un archivo llamado `index.js` dentro de la carpeta de tu proyecto. Este archivo es El punto de entrada para tu aplicación. Para tu primera iteración, probarás tu Conexión de API a la API de Gemini de Google. Para ello, usarás el Biblioteca `axios` para realizar solicitudes HTTP. También usarás `dotenv` biblioteca para administrar sus variables de entorno, como su clave API. Instale las dos bibliotecas ejecutando `npm install axios@^1.11.0`

dotenv@^17.2.1 en el directorio raíz de su proyecto en su línea de comando.

En tu editor de texto, agrega un archivo llamado env. Este archivo almacenará tus variables de entorno (contraseñas y claves API) que permiten que tu proyecto se ejecute como servidor sin revelar información confidencial de tu código. En este archivo, agrega tu clave API para la API de Google Gemini. Puedes encontrar esta clave en tu Google Cloud Console. El archivo env debería tener un aspecto similar al siguiente (para la clave API de ejemplo, GISaSrW-fsM3TNYp2UkmucNuSnkPKmGUtIsaDF4):

```
CLAVE DE API DE GEMINI=GISaSrW-fsM3TNYp2UkmucNuSnkPKmGUtIsaDF4
```

Esta es la clave API que utilizará durante cada solicitud de API externa realizada desde su aplicación.

CONSEJO

Esta clave está vinculada directamente a tu cuenta de Google y, al igual que otros servicios de IA, puede generar cargos por uso excesivo. Asegúrate de consultar tu consola de Google Cloud para conocer los límites de uso o la información de facturación.

A continuación, dirígete al archivo index.js de tu proyecto y añade el código del **Ejemplo 13-1**. Este código define una función que envía solicitudes de usuario al modelo de IA Gemini de Google y devuelve una respuesta generada.

La API_URL se almacena como una constante, integrando la clave de API de forma segura desde las variables de entorno para evitar la codificación rígida de credenciales confidenciales. La función generateResponse realiza una solicitud POST asíncrona mediante axios, formateando la solicitud de usuario según la estructura requerida por la API. El cuerpo de la solicitud POST para la API de Gemini se estructura como un objeto que contiene una matriz de contenidos, donde cada elemento representa un intercambio de mensajes con la IA, especificando un rol (p. ej., "usuario") y una matriz de partes.

Contiene la entrada de texto real. Este formato es crucial porque permite conversaciones multi-turno, mantiene el contexto y garantiza que las indicaciones estén correctamente segmentadas para su procesamiento, lo que permite al modelo generar respuestas contextuales.

NOTA

Cada API de IA tiene su propia estructura para enviar solicitudes. El formato del cuerpo POST de la API de Gemini es diferente al de la API GPT-4 de OpenAI. Asegúrese de consultar la documentación de la API para conocer el formato correcto.

La respuesta se procesa mediante encadenamiento opcional para extraer el texto generado por la IA y, si no se devuelve ningún resultado, se proporciona un mensaje de respaldo. Si la solicitud falla, la función gestiona los errores correctamente devolviendo un mensaje de error detallado, lo que evita fallos y facilita la depuración.

NOTA

En versiones anteriores de Node, es posible que haya usado una expresión de función de invocación inmediata (IIFE) asíncrona para llamar a `generateResponse()`. En Node 18+ con compatibilidad con módulos ES, puede usar la función `await` de nivel superior.

Ejemplo de prueba de una solicitud de API de IA en `index.js` [import axios from](#)

```
"axios"; import dotenv from "dotenv";
```

1

```
dotenv.config();
```

```
constante BASE =
```

```
"https://generativelanguage.googleapis.com/v1beta/models"; constante MODELO = "gemini-2.0-  
flash:generateContent"; constante API_URL = `${BASE}/${MODELO}? clave=$  
{process.env.GEMINI_API_KEY}`;
```

2

```

función asíncrona generateResponse(prompt) {
  try
    { const { data } = await axios.post( API_URL,
      { contenido:
        [{ rol: "usuario", partes: [{ texto: prompt }]}
      ] },
      { encabezados: { "Tipo-de-contenido": "application/json" } } );

    devolver datos?.candidatos?.[0]?.contenido?.partes?.[0]?.texto || "Sin respuesta.";

  } catch (error) { return
    `Error: ${error.response?.data || error.message}`; }
}

const prompt = "En una oración, explique Node."; const response =
await generateResponse(prompt); console.log(" Respuesta de AI:",
response);

```

- ❶ Importa axios para realizar solicitudes HTTP y dotenv para cargar variables de entorno
- ❷ Almacena la URL de la API de Gemini e incorpora la clave API
- ❸ Define una función asincrónica para enviar la solicitud.
- ❹ Envía una solicitud POST con el mensaje del usuario como carga útil de la solicitud
- ❺ Establece Content-Type en application/json para garantizar el formato de solicitud adecuado
- ❻ Extrae la respuesta generada por IA a partir de los datos devueltos.
- ❼ Maneja los errores con elegancia, devolviendo un mensaje de error significativo
- ❽ Define el mensaje que el usuario debe enviar a la IA.

- 9 Llama a la función `generateResponse()` usando `await` de nivel superior .
- 10 Registra la respuesta de la IA en la consola.

NOTA

Los nombres de los modelos Gemini pueden cambiar con el tiempo. Puedes encontrar la lista actual en la [referencia de modelos de Google Gemini](#).

Ahora ejecute `node index.js` en el directorio raíz de su proyecto desde la línea de comandos para ejecutar este código. Debería ver un resultado que aísla la respuesta de LLM a su solicitud, con un texto como "Respuesta de IA: Node.js es un entorno de ejecución de JavaScript que permite a los desarrolladores ejecutar código JavaScript en el servidor, lo que permite la creación de aplicaciones web escalables y eficientes".

GRANDES MODELOS DE LENGUAJE Y API DE IA

Los Modelos de Lenguaje Grande (LLM) son sistemas avanzados de IA entrenados con grandes cantidades de datos de texto para comprender y generar lenguaje similar al humano. Utilizan técnicas de aprendizaje profundo, en particular redes neuronales, para predecir las palabras más probables en una secuencia basándose en la información recibida. Los LLM impulsan las API de IA, que permiten a los desarrolladores interactuar con estos modelos mediante solicitudes programáticas.

NOTA

Una red neuronal es un modelo de aprendizaje automático inspirado en el cerebro humano, que consta de capas de nodos interconectados (neuronas) que procesan datos de entrada, aprenden patrones y hacen predicciones a través de conexiones ponderadas y funciones de activación.

Usar un LLM es como pedir ayuda a una sala llena de miles de millones de personas en lugar de a una sola: en lugar de depender de una sola opinión, la IA se basa en innumerables conversaciones, libros, idiomas y fuentes para ofrecer la respuesta más relevante. No piensa como un humano, pero predice la mejor respuesta posible basándose en patrones aprendidos de grandes cantidades de texto.

Al enviar una solicitud a una API de IA, el modelo procesa la solicitud, analiza el texto y genera una respuesta relevante. Las API de IA permiten una integración fluida de las capacidades de IA en las aplicaciones, proporcionando funcionalidades como completar texto, resumir y responder preguntas. A continuación, se presentan algunos conceptos y componentes clave relacionados con el uso de LLM y API de IA:

Tokenización

Los LLM descomponen el texto de entrada en unidades más pequeñas llamadas tokens (palabras, subpalabras o caracteres). El modelo procesa estos

tokens y predice el siguiente token en la secuencia para generar respuestas coherentes.

Arquitectura del transformador

La mayoría de los LLM modernos utilizan una red neuronal basada en transformadores, que se apoya en mecanismos como la autoatención para comprender las relaciones entre las palabras, incluso en pasajes largos de texto.

Inferencia

Cuando una API de IA recibe una solicitud, el modelo ejecuta un proceso de inferencia, utilizando sus patrones aprendidos para generar una respuesta probable. Esto es diferente del entrenamiento, ya que el modelo aplica el conocimiento existente en lugar de aprender nueva información.

Temperatura y tokens máximos

Las API de IA a menudo proporcionan parámetros para controlar la generación de respuestas. La temperatura afecta la aleatoriedad (los valores más altos hacen que las respuestas sean más creativas), mientras que max_tokens limita la longitud de la respuesta.

Las API de IA como GPT-4 de OpenAI, Gemini de Google y Claude de Anthropic proporcionan interfaces estructuradas para acceder a las capacidades de LLM en aplicaciones en tiempo real.

Para obtener más detalles sobre las API de IA, visite la [documentación de la API de OpenAI](#). o la [documentación de Google Vertex AI](#).

Una vez establecida la conexión a la API de IA, puede personalizar el asistente de IA para proporcionar respuestas más relevantes para aprender programación y prepararse para entrevistas técnicas. Esto implica modificar la estructura de la solicitud para incluir una instrucción que guíe el comportamiento de la IA.

Personalización del Asistente de IA para el aprendizaje Asistencial

La ingeniería de indicaciones consiste en diseñar instrucciones precisas y eficaces para guiar a modelos de IA como Gemini en la generación de respuestas útiles y precisas. Dado que los LLM generan texto basado en patrones de entrada, la estructura de una indicación influye significativamente en la calidad y la relevancia del resultado. Al elaborar cuidadosamente una indicación, los desarrolladores pueden controlar el comportamiento de la IA, proporcionar el contexto necesario y refinar su estilo de respuesta.

INGENIERÍA INMEDIATA: CREANDO UNA IA EFICAZ

INSTRUCCIONES

Mediante la ingeniería de indicaciones, los desarrolladores pueden indicar a la IA que asuma un rol específico, como profesor, programador o figura histórica. Por ejemplo, una indicación como "Eres un ingeniero de software con experiencia. Explica la recursión a un principiante" generará una respuesta más estructurada y con mayor conocimiento que simplemente preguntar "¿Qué es la recursión?".

Además, las indicaciones pueden incluir restricciones, requisitos de formato o ejemplos para ajustar el resultado.

NOTA

Una propuesta bien diseñada es como preparar una receta de cocina diferente para niños que para adultos. La receta resultante debe ser adecuada para la comprensión y el nivel de habilidad del público.

Las siguientes son algunas técnicas comunes utilizadas en la ingeniería rápida:

Incitación a desempeñar roles

Asigna a la IA una personalidad, como "Usted es un experto en ciberseguridad", lo que ayuda a dar forma a las respuestas en función de la experiencia.

Inyección de contexto

Proporciona información de fondo dentro del mensaje para ayudar al modelo a generar respuestas con el contexto relevante.

Incitación de pocos disparos

Incluye ejemplos dentro de la indicación para guiar el estilo de respuesta de la IA y el resultado esperado.

Solicitud de formato o (estructuración de salida)

Indica cómo debe formatearse la respuesta, como "Proporcione un resumen con viñetas" o "Explique utilizando una analogía simple".

Una ingeniería de indicaciones eficaz mejora la usabilidad de la IA, permitiendo respuestas más precisas, creativas y estructuradas. Es una habilidad crucial para los desarrolladores que integran la IA en aplicaciones, chatbots y flujos de trabajo de automatización.

Para obtener más información sobre ingeniería rápida, consulte [la guía de ingeniería rápida](#) de OpenAI o la documentación de Vertex AI de Google .

Para asegurarse de que el asistente de IA proporcione respuestas centradas en el aprendizaje, modifique la estructura de la solicitud para incluir una instrucción, como se muestra en [el Ejemplo 13-2](#).

En este código, se modifica `generateResponse`, que envía una solicitud al usuario a la API de IA de Gemini de Google y recupera una respuesta generada. Se añade un objeto de contenido de solicitud adicional que indica a la IA que actúe como un asistente experto especializado en temas técnicos. Esta solicitud se incluye en el cuerpo de la solicitud, lo que garantiza que la IA comprenda su función y proporcione respuestas relevantes. A continuación, se extrae la respuesta de la IA y se devuelve al usuario.

Ejemplo 132. Guiando a la IA para enseñar programación en `index.js`

```
...  
función asíncrona generateResponse(prompt) {  
  intenta  
    { const { datos } = await axios.post( API_URL, {
```

Contenido: [

```

    {
      rol: "usuario",
      partes: [{ texto: "Eres un asistente de IA que
ayuda a los usuarios
      +
      "Aprende a programar y prepárate para entrevistas técnicas".
      +
      "Proporcione explicaciones claras con ejemplos cuando
necesario." }], ❶

    { rol: "usuario", partes: [{ texto: mensaje }], }, { encabezados: ❷

    { "Content-Type": "application/json" } } );

return
( data?.candidates?.[0]?.content?.parts?.[0]?.text || "No se recibió
respuesta de la IA." ); } catch (error) {

    console.error(" Error en la solicitud de API:",
error.response?.data || error.message); return { error:
"No se
pudo obtener la respuesta de IA", detalles:
error.response?.data || error.message, };

}
}
...

```

- ❶ Instruya a la IA para que actúe como asistente de entrevista técnica, garantizando que las respuestas se centren en conceptos de programación y explicaciones claras con ejemplos.
- ❷ Envía el mensaje real del usuario, que cambia dinámicamente según lo que el usuario quiera preguntarle a la IA.

También puedes cambiar la indicación a "Explica la recursión en una sola frase". Ahora ejecuta `node index.js` de nuevo. Deberías ver una respuesta de la IA más relevante para el aprendizaje.

Conceptos de programación. Por ejemplo: «La recursión es una técnica de programación donde una función se llama a sí misma dentro de su propia definición para resolver una instancia más pequeña del mismo problema hasta alcanzar un caso base».

Configuración del servidor Fastify: Ahora que

cuenta con un asistente de IA funcional con indicaciones personalizadas, puede configurar un servidor Fastify para gestionar las solicitudes y respuestas de los usuarios. De esta forma, puede crear una aplicación web que permita a los usuarios interactuar con el asistente de IA mediante una interfaz intuitiva en lugar de la línea de comandos.

Para comenzar, instala Fastify para configurar las rutas de tu servidor. Ejecuta el siguiente comando en el directorio raíz de tu proyecto:

```
npm instalar fastify@^5.4.0
```

añadir el código del **Ejemplo 13-3** al código de `js` para incorporar Fastify. Esto, al índice, inicializa un servidor Fastify con el registro habilitado, lo que permite rastrear solicitudes y errores fácilmente. El servidor escucha en un puerto definido por el entorno (`process.env.PORT`) o el predeterminado es 3000, lo que garantiza flexibilidad para diferentes entornos de implementación. Al usar "await" dentro de una función de inicio asíncrono, el servidor puede gestionar los fallos de inicio correctamente y registrar la dirección de ejecución tras un inicio exitoso.

CONSEJO

Puede agregar la variable `PORT` a su archivo `env` para configurar un puerto personalizado para su servidor. Por ejemplo, puede agregar `PORT=4000` para ejecutar el servidor en el puerto 4000 en lugar del 3000 predeterminado.

Ejemplo 133. Iniciar un servidor Fastify con variables de entorno

```

importar Fastify desde 'fastify'; importar 1
axios desde 'axios'; importar dotenv
desde 'dotenv';

dotenv.config();

constante fastify = Fastify({ logger: true }); constante 2
PUERTO = proceso.env.PUERTO || 3000; 3

const start = async () => { try { const

    address = await fastify.listen({ port: PORT }); console.log(`Servidor 4
    ejecutándose en ${address}`); 5
  } atrapar (err) {
    console.error("El servidor no pudo iniciarse:", err); process.exit(1);
    6
  }
};

```

comenzar(); **7**

- 1** Importa Fastify para crear un servidor web
- 2** Inicializa Fastify con el registro habilitado
- 3** Establece el puerto del servidor desde el entorno o el valor predeterminado es 3000
- 4** Utiliza await para iniciar el servidor Fastify de forma asincrónica
- 5** Registra la dirección del servidor una vez que se está ejecutando
- 6** Maneja errores de inicio del servidor y sale si es necesario
- 7** Llama a la función de inicio asincrónico

A continuación, agregue el código del **Ejemplo 13-4** a index.js para gestionar las consultas del usuario. Este código define una ruta POST en /query, que permite a los usuarios enviar una solicitud al asistente de IA. Se comprueba si hay una solicitud en el cuerpo de la solicitud y, si no la hay, se devuelve un código 400 (Solicitud incorrecta).

Garantizar la correcta validación de la entrada. Si se proporciona un mensaje válido, el servidor llama a `generateResponse(prompt)`, que procesa la solicitud a través de la API de Gemini. La respuesta generada por la IA se devuelve al cliente, y cualquier error detectado durante el proceso se registra y se devuelve con un código 500 (error interno del servidor), lo que garantiza una depuración clara y un comportamiento fiable de la API.

Ejemplo de manejo de consultas de IA con Fastify

```
fastify.post("/query", async (request, reply) => { ❶
  try
    { const { prompt } = request.body; if (!prompt) ❷
      { return
        reply.status(400).send({ error: "Se requiere el mensaje" }); } ❸

    const respuesta = await generarRespuesta(prompt); ❹

    responder.enviar({ respuesta }); } ❺
  catch (error)
    { console.error(" Error de API de Gemini:", error);
      responder.status(500).enviar({ error: ❻
        "Error al comunicarse con la API de Gemini", detalles:
          error.message, });
    }
});
```

- ❶ Define una ruta POST de Fastify para manejar consultas de IA
- ❷ Extrae el mensaje del cuerpo de la solicitud.
- ❸ Devuelve un error 400 de solicitud incorrecta si no se proporciona ningún mensaje
- ❹ Llama a `generateResponse(prompt)` para obtener la salida de IA
- ❺ Envía la respuesta de IA al cliente.
- ❻ Registra errores y devuelve un error interno del servidor 500 si la solicitud falla

En este momento, puede eliminar el IIFE del código anterior, ya que ya no necesita ejecutar la función inmediatamente. También puede eliminar la instrucción `console.log` que registra la respuesta de la IA, ya que ahora se enviará de vuelta al cliente a través del servidor Fastify.

Con este código, al ejecutar `node index.js` en la línea de comandos, se iniciará el servidor Fastify, pero ya no se ejecutará ninguna consulta de API. Debería aparecer un mensaje indicando que el servidor se está ejecutando en `httplocalhost`.

```
:// :3000 .
```

Con el servidor en funcionamiento, ahora puede enviar una solicitud POST al punto final `/query` con un cuerpo JSON que contiene un mensaje de una nueva ventana de línea de comandos. Por ejemplo, intente ejecutar:

```
curl -X POST http://localhost:3000/query \-H "Content-Type:
application/json" \-d '{ "prompt": "Explique la recursión
en una sola oración con una analogía." }'
```

Recibirás una respuesta similar a la siguiente:

```
{"response":"La recursión es como un juego de muñecas rusas, donde cada muñeca
contiene una versión más
pequeña de sí misma, hasta que se llega a la muñeca más pequeña que finalmente se puede
sostener: una función
resuelve un problema dividiéndolo en subproblemas más pequeños y autosimilares hasta
que llega a un caso base
que se puede resolver directamente.\n"}
```

Su aplicación ahora está lista para gestionar consultas de usuarios a través de Internet. Sin embargo, actualmente no es posible rastrear las interacciones de los usuarios ni almacenar su progreso de aprendizaje. Para ello, deberá implementar una base de datos para gestionar los perfiles de usuario y la autenticación.

Configuración de su base de datos y usuario Autenticación

Para implementar la autenticación de usuarios, deberá crear una base de datos para almacenar los perfiles de usuario y su progreso de aprendizaje. En este ejemplo, utilizará SQLite3 como solución de base de datos. SQLite3 es una base de datos ligera basada en archivos, fácil de configurar e ideal para el desarrollo de prototipos de aplicaciones. Permite almacenar datos estructurados, como información del perfil de usuario y su progreso de aprendizaje.

Para comenzar, instale los paquetes `sqlite3` y `sqlite` ejecutando el siguiente comando en el directorio raíz de su proyecto en su línea de comandos:

```
npm install sqlite3@^5.1.7 sqlite@^5.1.1"
```

¿POR QUÉ UTILIZAR SQLITE3?

SQLite3 es una excelente opción para crear prototipos de aplicaciones en Node, ya que permite almacenar y gestionar perfiles de aprendizaje de usuarios. A diferencia de las bases de datos relacionales tradicionales, que requieren un servidor dedicado, SQLite es una base de datos autónoma basada en archivos, lo que la hace ligera, fácil de configurar e ideal para la creación rápida de prototipos.

NOTA

SQLite funciona con un único archivo de base de datos, lo que elimina la necesidad de configuraciones complejas y lo convierte en una excelente opción para el desarrollo y aplicaciones a pequeña escala.

Para un perfil de aprendizaje de usuario, SQLite3 permite el almacenamiento de datos estructurados con un esquema simple. Sigue los principios ACID (atomicidad, consistencia, aislamiento y durabilidad), lo que significa que los datos se mantienen fiables incluso durante fallos de alimentación o caídas de tensión.

Además, SQLite3 no requiere configuración ni servidor de base de datos independiente, lo que facilita su integración en una aplicación Node mediante el paquete `sqlite3` :

No requiere configuración

Se ejecuta como un solo archivo, lo que elimina la sobrecarga de administración de la base de datos.

Rápido y ligero

Ideal para almacenamiento local en un prototipo pequeño o una aplicación de prueba de concepto.

Integración sencilla

Se utiliza fácilmente con el paquete Node `sqlite3` para operaciones CRUD básicas

Almacenamiento de datos estructurados

Admite consultas SQL para administrar los perfiles de aprendizaje de los usuarios de manera eficiente

Portátil y escalable

Se puede actualizar a PostgreSQL o MySQL a medida que crece la aplicación.

NOTA

El paquete `sqlite` se utiliza para brindar soporte `async/await`, lo que facilita el trabajo con operaciones de base de datos asincrónicas.

Este proyecto requiere una base de datos, ya que se almacenarán las contraseñas cifradas, las direcciones de correo electrónico y los perfiles de aprendizaje de los usuarios. El perfil de aprendizaje incluirá información sobre las consultas previas, las fortalezas y las debilidades del usuario. Esta información ayudará al asistente de IA a proporcionar respuestas personalizadas según el perfil de aprendizaje del usuario.

Con `sqlite3` instalado, puede crear una conexión a la base de datos y definir el esquema para sus perfiles de usuario. El esquema incluirá campos para el correo electrónico, la contraseña cifrada y los datos del perfil de aprendizaje del usuario.

El perfil de aprendizaje se almacenará como un objeto JSON, lo que le permitirá actualizar y recuperar fácilmente información específica del usuario.

También implementará un sistema de autenticación básico para permitir que los usuarios se registren e inicien sesión. Esto le permitirá almacenar perfiles de usuario y realizar un seguimiento de su progreso de aprendizaje a lo largo del tiempo, distinguiendo las interacciones de los usuarios entre sí.

Para definir una conexión de base de datos y crear una tabla de usuarios , agregue el código del **Ejemplo 13-5** a un nuevo archivo llamado dbjs.

Este código inicializa una base de datos SQLite para almacenar perfiles de usuario, garantizando así que el progreso de aprendizaje y los detalles de autenticación se guarden de forma persistente. La conexión a la base de datos, dbPromise, está configurada para interactuar con un archivo local (la base de datos de usuarios). La función initializeDb garantiza la existencia de la tabla de usuarios , creándola si es necesario.

La tabla incluye un campo de identificación autoincrementable como clave principal, un ID de usuario único para evitar cuentas duplicadas. En este caso, usaremos direcciones de correo electrónico como ID. Por último, se añade una columna de contraseña para la autenticación. Además, el campo learning_profile ahora tiene el valor predeterminado "Este usuario aún no tiene un perfil de aprendizaje registrado". Esto garantiza que, al crear un nuevo usuario, su perfil contenga texto predeterminado significativo en lugar de aparecer vacío.

Una vez configurada la base de datos, se registra un mensaje de confirmación que confirma que el sistema está listo para almacenar y recuperar datos del usuario.

Ejemplo 135. Inicialización de una base de datos SQLite para perfiles de usuario

```
importar sqlite3 desde "sqlite3"; importar { open } 1
desde "sqlite"; importar ruta desde "ruta";
```

```
const dbPromise = open({ nombre de 2
  archivo: path.resolve("./users.db"), controlador:
  sqlite3.Database, });
```

```
const initializeDb = async () => { const db = await
  dbPromise;
```

```
  await db.exec(` CREAM 3
    TABLA SI NO EXISTE usuarios ( 4
      id ENTERO CLAVE PRINCIPAL AUTOINCREMENT, userId 5
      TEXTO ÚNICO, contraseña 6
      TEXTO NO NULO, learning_profile 6
      TEXTO PREDETERMINADO 'Este usuario no tiene
```

```
perfil de aprendizaje todavía.' ) 7
```

```
`);
```

```
console.log(" Tabla de usuarios inicializada"); }); 8
```

```
inicializarDb();
```

```
exportar dbPromise predeterminado ;
```

- 1 Importa sqlite3 y sqlite para habilitar la interacción con la base de datos
- 2 Abre una conexión al archivo de base de datos usersdb .
- 3 Ejecuta una consulta SQL para crear la tabla de usuarios si no existe
- 4 Define id como la clave principal, que se incrementa automáticamente para cada usuario
- 5 Garantiza que el ID de usuario sea único, lo que evita cuentas duplicadas
- 6 Almacena contraseñas en hash de forma segura (se deben evitar las contraseñas en texto sin formato)
- 7 Guarda el perfil de aprendizaje del usuario como un campo de texto
- 8 Registra un mensaje de éxito una vez que se inicializa la base de datos

Para integrar esta conexión de base de datos en su servidor Fastify, agregue "import dbPromise from "./db.js";" al inicio de su archivo index.js . Esto le permite acceder a la conexión de base de datos en toda su aplicación. Cuando sea necesario, puede llamar a " await dbPromise" para interactuar con la base de datos.

Antes de poder usar la base de datos, debe crear un sistema de registro e inicio de sesión de usuarios. Esto permitirá a los usuarios crear cuentas, iniciar sesión y almacenar sus perfiles de aprendizaje de forma segura.

Instale jsonwebtoken y bcrypt para la autenticación de usuarios. Ejecute el siguiente comando en el directorio raíz de su proyecto en su comando línea:

```
npm install jsonwebtoken@^9.0.2 bcrypt@^6.0.0
```

NOTA

La biblioteca jsonwebtoken le ayudará a crear y verificar archivos web JSON. Tokens (JWT) para sesiones de usuario seguras, mientras que bcrypt se utilizará para hash las contraseñas de los usuarios antes de almacenarlas en la base de datos.

Luego agregue el siguiente código en la parte superior de su archivo index.js para importar las bibliotecas necesarias y configure su secreto JWT:

```
importar jwt desde "jsonwebtoken";  
importar bcrypt desde "bcrypt";
```

...

```
const JWT_SECRET = proceso.env.JWT_SECRET ||  
"secretoJWTdummy";
```

NOTA

De manera similar a la clave API, el secreto JWT debe almacenarse en su archivo env . Este secreto se utiliza para firmar y verificar los JWT, lo que garantiza que solo los usuarios autorizados Los usuarios pueden acceder a rutas protegidas. Por ahora, también puedes usar una ruta ficticia. secreto para fines de prueba.

Para dar soporte a las cuentas de usuario, necesitará una forma para que los usuarios puedan...

Registre su ID (correo electrónico) y contraseña. Agregue el código del [Ejemplo 13-6](#). para index.js para implementar un punto final de registro de usuario.

Este código implementa el registro de usuarios, lo que permite a los nuevos usuarios crear una cuenta y almacenar sus credenciales de forma segura. Cuando un usuario introduce su correo electrónico y contraseña, el servidor comprueba si se han proporcionado ambos campos. La contraseña se cifra mediante bcrypt antes de almacenarse en la base de datos SQLite, lo que garantiza que las credenciales del usuario no se almacenen en texto plano.

Además, un perfil de aprendizaje se inicializa como un objeto vacío, lo que permite a la IA rastrear el progreso del usuario a lo largo del tiempo. Una vez registrado, el servidor genera un token JWT, que se envía en la respuesta, lo que permite al usuario autenticar futuras solicitudes. Si se produce algún error, se registra y se devuelve una respuesta de error 500.

NOTA

El token JWT es una forma segura de gestionar las sesiones de usuario. Contiene información codificada sobre el usuario y está firmado con una clave secreta, lo que permite al servidor verificar su autenticidad sin necesidad de almacenar datos de la sesión. El cliente lo utiliza para autenticar las solicitudes a rutas protegidas.

Ejemplo 136. Punto final de registro de usuario

```
...
fastify.post("/register", async (solicitud, respuesta) => { const { correo ①
  electrónico, contraseña } = solicitud.cuerpo; si (!correo ②
  electrónico || !contraseña) { devolver
    respuesta.estado(400).send({ error: "Se requiere correo electrónico
y contraseña" }); ③
  }

  prueba
  { const db = await dbPromise; const ④
    hashedPassword = await bcrypt.hash(contraseña, 10);
  } ⑤

  await
    db.run( "INSERTAR EN usuarios (ID de usuario, contraseña,
```

```

perfil_de_aprendizaje) "      +
    "VALORES (?, ?, ?)",
    [ ❸
      correo
      electrónico,
      contraseña cifrada, JSON.stringify({ consultas anteriores: [], fortalezas:
    {}, debilidades: {} })), ]);

```

```

    const token = jwt.sign({ userId: email }, JWT_SECRET, { expiresIn: "7d" });
    reply.send({ token }); ❹
    catch (error) { console.error(" ❺
    Error de registro:",
    error); reply.status(500).send({ error: "Error al registrar el
    usuario" }); } });...
    ❻

```

- ❶ Define una ruta POST en /register para gestionar los registros de usuarios
- ❷ Extrae el correo electrónico y la contraseña del cuerpo de la solicitud.
- ❸ Devuelve un error 400 si falta alguno de los campos
- ❹ Abre una conexión a la base de datos SQLite
- ❺ Crea un hash de la contraseña del usuario usando bcrypt para un almacenamiento seguro
- ❻ Inserta el nuevo usuario en la tabla de usuarios con un perfil de aprendizaje vacío
- ❼ Genera un token JWT para la autenticación que caduca en siete días.
- ❽ Envía el token como respuesta para que el usuario pueda autenticar futuras solicitudes
- ❾ Maneja errores y devuelve un error 500 si falla el registro

A continuación, necesitará un punto final que admita el inicio de sesión de los usuarios. Agregue el código del **Ejemplo 13-7** a `index.js` para implementar una ruta que permita a los usuarios iniciar sesión en la aplicación con su información registrada.

Este código implementa una ruta que permite a los usuarios registrados autenticarse y recibir un token JWT para futuras solicitudes. El correo electrónico y la contraseña se extraen de la solicitud y se validan. Se consulta la base de datos para obtener el registro del usuario y la contraseña proporcionada se compara con la contraseña cifrada mediante `bcrypt`. Si la autenticación es correcta, se genera un token JWT y se devuelve al usuario, lo que permite el acceso seguro a las rutas protegidas. Si las credenciales son incorrectas, se envía una respuesta 401 "No autorizado" y cualquier error del servidor genera una respuesta 500.

Ejemplo 137. Punto final de inicio de sesión del usuario

```
...
fastify.post("/login", async (request, reply) => { const { email,      ❶
  password } = request.body; if (!email || !password) { return      ❷
    reply.status(400).send({ error: "Se
      requiere correo electrónico y contraseña" });      ❸
  }

  try
    { const db = await dbPromise; const      ❹
      usuario = await db.get("SELECCIONAR * DE usuarios DONDE userId
      = ?", [correo electrónico]);      ❺

      si (!usuario || !(await bcrypt.compare(contraseña,
      usuario.contraseña))) {      ❻
        devolver respuesta.status(401).send({ error: "Correo electrónico no válido
        o contraseña" }); }

      const token = jwt.sign({ userId: email }, JWT_SECRET, { expiresIn: "7d" });
      responder.send({ token });      ❼
      catch (error) {      ❽

        console.error(" Error de inicio de sesión:", error);
        reply.status(500).send({ error: "Error al autenticar
```

9

```
usuario" }); } });
```

...

- 1 Define una ruta POST en /login para autenticar usuarios
- 2 Extrae el correo electrónico y la contraseña del cuerpo de la solicitud.
- 3 Devuelve un error 400 si falta alguno de los campos
- 4 Se conecta a la base de datos SQLite
- 5 Consulta la base de datos para un usuario con el correo electrónico proporcionado
- 6 Verifica la contraseña usando bcrypt, devolviendo un error 401 si no es válida
- 7 Genera un token JWT para la autenticación de sesión
- 8 Envía el token para que el usuario pueda autenticar futuras solicitudes
- 9 Maneja errores y devuelve un error 500 si falla la autenticación

Con las rutas de registro e inicio de sesión configuradas, puede probar el proceso de autenticación. Ejecute su aplicación ejecutando `node index.js` y el siguiente comando `cURL` con una nueva combinación de correo electrónico y contraseña desde una nueva ventana de línea de comandos:

```
curl -X POST "http://localhost:3000/register" \
  -H "Tipo de contenido: aplicación/json" \ -d '{
    "correo electrónico": "jon@jonwexler.com",
    "contraseña": "contraseña123" }'
```

Deberías recibir un token JWT como el siguiente:


```
{"token":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VySWQ iOiJ1c2VyQG..."}
```

Puede usar este token para autenticar futuras solicitudes. Si se pierde, se elimina o caduca, puede generar un nuevo comando para los usuarios registrados mediante la ruta de inicio de sesión. Por ejemplo, puede iniciar sesión con las mismas credenciales que usó para registrarse ejecutando el siguiente comando cURL:

```
curl -X POST "http://localhost:3000/login" \
  -H "Tipo de contenido: aplicación/json" \ -d '{
    "correo electrónico": "jon@jonwexler.com",
    "contraseña": "contraseña123" }'
```

Ahora que tiene un token JWT, implemente una función de verificación de JWT como middleware para proteger sus rutas de API. Esta función comprobará la validez del token antes de permitir el acceso a ciertos endpoints.

Agregue el código del **Ejemplo 13-8** a index.js para implementar este middleware.

Este código agrega autenticación JWT para proteger las rutas API al verificar que los usuarios proporcionen un token válido antes de acceder a puntos finales seguros.

La función `verifyJWT` extrae el encabezado de autorización de las solicitudes entrantes y comprueba si se proporciona un token. Si no se proporciona, devuelve un error 401. El token se extrae del formato Bearer <token> y se valida mediante `jwt.verify()` con la clave secreta de la aplicación.

Si la operación es correcta, los datos de usuario decodificados se almacenan en `request.user`, lo que permite acceder a ellos para la gestión de solicitudes posteriores. Si el token no es válido o ha caducado, la función devuelve un error 401, lo que garantiza que solo los usuarios autenticados puedan continuar.

Ejemplo 138. middleware de autenticación JWT

```

const verifyJWT = async (solicitud, respuesta) => {
  try
    { const authHeader = request.headers.authorization; if (!authHeader)
    { return
      reply.status(401).send({ error: "Falta el token de autenticación" });
    }

    const token = authHeader.split(" ")[1]; const
    decodificado = jwt.verify(token, JWT_SECRET); solicitud.usuario
    = decodificado; } catch (error) {

    devolver respuesta.status(401).send({ error: "Token inválido o
    expirado" });

  }
};

```

- ➊ Define una función asíncronica para verificar la autenticación JWT en Rutas Fastify
- ➋ Extrae el encabezado de autorización de la solicitud entrante
- ➌ Devuelve una respuesta 401 no autorizada si no se proporciona ningún token de autenticación
- ➍ Extrae el token real del formato Bearer <token>
- ➎ Utiliza `jwt.verify()` para decodificar y validar el token usando la clave secreta `JWT_SECRET`
- ➏ Almacena los datos de usuario decodificados en `request.user`, haciéndolos accesibles en rutas protegidas
- ➐ Maneja fallas de autenticación, devolviendo 401 No autorizado si el token no es válido o ha expirado

El último paso de autenticación es incorporar el middleware `verifyJWT` en la ruta `/query`. Esto garantiza que solo

Los usuarios autenticados pueden acceder al asistente de IA. Además, se actualiza la ruta /query para encontrar el perfil de aprendizaje del usuario en la base de datos y usarlo para generar una respuesta personalizada. El perfil de aprendizaje se almacenará como texto sin formato en la base de datos. Reemplace la ruta /query en index.js con el código del **Ejemplo 13-9**.

Este código mejora el asistente de IA personalizando las respuestas según el perfil de aprendizaje de cada usuario. La ruta está protegida con autenticación JWT para garantizar que solo los usuarios conectados puedan consultar la IA. Los datos de aprendizaje previos del usuario se extraen de la base de datos y se utilizan en la consulta de IA para proporcionar respuestas personalizadas. El perfil de aprendizaje se actualiza con la información de la IA, lo que permite al asistente refinar sus recomendaciones con el tiempo. Si surge algún problema, se devuelve un error 500.

La función generateResponse se reemplaza por generateResponseWithSummary. Esta función es similar a la anterior, pero incluye lógica adicional para gestionar los perfiles de aprendizaje del usuario. El asistente de IA ahora puede proporcionar respuestas personalizadas según el perfil de aprendizaje, las fortalezas y las debilidades del usuario.

Ejemplo: Consulta al asistente de IA con perfiles de aprendizaje de usuario

```
fastify.post("/query", { preHandler: verifyJWT }, async (request, reply) => { try
{ const { prompt } = ❶

  request.body; const userId = request.user.userId; ❷
  const db = await dbPromise; ❸
  ❹

  const fila = esperar db.get(
    "SELECCIONAR perfil_de_aprendizaje DE usuarios DONDE userId = ?",
    [userId] );
    ❺

  const perfilDeAprendizaje =
    row?.learning_profile || "Este usuario aún no tiene un perfil de aprendizaje
    registrado."; ❻
```

```

    const { respuesta, updatedProfileSummary } = await
generateResponseWithSummary( prompt,

    learningProfile );
    7

    esperar db.run("ACTUALIZAR usuarios ESTABLECER perfil_de_aprendizaje = ?
¿DONDE userId = ?", [
    resumenPerfilActualizado,

    IDDeUsuario, ]);

    responder.send({ respuesta, resumenPerfilActualizado }); }
catch (error) {
    console.error(" Error de consulta:", error);
    reply.status(500).send("Error al procesar la consulta");
    9
    10
} });

```

- 1 Define una ruta POST en /query y la protege con verifyJWT para garantizar que solo los usuarios autenticados puedan acceder a ella
- 2 Extrae el mensaje del cuerpo de la solicitud.
- 3 Recupera el ID del usuario autenticado de request.user
- 4 Se conecta a la base de datos SQLite
- 5 Obtiene el perfil de aprendizaje del usuario de la base de datos.
- 6 Utiliza un mensaje de perfil predeterminado si no hay datos de aprendizaje disponibles
- 7 Llama a generateResponseWithSummary() para enviar el mensaje y el perfil de aprendizaje a la IA y obtener una respuesta actualizada
- 8 Actualiza el perfil de aprendizaje del usuario en la base de datos con el

Los nuevos conocimientos de la IA

9

Envía la respuesta generada por IA y el perfil de aprendizaje actualizado al usuario.

 Maneja errores y devuelve un error 500 si la solicitud falla

Para finalizar el desarrollo, se implementa la nueva función `generateResponseWithSummary` en `index.js` para gestionar la generación de respuestas de la IA y las actualizaciones del perfil de aprendizaje. Esta función está diseñada para enviar una solicitud estructurada a la API de Gemini, que incluye la solicitud del usuario y su perfil de aprendizaje. La IA debe devolver su respuesta en un formato JSON específico, que incluye tanto la respuesta como un resumen actualizado del perfil.

Por ejemplo, si un usuario pregunta sobre conceptos complejos de JavaScript y consultas básicas de implementación de Node, la IA analizará su perfil de aprendizaje y ofrecerá una respuesta personalizada que refleje sus fortalezas en JavaScript básico y sus debilidades en aplicaciones Node listas para producción. En ese contexto, el asistente de IA puede proporcionar respuestas más completas para ayudar al usuario a aprender y crecer, o guiarlo hacia libros prácticos sobre Node, como "Proyectos Node.js", de Jonathan Wexler.

Agrega el código del **Ejemplo 13-10** a `index.js` para implementar la función `generateResponseWithSummary`.

Esta función mejora el asistente de IA al adaptar las respuestas al contexto según el perfil de aprendizaje del usuario. Garantiza que cada respuesta tenga en cuenta las interacciones previas del usuario, lo que permite interacciones de IA personalizadas y en constante evolución. La función crea una solicitud estructurada que proporciona a la IA tanto la pregunta del usuario como su perfil de aprendizaje actual. La IA recibe instrucciones para devolver su respuesta en formato JSON válido, lo que garantiza una salida estructurada que incluye tanto la respuesta como un resumen actualizado del perfil. Se envía una solicitud POST a la API de Gemini mediante `axios`, cuyos datos contienen la entrada del usuario y las instrucciones de

La respuesta de la API se extrae, se limpia y se analiza como JSON antes de devolverse.

CONSEJO

Puede simplificar este proceso registrando la respuesta sin procesar de la IA. De esta forma, puede identificar la estructura exacta de la salida de la IA, lo que facilita el análisis y la extracción de la información relevante. Esto es especialmente útil al trabajar con modelos de IA complejos que pueden devolver datos en diversos formatos.

El objeto `requestData` está estructurado para proporcionar a la IA instrucciones contextuales, incluyendo explícitamente el perfil de aprendizaje del usuario y pidiéndole que lo actualice según la última consulta. La solicitud está estructurada para garantizar una salida JSON estricta, evitando inconsistencias en el formato de respuesta de la IA. La función realiza una solicitud asíncrona a `API_URL` con los datos estructurados de la solicitud.

Los encabezados especifican que el cuerpo de la solicitud está en formato `aplicación/json`, lo que garantiza una comunicación API adecuada.

La línea `response?.data?.candidates?.[0]?.content?.parts?.`

`[0]?.text || {}` accede de forma segura a la respuesta de la IA al gestionar los casos en los que faltan los datos esperados. El operador de encadenamiento opcional `?.` evita errores si alguna propiedad de la estructura de la respuesta no está definida. El último `|| {}` garantiza que, en ausencia de una respuesta válida, se devuelva un objeto JSON vacío en lugar de `undefined`. La función también incluye un paso para limpiar la respuesta antes del análisis, eliminando el formato innecesario. La IA suele devolver JSON dentro de bloques de código Markdown (p. ej., ````json {...}````), e incluir las instrucciones `replace()` y `trim()` garantiza que solo se conserve JSON sin procesar antes del análisis.

Una vez limpiada la respuesta, se utiliza `JSON.parse(cleanedJson)` para convertirla de una cadena a un objeto JavaScript y facilitar su acceso. Si el análisis falla, se registra un mensaje de error y se devuelve una respuesta de error estructurada en lugar de bloquear la función.

La función extrae la respuesta generada por la IA y el resumen actualizado del perfil, recurriendo al perfil de aprendizaje original si la IA no proporciona una actualización válida. En caso de error de la API, la función registra el error y devuelve un mensaje de error estructurado.

Al estructurar así las interacciones de IA, el asistente aprende de interacciones anteriores y proporciona respuestas cada vez más relevantes. El uso de la gestión de errores, el encadenamiento opcional y la validación de respuestas garantiza que, incluso si la API devuelve un resultado inesperado, el sistema se mantenga robusto y funcional.

Perfiles de 1310. Generando respuestas de IA con aprendizaje personalizado
ejemplo

```
const generateResponseWithSummary = async (prompt, learningProfile)
```

```
=> { try { const requestData ❶
```

```
=
```

```
{ contents: [ {
```

```
    rol: "usuario",
```

```
    partes: [ {
```

```
      texto: "Eres un asistente de IA que ayuda a los usuarios a  
aprender programación.
```

```
      El usuario tiene el siguiente perfil de aprendizaje:  
"${perfildeaprendizaje}".
```

```
      Con base en esto, responda su consulta y actualice su perfil  
con un resumen de una  
oración de sus fortalezas y  
debilidades.
```

```
      Responder en **formato JSON válido**: {
```

```
        "response": "Tu respuesta generada por IA",  
        "updatedProfileSummary": "Perfil actualizado  
resumen."
```

```

    }

    `, }, { text: `Consulta de usuario: ${prompt}

    `, ], 2, ], },

    constante respuesta = await axios.post(API_URL, requestData,
{
    encabezados: { "Tipo-de-contenido": "application/json" }, });

    const rawText = respuesta?.datos?.candidatos?.
[0]?.contenido?.partes?.[0]?.texto || "{}";
    const
    installedJson = rawText.replace(/``json|``/g, "").trim();

    try
    { const parsedResponse = JSON.parse(cleanedJson); return
    { respuesta:
        parsedResponse.response || "No se recibió una respuesta
válida.",
        updatedProfileSummary:
        parsedResponse.updatedProfileSummary ||
        learningProfile,

    }; } catch
    { console.error(" Formato JSON no válido de la API:",
installedJson);
    return { error: "Respuesta JSON no válida", details: installedJson }; }

    } captura (error) {
        console.error(" Error de API:", error.response?.data || error.message);
    return { error: "No
        se pudo generar la respuesta", detalles: error.response?.data ||
        error.message };

    }
};

```


- ❶ Define una función asincrónica para consultar la IA y actualizar el perfil de aprendizaje del usuario.
- ❷ Construye la estructura del mensaje, incluido el perfil de aprendizaje del usuario y las instrucciones para que la IA devuelva una respuesta JSON.
- ❸ Envía la solicitud a la API de Gemini con el mensaje y los encabezados.
- ❹ Extrae la respuesta generada por IA del objeto de respuesta de API
- ❺ Limpia la respuesta eliminando el formato innecesario (por ejemplo, bloques de código Markdown)
- ❻ Analiza la respuesta JSON de la IA y devuelve la respuesta generada junto con el perfil de aprendizaje actualizado.

Ahora puede crear un nuevo usuario con la ruta de registro e iniciar sesión con la ruta de inicio de sesión, o usar el token JWT y la cuenta de usuario del paso anterior. Después, puede usar el token JWT para autenticar las solicitudes al punto final /query . Por ejemplo, ejecute el siguiente comando cURL para consultar al asistente de IA sobre la recursión con un token JWT:

```
curl -X POST "http://localhost:3000/query" \-H "Autorización: Portador  
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1Ij0i..."  
\-H "Content-Type: application/json" \-d '{ "prompt": "¿Cuál es la mejor  
manera de aprender  
  
¿recursión?" }'
```

Cuando envía la solicitud, el asistente de IA responde con un objeto JSON estructurado que incluye tanto la respuesta generada como un resumen actualizado del perfil de aprendizaje del usuario:

```
{ "answer": "La recursión puede ser un concepto complicado...",  
  "updatedProfileSummary": "El usuario es nuevo en programación"
```

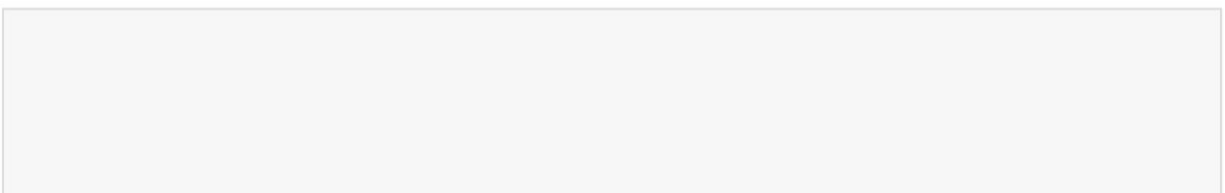
conceptos y requiere una comprensión fundamental de cómo aprender nuevos temas, específicamente la recursión; por lo tanto, son fuertes en hacer la pregunta correcta, pero débiles en el conocimiento fundamental sobre la recursión." }

Intentar consultar la IA sin un token JWT válido devolverá un error con el mensaje "Token de autenticación faltante", lo que garantiza que solo los usuarios autenticados puedan acceder al asistente de IA.

Ahora que cuenta con un asistente de aprendizaje con IA en funcionamiento, autenticación y respuestas personalizadas, el siguiente paso es crear una interfaz del lado del cliente para las interacciones del usuario. Una interfaz web o móvil permitirá a los usuarios interactuar con el asistente de IA de forma más intuitiva, facilitando el envío de consultas, la revisión de respuestas anteriores y la visualización de su progreso de aprendizaje. Puede lograrlo desarrollando un frontend con React, Vue o Svelte e integrándolo con su backend de Fastify mediante solicitudes API.

Además, considere mejorar la experiencia del usuario implementando conversaciones en tiempo real con WebSockets, almacenando análisis de aprendizaje más detallados y refinando las explicaciones generadas por IA con técnicas de ingeniería de indicaciones más avanzadas. También puede considerar las siguientes mejoras:

- Agregar paginación o filtrado para respuestas de IA anteriores
- Permitir a los usuarios editar su perfil de aprendizaje manualmente en lugar de actualizaciones solo con IA
- Integrar un mecanismo de retroalimentación donde los usuarios puedan calificar las respuestas de la IA para refinar resultados futuros



EJERCICIOS DEL CAPÍTULO

1. Realice un seguimiento del progreso del usuario con análisis de aprendizaje.

En este ejercicio, implementará un sistema para monitorizar el progreso del usuario según sus interacciones con la IA. Cada vez que un usuario interactúa con el asistente, la IA debe registrar su consulta y categorizarla según sus fortalezas o debilidades, según su perfil de aprendizaje.

- a. Actualice la función `generateResponseWithSummary` para incluir un componente de análisis de aprendizaje.
- b. Agregue un mecanismo para categorizar las respuestas (por ejemplo, “Fuerte en algoritmos”, “Débil en recursión”). Almacene estas categorías en el perfil de aprendizaje del usuario en la base de datos.
- c. Cuando el usuario envía una consulta, analiza su perfil para sugerir áreas de mejora y mostrar comentarios basados en las respuestas anteriores de la IA.
- d. Después de agregar esta funcionalidad, cree múltiples Interactúe con el asistente y compruebe que el perfil de aprendizaje se actualiza correctamente. El perfil del usuario debe reflejar su progreso, destacando las áreas en las que destaca o necesita profundizar.

2. Adapte las respuestas de IA según el tipo de consulta:

En este ejercicio más simple, modificará la respuesta del asistente según el tipo de consulta: si el usuario solicita una explicación, ayuda con la codificación o una pregunta de práctica:

- a. Actualice la función `generateResponseWithSummary` para verificar si hay palabras clave simples en el mensaje del usuario (por ejemplo, “explicar”, “código”, “practicar”).
- b. Modifique ligeramente la respuesta en función de la Palabra clave. Por ejemplo:
 - Si la consulta contiene la palabra “explicar”, la IA debería proporcionar una explicación detallada.
 - Si contiene la palabra “código”, la IA debería devolver un ejemplo de código.
- c. Mantenga los cambios mínimos para garantizar que la IA aún pueda responder cualquier pregunta adecuadamente.
- d. Intente enviar consultas con las palabras «explicar», «código» y «práctica», y compruebe que las respuestas cambian según el tipo de consulta.

Resumen En este

capítulo, creaste un asistente de aprendizaje basado en IA que se integra con la API Gemini de Google. El asistente está diseñado para ayudar a los usuarios a aprender conceptos de programación y prepararse para entrevistas técnicas mediante respuestas adaptativas. En el proceso, tú:

- Conectó una aplicación Node a la API de Gemini usando Fastify, lo que permitió respuestas impulsadas por IA.
- Diseñó respuestas de IA que sean conscientes del contexto, aprovechando los perfiles de aprendizaje del usuario para brindar asistencia personalizada.
- Se implementó una base de datos de perfiles de usuario en SQLite3, lo que permite al asistente realizar un seguimiento del progreso del usuario y adaptar las respuestas a lo largo del tiempo.

tiempo

- Se aseguró el sistema con autenticación JWT, garantizando que solo los usuarios registrados puedan acceder al asistente de IA.
- Manejo optimizado de solicitudes de IA mediante la estructuración de indicaciones, la limpieza de la salida de IA y la gestión de respuestas JSON

Al integrar estas funciones, Interview Atlas ofrece una sólida experiencia de aprendizaje basada en IA, guiando dinámicamente a los usuarios según sus fortalezas y debilidades. Esta base puede ampliarse aún más con ingeniería avanzada de indicaciones, ciclos de retroalimentación mejorados y una mayor personalización de la IA.

Apéndice A. Hoja de referencia de nodos e inicialización del proyecto

El desarrollo de aplicaciones en Node puede ser tan simple como una sola línea de JavaScript o tan complejo como un sistema distribuido alojado globalmente. Afortunadamente, Node es una plataforma versátil que puede gestionar ambos extremos. Esta guía rápida proporciona una referencia rápida para configurar un proyecto en Node, comprender el archivo package.json y escribir código JavaScript moderno. También destaca áreas clave para el desarrollo profesional como ingeniero en Node.

En este apéndice cubriremos los siguientes temas:

- Inicialización de un proyecto Node usando npm
- Estructura de directorio recomendada para proyectos de Node
- Comprender el archivo package.json y escribir scripts significativos
- Patrones de sintaxis de nodo modernos
- Áreas de crecimiento para los ingenieros de Node

Puede usar este apéndice como referencia rápida o como punto de partida para sus proyectos de Node. Está diseñado para ser conciso y fácil de seguir, brindándole la información esencial que necesita para comenzar a desarrollar en Node.

NOTA

Para conocer los pasos de instalación del nodo, consulte [el Capítulo 1](#).

Inicialización de un proyecto de nodo (usando npm)

Gran parte del desarrollo de Node implica el uso de la CLI para instalar paquetes, ejecutar scripts y administrar el proyecto. El primer paso en cualquier proyecto de Node es inicializarlo con npm, el gestor de paquetes de Node. Esto se hace ejecutando el siguiente comando en su terminal:

```
mkdir node-app && cd node-app npm init -y
```

NOTA

"node-app" es solo un nombre de ejemplo. Puedes reemplazarlo por el nombre de tu proyecto. Usa nombres en minúsculas separados por guiones (p. ej., "node-app-server") que sean cortos, descriptivos y eviten caracteres especiales para garantizar la compatibilidad entre npm, URL y sistemas de archivos.

Este comando configura un archivo package.json básico, que actúa como manifiesto de tu proyecto, definiendo su nombre, versión, dependencias y scripts. El indicador -y completa automáticamente los valores predeterminados, lo que te permite empezar rápidamente sin tener que responder a las solicitudes de forma para ajustarlo. Siempre puedes editar la configuración interactiva. json más tarde del paquete generado o añadir metadatos.

NOTA

Un manifiesto de proyecto es un archivo que define metadatos esenciales sobre su proyecto (como su nombre, versión, dependencias, scripts y configuración) para que las herramientas (como npm, sistemas de compilación o canalizaciones de CI) puedan entender cómo compilarlo, ejecutarlo o administrarlo.

Este archivo fundamental es crucial para que otros desarrolladores y herramientas puedan comprender y trabajar con tu proyecto. Incluso si empiezas con...

los valores predeterminados, refinar su manifiesto de manera anticipada puede ayudar a evitar futuras confusiones o configuraciones incorrectas.

CONSEJO

Si bien npm es el administrador de paquetes predeterminado incluido con Node y funciona muy bien para la mayoría de los proyectos, algunos desarrolladores usan herramientas alternativas como Yarn para funciones más avanzadas.

npm es el gestor de paquetes predeterminado de Node, diseñado para gestionar dependencias y módulos de código JavaScript reutilizables. Cuando un desarrollador ejecuta `npm install`, la herramienta lee el archivo `package.json` en el directorio del proyecto, que lista todas las dependencias necesarias y sus versiones. A continuación, descarga estos paquetes del registro de npm (una extensa base de datos pública de bibliotecas JavaScript de código abierto) al directorio local `node_modules/`.

Para mejorar el rendimiento y reducir las descargas redundantes, npm también mantiene una caché global en el equipo local. La resolución de dependencias se realiza mediante un grafo dirigido para garantizar que todos los paquetes necesarios (y sus dependencias) se instalen en la versión correcta, evitando así conflictos. Se utilizan archivos de bloqueo como `package-lock.json` para registrar las versiones exactas instaladas, lo que garantiza entornos consistentes en todos los equipos e implementaciones.

HILOS VERSUS NPM

Yarn es un gestor de paquetes JavaScript moderno, creado por Facebook (Meta), que ayuda a los desarrolladores a instalar, actualizar y gestionar las dependencias de sus proyectos, al igual que npm. Originalmente, se diseñó para solucionar algunas de las deficiencias de npm en aquel momento, ofreciendo instalaciones más rápidas, mejor almacenamiento en caché y una consistencia más fiable de los archivos de bloqueo entre equipos.

Hoy, Yarn v4+ presenta características avanzadas como:

Conectar y usar (PnP)

Una forma de acelerar el inicio del proyecto evitando la carpeta tradicional `node_modules`.

Instalación cero

Le permite confirmar dependencias en su repositorio para que no necesite ejecutar la instalación por separado en cada máquina.

Para usar Yarn sin instalarlo manualmente, Node incluye una herramienta auxiliar integrada llamada Corepack (desde Node v16.10 y habilitada por defecto en v20+). Esta herramienta permite habilitar rápidamente Yarn y otros gestores de paquetes. Simplemente ejecute:

```
corepack habilitar  
yarn init -2
```

Esto inicializa un nuevo proyecto de Yarn usando la última versión y lo prepara con valores predeterminados modernos.

Aunque Yarn no está tan extendido como npm, puede ser una excelente opción para equipos que valoran la velocidad y la fiabilidad. Además, cuenta con una comunidad y un ecosistema sólidos, con numerosas bibliotecas y herramientas populares que lo respaldan. Para más información, consulta la documentación [de Yarn](#).

Estructura de directorio recomendada: Una estructura

de proyecto clara y escalable facilita la claridad, la mantenibilidad y la colaboración en equipo, especialmente a medida que aumenta la complejidad. A continuación, se presentan tres diseños recomendados según el tamaño y los objetivos de su proyecto Node.

Suponiendo un proyecto llamado node-app, una estructura básica podría verse así:

```
node-app/ |—
index.js  |— ❶
package.json
```

❶ index.js representa su archivo fuente.

Esta configuración es ideal para proyectos pequeños o individuales que aún no requieren pruebas ni configuraciones. Minimiza la sobrecarga y te permite concentrarte en escribir código inmediatamente. Es ideal para empezar rápidamente con un script o una utilidad sencilla. La mayoría de tus proyectos a lo largo del libro seguirán esta estructura.

Para aplicaciones pequeñas y medianas con necesidades básicas de prueba y configuración, puede compartimentar su código y agregar algunos directorios más:

```
node-app/ |—
src/      |— ❶
index.js  |— test/
|         |— .env ❷
|         |— .gitignore ❸
|         |— package.json
|         |— README.md
```

- ❶ src contiene la lógica de su aplicación principal.
- ❷ prueba contiene sus pruebas unitarias o de integración.
- ❸ .env contiene sus variables de entorno, que deben mantenerse fuera del control de versiones.

Esto añade separación para las pruebas y la configuración del entorno, lo cual resulta útil al trabajar con API o servicios destinados a producción. También facilita la integración de herramientas de CI, registro e implementación de forma más eficiente. Esta es una estructura común para proyectos serios, pero no empresariales.

Por último, para aplicaciones de pila completa o de escala empresarial con interfaces, entornos múltiples y automatización de implementación, puede utilizar un diseño más complejo como el siguiente:

```

node-app/ |—
src/ | |—
servidor/ | |— cliente/ ❶
| |— componentes/ | ❷
| |— index.html |— público/ |—
config/ |— scripts/ |— prueba/
|— .env ❸
|— ."ignorar ❹
|— paquete.json ❺
|—
README.md

```

- ❶ El servidor es donde se encuentra la lógica del backend de Node.
- ❷ El cliente es donde se encuentran los componentes de UI (React, Vue, etc.).
- ❸ público es donde se encuentran sus activos estáticos (íconos, fuentes, etc.).

- ④ config es donde se encuentran los archivos de configuración específicos de su entorno.
- ⑤ Los scripts son donde se encuentran sus scripts de automatización y DevOps.

Esta estructura admite aplicaciones con código backend y frontend, y separa claramente las preocupaciones de los equipos más grandes. Admite scripts de automatización, múltiples entornos de implementación y estrategias de prueba avanzadas. Es ideal para aplicaciones escalables y lo suficientemente flexible como para crecer con nuevos microservicios o equipos.

A medida que tu proyecto evoluciona, puedes adaptar cualquiera de estas estructuras a tus necesidades. La clave es mantener todo organizado y modular para que puedas encontrar y gestionar fácilmente tu código a medida que crece.

Comprensión de package.json y scripts. El archivo package.json es el manifiesto de tu proyecto de Node. Define metadatos, scripts y reglas de dependencia que otras herramientas (como npm o Yarn) utilizan para comprender el funcionamiento de tu aplicación.

En esencia, este archivo debería verse así:

```
{
  "nombre": "node-app",
  "versión": "1.0.0", "tipo":
  "módulo", "scripts":
  { "inicio": "node
    index.js", "dev": "node --watch src/
    index.js" }, "dependencias": {},

  "devDependencies": {}
}
```

Este archivo le da a su proyecto un nombre y una versión, y le dice a Node que trate su código como módulos ES modernos usando "type": "module".

La sección de scripts le permite ejecutar comandos útiles como npm

Inicia o ejecuta `npm run dev` sin escribir el comando completo. Las secciones "dependencies" y "devDependencies" vacías significan que aún no has instalado ninguna biblioteca externa, pero se completarán automáticamente al agregar paquetes.

La configuración "type": "module" habilita la compatibilidad con la sintaxis nativa de importación/exportación, reemplazando el antiguo patrón "require" de CommonJS. El indicador `--watch` para Node (versión 20+) recarga la aplicación automáticamente al cambiar los archivos, eliminando la necesidad de herramientas como nodemon. La sección de scripts funciona como una CLI unificada para las tareas comunes de desarrollo: automatiza el análisis de errores, las pruebas y el formateo, utilizando las herramientas instaladas como dependencias de desarrollo.

CONSEJO

Esto es todo lo que necesitas para empezar con un proyecto pequeño o un servidor API sencillo. Es ligero, legible y sirve como la única fuente de información para la configuración de tu proyecto. Considéralo el manual de instrucciones de tu proyecto para herramientas y otros desarrolladores.

El centro de configuración de paquetes para JSON se convierte en un potente conjunto de herramientas de compilación, pruebas, formato y flujos de trabajo en equipo. A medida que tu proyecto crece, puedes agregar campos como motores para especificar versiones de Node, archivos para controlar lo que se publica o espacios de trabajo para repositorios mono. La sección de scripts también puede incluir comandos para el análisis de código, las pruebas y el formato.

```
{
  "nombre": "node-app",
  "versión": "1.0.0", "tipo":
  "módulo", "scripts":
  { "inicio": "node
    src/index.js", "dev": "node --watch src/
    index.js", "lint": "eslint .",
```

```
"prueba": "ejecutar vitest",  
"formato": "más bonito --write ." },  
  
"dependencias": {},  
"devDependencies": {}  
}
```

A medida que se añaden dependencias, el archivo crece para registrar las versiones y los alcances exactos (producción versus desarrollo). Los proyectos complejos también pueden usar campos como motores, exportaciones, archivos y espacios de trabajo para controlar el comportamiento de la implementación, la compatibilidad y la organización de monorepositorio. Comprender cómo estructurar y automatizar con `package.json` es clave para gestionar aplicaciones Node escalables y fáciles de mantener.

CONSEJO

Documente sus scripts en README para que los colaboradores sepan cómo ejecutar y mantener el proyecto. Esto ayuda a evitar confusiones y garantiza que el equipo siga un flujo de trabajo consistente. El paquete JSON es un motor de productividad cuando se usa eficazmente.

Patrones de sintaxis de nodo modernos A partir de

Node 20+, muchas características de JavaScript que alguna vez fueron opcionales o experimentales ahora son totalmente compatibles y se consideran la mejor práctica. Comprender estos patrones hará que su código sea más limpio, más fácil de mantener y más alineado con los estándares modernos utilizados en el desarrollo tanto de backend como de frontend.

En concreto, esta sección cubre:

- Compatibilidad nativa con ESM
- Espera de alto nivel
- Desestructuración de valores predeterminados

- Encadenamiento opcional
- Coalición nula
- Búsqueda incorporada

Como se mencionó anteriormente en este apéndice, Node moderno usa módulos ES (ESM) por defecto cuando "type": "module" se configura en se utiliza JSON. Este estilo de importación/exportación es el mismo que en navegadores y herramientas de paquetes modernos como Deno y Vite, lo que ayuda a unificar el desarrollo frontend y backend. Reemplaza la antigua sintaxis require() utilizada en CommonJS, lo que mejora la interoperabilidad y la viabilidad futura de su código.

A continuación se muestra cómo importar módulos con ESM:

```
importar fs desde 'node:fs/promises';  
importar express desde 'express';
```

Estas líneas de código se encuentran naturalmente al principio de los archivos, al igual que en el JavaScript del navegador. Esto facilita la lectura y comprensión de las dependencias a simple vista.

Con ESM, Node ahora permite usar await en el nivel superior, sin necesidad de encapsular todo en una función asíncrona . Esto simplifica la carga de scripts puntuales y la configuración, especialmente en archivos de entrada como index.js. Hace que el código asíncrono sea más limpio y más fácil de seguir, especialmente para principiantes.

El siguiente ejemplo demuestra el uso de await de nivel superior :

```
constante data = await fs.readFile('./config.json', 'utf-8');
```

Esta línea muestra cómo leer un archivo asíncronicamente usando await con el módulo fs/promises integrado de Node . Lee el contenido de configjson (como texto, usando codificación UTF-8) y almacena el resultado en la variable data . Dado que await se usa en el nivel superior,

Este patrón solo es posible en un módulo ES (tipo: "módulo" en json) y simplifica lo asincrónico que solía requerir envolver todo el paquete en una función

Otra función útil de JavaScript moderno es la desestructuración de objetos con valores predeterminados. En el siguiente ejemplo, la variable de entorno PORT se extrae de process.env y, si no está definida, se utiliza un valor predeterminado de 3000.

Este patrón es especialmente útil en los archivos de configuración, ya que garantiza el correcto funcionamiento de la aplicación incluso cuando faltan ciertas variables de entorno. La desestructuración también reduce el código repetitivo y hace que la lógica de configuración sea más clara y fácil de entender:

```
const { PUERTO = 3000 } = proceso.env;
```

El encadenamiento opcional (?.) y la coalescencia nula (??) son funciones modernas de JavaScript introducidas en ECMAScript 2020. Permiten acceder de forma segura a propiedades de objetos profundamente anidados y asignar valores de reserva sin escribir largas cadenas de comprobaciones condicionales.

Estas funciones son compatibles con Node desde la versión 14 (publicada en abril de 2020) y ahora se adoptan ampliamente en bases de código JavaScript tanto frontend como backend.

El siguiente ejemplo demuestra cómo estos operadores permiten acceder de forma segura a las propiedades de objetos profundamente anidados sin bloquear la aplicación si algún valor es indefinido o nulo. El operador ?? garantiza el uso de un valor de reserva cuando el lado izquierdo es nulo o indefinido (pero no valores falsos como 0 o ""). Esto resulta en un código más limpio y tolerante a fallos, con menos comprobaciones manuales.

```
const nombre_usuario = usuario?.perfil?.nombre ?? 'NOMBRE_INVITADO';
```

Por último, a partir de Node 18+, fetch() está disponible globalmente, al igual que en el navegador, sin necesidad de instalar bibliotecas externas como node-fetch. Esto

Permite realizar solicitudes HTTP de forma sencilla y concisa mediante API nativas. Es especialmente útil para desarrolladores full-stack que vienen del frontend, ya que reduce la curva de aprendizaje y reduce las dependencias.

```
const res = await fetch('https://api.example.com/data'); const data =  
await res.json(); console.log(data);
```

Modern Node integra plenamente funciones como módulos ES, espera de nivel superior y búsqueda nativa, lo que permite un código más limpio y legible, en estrecha sintonía con el JavaScript moderno basado en navegador. Estas funciones simplifican las operaciones asíncronas, mejoran la seguridad de la configuración y reducen la sobrecarga de dependencias. Al dominar estos patrones, los desarrolladores pueden escribir aplicaciones más robustas, fáciles de mantener y con garantía de futuro.

Áreas de crecimiento para los ingenieros de Node: A

medida que te familiarices con la configuración y la sintaxis de los proyectos, es importante ampliar tus conocimientos más allá de lo básico. Node se utiliza en una amplia variedad de entornos, desde pequeños servicios de backend hasta API distribuidas globalmente y sistemas en tiempo real. Cuanto más familiarices con las herramientas y patrones, más eficaz y versátil serás como ingeniero de Node.

Aquí hay varias áreas que vale la pena explorar a medida que avanza:

Arquitectura de backend

Aprenda a estructurar API y servicios usando REST, GraphQL o modelos basados en eventos. Comprender la arquitectura en capas (p. ej., controladores, servicios, acceso a datos) le ayuda a crear aplicaciones escalables y modulares. Frameworks como Express y Fastify pueden ser buenos puntos de partida, pero saber por qué se usan ciertos patrones es aún más importante que cuáles.

Bases de datos y persistencia

Aprenda a interactuar con bases de datos SQL y NoSQL. Familiarícese con herramientas como PostgreSQL, MongoDB, Prisma y Sequelize. Comprenda las transacciones de bases de datos, la indexación y la optimización de consultas: estas habilidades son esenciales para crear aplicaciones de alto rendimiento.

Mejores prácticas de seguridad

Aprenda a gestionar la autenticación y la autorización utilizando técnicas como JWT, OAuth2 y sesiones basadas en cookies.

Aprenda a proteger sus API contra amenazas como XSS, CSRF, ataques de inyección y limitación de velocidad. La seguridad cobra mayor importancia a medida que su código pasa del desarrollo local a entornos de producción.

Pruebas y automatización

Escribir pruebas automatizadas mejora la confianza en tu código. Aprende a escribir pruebas unitarias, de integración y de extremo a extremo con herramientas como Vitest, Jest o Supertest. Combínalo con flujos de trabajo automatizados con GitHub Actions u otras herramientas de CI/CD para optimizar la implementación y reducir errores.

DevOps y despliegue

Explora cómo contenerizar tu aplicación con Docker y gestionarla con herramientas como PM2, systemd o Kubernetes. Aprende a implementar en plataformas como Vercel, Heroku o AWS. Comprender cómo se ejecuta tu aplicación en producción te convertirá en un desarrollador y miembro del equipo más eficaz.

Monitoreo y observabilidad

Agregue seguimiento de errores y monitoreo de rendimiento con herramientas como Sentry, Prometheus u OpenTelemetry. El registro y las métricas ayudan.

Comprendes el comportamiento real y solucionas los problemas con mayor rapidez. Esto se vuelve crucial a medida que tu aplicación gana usuarios y complejidad.

Seguridad de tipos y herramientas

Adopte TypeScript o JSDoc para mejorar la seguridad de tipos y la experiencia del desarrollador. Los editores y herramientas con reconocimiento de tipos ayudan a prevenir errores y a mejorar la legibilidad. Incluso los proyectos pequeños se benefician de la escritura una vez que escalan.

Sistemas de transmisión y en tiempo real

Aprenda a trabajar con WebSockets, eventos enviados por el servidor (SSE) y API de streaming. Esto es esencial para aplicaciones con paneles en vivo, funciones multijugador o edición colaborativa.

Bibliotecas como Socket.IO o EventSource nativa admiten este patrón.

Al centrarte en estas áreas a lo largo del tiempo, adquirirás las habilidades necesarias para construir y mantener sistemas complejos con confianza. A medida que el ecosistema crece y evoluciona, el aprendizaje continuo garantizará que tus conocimientos de Node sigan siendo relevantes y valiosos.

Resumen

En este apéndice usted:

- Inicialicé un nuevo proyecto de Node usando npm y exploré cómo configurar el archivo package.json
- Aprendió a estructurar el directorio de su proyecto en función de la escala y la complejidad.
- Comparó npm con herramientas alternativas como Yarn y exploró cómo usar Corepack para habilitarlas

- Se exploraron patrones de sintaxis de Node modernos, como módulos ES, espera de nivel superior y encadenamiento opcional .
- Identificó áreas clave para el crecimiento profesional como ingeniero de Node, incluidas las pruebas, DevOps y las mejores prácticas de seguridad.

Apéndice B. Configuración de sus herramientas de desarrollo

Para escribir, depurar y mantener aplicaciones Node de calidad profesional, necesitas un entorno de desarrollo sólido. Este apéndice te guía en la configuración de VS Code (el IDE más popular para JavaScript y Node), junto con extensiones esenciales que admiten formato, linting, depuración y control de versiones. Aprenderás a configurar Git para el seguimiento de cambios, administrar la configuración a nivel de proyecto y probar APIs con herramientas como Postman para simular solicitudes y verificar el comportamiento de los endpoints. También verás cuándo usar puntos de interrupción en lugar de console.log y cómo distinguir entre reglas de estilo (gestionadas por Prettier) y reglas lógicas (gestionadas por ESLint).

Si aún no has instalado VS Code, puedes obtenerlo del [sitio oficial](#) o consultar la guía de instalación en [el Capítulo 1](#).

Usar Git desde la línea de comandos. Aunque VS Code

incluye una interfaz Git integrada, muchos desarrolladores profesionales confían en la CLI de Git para mayor velocidad, scripting y control. Usar Git desde la terminal te ayuda a comprender lo que ocurre en segundo plano y te permite trabajar en cualquier entorno, incluso sin una interfaz gráfica de usuario.

Para empezar, instala Git desde <https://git-scm.com>. Una vez instalado, verifícalo ejecutando: Para verificar que Git esté instalado en tu sistema, ejecuta el siguiente comando:

git --versión

❶

❶

Muestra la versión actualmente instalada de Git (por ejemplo, la versión 2.42.0 de Git)

Si el número de versión se muestra correctamente, está listo para usar Git desde la línea de comandos.

Para comenzar a realizar el seguimiento de su proyecto, inicialice un repositorio, prepare sus archivos y realice su primera confirmación:

```
git init git ❶  
add .git ❷  
commit -m "Confirmación inicial" ❸
```

- ❶ Crea un nuevo repositorio Git en la carpeta actual
- ❷ Prepara todos los archivos en el directorio actual para su confirmación
- ❸ Confirma los cambios programados con un mensaje que describe la instantánea.

Una vez configurado tu repositorio, estos son algunos comandos Git esenciales que usarás regularmente:

```
estado de git, ❶  
diferencia de ❷  
git, registro ❸  
de git, restauración ❹  
de git, reinicio de git ❺
```

- ❶ Muestra el estado actual de su directorio de trabajo y los cambios programados
- ❷ Muestra las diferencias entre los archivos modificados y la última confirmación
- ❸

Muestra una lista de confirmaciones anteriores, incluidos identificadores, fechas y mensajes.

- ④ Revierte los cambios en un archivo al último estado confirmado
- ⑤ Desactiva un archivo o restablece su historial de confirmaciones a un estado anterior

CONSEJO

Utilice un archivo `.gitignore` para evitar que archivos confidenciales o innecesarios (como `.env`, `node_modules` o registros del sistema) se envíen al control de versiones.

Con tu flujo de trabajo de Git implementado, el siguiente paso es mejorar tu entorno de desarrollo. VS Code admite una amplia gama de extensiones que te ayudan a automatizar tareas, optimizar la calidad del código y acelerar tu productividad.

Extensiones de VS Code recomendadas

Las extensiones de VS Code son complementos livianos que mejoran la funcionalidad, la productividad y la personalización del editor de VS Code.

Permiten a los desarrolladores adaptar su entorno con herramientas de depuración, revisión, formateo, creación de temas, compatibilidad de idiomas y más.

Las extensiones se gestionan a través del Marketplace de Extensiones integrado, lo que facilita la búsqueda, instalación y actualización de herramientas directamente desde el editor. Al integrar potentes funciones como la integración con Git, sugerencias de código y utilidades específicas del entorno, las extensiones ayudan a optimizar los flujos de trabajo de desarrollo, reducir los cambios de contexto y mejorar la calidad del código.

Se recomienda lo siguiente para todos los desarrolladores de Node:

ESLint

Aplica un estilo de codificación consistente y detecta errores de sintaxis. Piensa en el linting como análisis de código, mientras que el formateo se centra en la apariencia del código, como el espaciado y la sangría. Usar extensiones de linting ayuda a revisar tu código para detectar posibles errores, infracciones de estilo o problemas con las mejores prácticas.

Más bonita

Formatea automáticamente su código según reglas definidas.

Puedes configurar VS Code para ajustar automáticamente el espaciado, la sangría y los saltos de línea. Herramientas como ESLint permiten ambas funciones, pero es recomendable usar ESLint para las reglas lógicas y Prettier para las de estilo.

Puede configurar VS Code para formatear automáticamente al guardar cambiando la configuración del editor para incluir "editor.formatOnSave": true.

Paquete de extensión de nodo

Agrega fragmentos, compatibilidad con REPL y más para el desarrollo de Node. Herramientas como esta mejoran tu capacidad de analizar y depurar tu código.

Si bien `console.log()` es rápido y familiar, el depurador integrado de VS Code es mucho más poderoso para inspeccionar variables, recorrer el código y evaluar expresiones en tiempo real.

Para utilizar el depurador:

1. Abra cualquier archivo JavaScript.
2. Haga clic en el margen izquierdo junto al número de línea para establecer un punto de interrupción.
3. Presione F5 o abra el panel de depuración y ejecute "Iniciar depuración".

Puede inspeccionar variables, utilizar una lista de vigilancia y recorrer el código línea por línea.

GitLens

Mejora la integración de Git con el historial, las anotaciones de errores y la lente de código. Esto será especialmente útil al modificar el código en cada capítulo. A medida que avanzas, podrás visualizar mejor tus cambios.

Para instalarlos, abra la barra lateral de Extensiones (Cmd+Shift+X o Ctrl+Shift+X) y busque cada nombre arriba.

Una vez instaladas, la mayoría de las extensiones se activarán automáticamente en función de los archivos del proyecto (como prettier, eslint o

A medida que sus aplicaciones se vuelven más complejas, a menudo necesitará interactuar directamente con los puntos finales del backend para verificar que las API respondan correctamente. Si bien escribir interfaces frontend o scripts de prueba es una forma de hacerlo, un enfoque más rápido y flexible es utilizar un cliente API dedicado.

Prueba de tu API con Postman.

Las aplicaciones

modernas suelen depender de las API para gestionar solicitudes, servir datos y conectarse con clientes frontend o servicios de terceros. Probar estas API manualmente con herramientas como curl o escribir scripts de prueba personalizados puede ser una tarea tediosa y propensa a errores. Postman es una aplicación gratuita basada en interfaz gráfica de usuario que simplifica este proceso, permitiéndote enviar solicitudes HTTP, inspeccionar respuestas, simular casos extremos y depurar más rápido sin escribir código adicional. Es especialmente útil para probar endpoints REST, verificar la autenticación y depurar cargas útiles durante el desarrollo.

Para empezar:

1. **Descargue Postman.**

2. Abra la aplicación y cree una nueva solicitud.
3. Elija un método (GET, POST, etc.) e ingrese la URL de su API (**localhostusers**).
(por ejemplo, **http://localhost:3000/**)
4. Agregue encabezados o parámetros de cuerpo según sea necesario.
5. Haga clic en Enviar para realizar la solicitud.

Postman optimiza el proceso de desarrollo de API al permitirte probar endpoints de forma interactiva mientras los construyes. En lugar de depender de solicitudes basadas en navegador o comandos curl específicos, Postman proporciona una interfaz visual donde puedes crear solicitudes, inspeccionar respuestas y depurar errores en tiempo real. Este bucle de retroalimentación inmediata te ayuda a confirmar que las rutas se comportan como se espera, que los formatos de respuesta son correctos y que la gestión de errores funciona correctamente.

Además de las solicitudes básicas, Postman te permite organizar tus endpoints en colecciones, que funcionan como documentación dinámica y conjuntos de pruebas para tu backend. Puedes simular flujos de autenticación reales adjuntando encabezados, tokens o credenciales, e incluso encadenar solicitudes para modelar recorridos de usuario completos. Esto facilita compartir APIs con compañeros de equipo o clientes, automatizar pruebas de regresión y mantener un comportamiento consistente en todos los entornos.

CONSEJO

Utilice Postman cuando necesite más control del que ofrece el navegador, como enviar solicitudes POST con cargas JSON o probar rutas protegidas con JWT.

Para complementar su flujo de trabajo de desarrollo de API, también necesitará herramientas que agilicen la forma en que su servidor responde a los cambios durante el desarrollo activo.

Monitoreo de cambios de archivos con nodemon. En un flujo de trabajo de desarrollo típico, a menudo es necesario reiniciar el servidor Node manualmente después de realizar cambios en el código. Esto ralentiza la iteración y genera fricción innecesaria. nodemon ayuda monitoreando automáticamente los archivos y reiniciando la aplicación al detectar un cambio, lo que agiliza y optimiza el desarrollo, especialmente para servicios backend o API.

Para instalar nodemon globalmente y que esté disponible en cualquier proyecto, ejecute `npm install -g nodemon`. Una vez instalado, puede ejecutar su aplicación de Node con reinicios automáticos dentro de la carpeta de su proyecto ejecutando `nodemon index.js`. Esto inicia `index.js` y lo reinicia al detectar cambios en los archivos.

Si su aplicación usa módulos ECMAScript (es decir, tipo: "módulo" en json), paquete es posible que deba incluir la extensión del archivo explícitamente:

NOTA

Si su aplicación utiliza entradas interactivas (p. ej., `readline`, `inquirer`, etc.), nodemon podría reiniciarse antes de que el usuario termine de interactuar. En esos casos, ejecute la aplicación con `node index.js`.

A medida que sus aplicaciones se vuelven más complejas, a menudo necesitará interactuar directamente con los puntos finales del backend para verificar que las API respondan correctamente. Si bien escribir interfaces frontend o scripts de prueba es una forma de hacerlo, un enfoque más rápido y flexible es utilizar un cliente API dedicado.

Resumen Un

entorno de desarrollo bien configurado le ayuda a escribir código más limpio y fiable, y a depurar problemas con mayor rapidez. En este apéndice, aprendió a:

- Configure su entorno de desarrollo con Node.js, Git y Visual Studio Code, y aprenda comandos esenciales para el control de versiones y la gestión de proyectos.
- Se instalaron extensiones clave de VS Code y se configuraron herramientas como Prettier, ESLint y el depurador integrado para mejorar la calidad del código y la eficiencia del desarrollador
- Probé las API de backend con Postman, exploré cómo simular flujos de autenticación y vi cómo depurar el comportamiento del servidor durante el desarrollo activo.

Con sus herramientas listas, estará preparado para un desarrollo más rápido y seguro en el resto del libro.

Apéndice C. Trabajar con bases de datos en proyectos de Node

La mayoría de las aplicaciones del mundo real dependen de bases de datos para almacenar y recuperar datos. Este apéndice presenta las herramientas de datos esenciales que se utilizan en los proyectos de Node, desde bases de datos completas como MongoDB, PostgreSQL y SQLite, hasta servicios de soporte como Redis y RabbitMQ, que gestionan el almacenamiento en caché y las colas de mensajes.

Aprenderás a instalar y conectarte a cada base de datos, a trabajar con ORM como Mongoose y Sequelize, y a comprender conceptos clave del modelado de datos, como esquemas, colecciones y documentos. También compararás opciones locales y alojadas en la nube, y explorarás cuándo usar cada herramienta según las necesidades de tu aplicación, ya sea para crear un prototipo rápido, una API web escalable o un procesador de tareas en segundo plano.

¿Por qué son importantes las bases de

datos? Las bases de datos permiten a las aplicaciones conservar datos entre sesiones, gestionar la entrada del usuario y recuperar información de forma eficiente. Node admite una amplia gama de tipos y controladores de bases de datos. Elegir la adecuada depende de la escala, la complejidad y la estructura de los datos de su proyecto.

Las aplicaciones modernas necesitan almacenar una gran cantidad de datos, como información de usuarios, productos o transacciones. Sin una base de datos, se tendría que usar almacenamiento temporal (por ejemplo, variables en memoria), que pierde datos cada vez que se reinicia el servidor. Una base de datos ayuda a mantener la consistencia de los datos, admite consultas complejas y se adapta al crecimiento de la aplicación.

En este apéndice, cubriremos:

MongoDB

Una base de datos NoSQL orientada a documentos ideal para datos flexibles tipo JSON

PostgreSQL

Una base de datos SQL relacional con potentes funciones y compatibilidad con ACID

SQLite

Una base de datos SQL liviana basada en archivos para desarrollo local o uso integrado

Cada sección incluye instrucciones de instalación, configuración de la conexión y ejemplos de uso utilizando controladores nativos y ORM populares como Mongoose y Sequelize.

Conceptos básicos: esquemas, tablas, colecciones y documentos

Antes de profundizar en las bases de datos específicas, aquí hay algunos términos clave y cómo se relacionan con los diferentes tipos de bases de datos:

Concepto	bases de datos SQL	NoSQL
Esquema	Estructura de la base de datos definición (tablas, columnas)	Modelo de mangosta definición de MongoDB
Mesa	Filas de estructuradas archivos	N/A (utilizar colecciones)
Recopilación	N/A (tablas en su lugar)	Grupos de documentos (similar a las tablas)
Documento	N/A (filas en su lugar)	Individual tipo JSON objetos

CONSEJO

Las bases de datos SQL utilizan un esquema fijo, mientras que las bases de datos NoSQL permiten estructuras de documentos flexibles.

MongoDB: NoSQL para esquemas flexibles

MongoDB almacena datos en colecciones de documentos tipo JSON, lo que facilita

Es ideal para aplicaciones con datos no estructurados o semiestructurados.

Los siguientes atributos lo hacen especialmente adecuado para Node
aplicaciones:

Orientado a documentos

MongoDB almacena datos como “documentos” en lugar de utilizar los métodos tradicionales.
Tablas. Estos documentos tienen un formato similar a JSON: pares clave-valor
simples como { "nombre": "Jon", "edad": 30 }. Esto es

Intuitivo para los desarrolladores de JavaScript porque se parece mucho a los Objetos JavaScript.

Base de datos NoSQL

A diferencia de las bases de datos SQL tradicionales, MongoDB es flexible y no requiere una estructura fija. Esto significa que puedes ajustar tus modelos de datos sin muchos problemas, lo cual es ideal para procesar rápidamente aplicaciones en evolución.

Escalabilidad

El escalamiento horizontal de MongoDB favorece el crecimiento de su aplicación. Las escalas, mientras que la E/S sin bloqueo de Node garantiza la capacidad de respuesta.

Instalación

Siga las instrucciones para su plataforma:

Plataforma	Instrucciones de instalación
macOS (Cerveza casera)	grifo de cerveza mongodb/brew && brew install mongodb-community
Ubuntu/Debian	https://oreilly.com/catalog/linfbg4/
Ventanas	https://oreilly.com/catalog/linfbg4/

ACERCA DE HOMEBREW PARA MACOS

Homebrew es un popular administrador de paquetes para macOS que simplifica la instalación de software al automatizar el proceso de descarga, descompresión, compilación y configuración de aplicaciones y bibliotecas.

Para instalar Homebrew, abra una nueva ventana de terminal y ejecute:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Para instalar MongoDB con Homebrew, ejecute los siguientes comandos:

```
brew tap mongodb/brew ❶  
brew install mongodb-community@5.0 ❷
```

❶ Añade el tap oficial de MongoDB Homebrew a tus fuentes de Homebrew. Un tap es un repositorio de fórmulas de Homebrew que puedes instalar.

❷ Instala la versión 5.0 de MongoDB Community Edition usando Homebrew.

Instalación de Windows

En Windows, MongoDB se instala utilizando la base de .instalador msi que configura datos como un servicio del sistema para facilitar su uso:

1. Visite la página de instalación de MongoDB para Windows y descargue el instalador.
2. Ejecute el instalador haciendo doble clic en el .archivo msi y lo siguiente Las instrucciones de instalación.

CONSEJO

Asegúrese de que la opción "Instalar MongoDB como servicio" esté seleccionada, para que MongoDB se inicie automáticamente después de la instalación.

Una vez completada la instalación, abra el Símbolo del sistema como administrador y ejecute el siguiente comando para iniciar MongoDB:

```
inicio neto MongoDB
```

Para detener MongoDB, utilice:

```
parada neta MongoDB
```

Instalación de Linux

Siga las **instrucciones de instalación oficiales** Para garantizar que su entorno esté configurado correctamente para MongoDB, ejecute el siguiente comando en una ventana de línea de comandos para iniciar MongoDB:

```
sudo systemctl start mongod
```

CONSEJO

Utilice los comandos de servicio MongoDB `sudo service mongod start` para iniciar y `sudo service mongod stop` para detener MongoDB.

Iniciando MongoDB localmente

Después de la instalación, puede ejecutar el servidor MongoDB usando:

```
mongod --dbpath ~/datos/db
```

CONSEJO

Asegúrese de que el proceso mongod se esté ejecutando antes de conectarse a su base de datos.

Conectando con Mongoose

Para conectarse en su aplicación, puede utilizar el controlador oficial de MongoDB o Mongoose (un ORM de nivel superior):

```
importar mongoose de 'mongoose';

esperar mongoose.connect('mongodb://localhost:27017/myapp');

const Usuario = mongoose.model('Usuario',
  { nombre: String,
    correo electrónico:
      String });

esperar nuevo Usuario({ nombre: 'Alice', email: 'alice@example.com' }).guardar();
```

Mongoose proporciona validación, middleware y aplicación de esquemas sobre MongoDB.

PostgreSQL: poder y estructura de SQL PostgreSQL es una popular base de datos relacional de código abierto conocida por su confiabilidad, consultas complejas y soporte para esquemas, uniones y transacciones.

Instalación

Siga las instrucciones para su plataforma:

Plataforma	Instrucciones de instalación
macOS (Homebrew)	brew install postgresql
Ubuntu/Debian	sudo apt install postgresql
Ventanas	https://oreilly.com/catalog/errata/csp/errata.html

Para iniciar la base de datos localmente:

```
pg_ctl -D /usr/local/var/postgres start
```

Luego puede usar herramientas CLI o GUI de psql como pgAdmin para administrar bases de datos.

Conectando con Sequelize

Sequelize es un ORM popular que asigna tablas SQL a JavaScript modelos:

```
importar { Sequelize, DataTypes } de 'sequelize';

constante sequelize = nuevo
Sequelize('postgres://usuario:contraseña@localhost:5432/mibasededa');

const Usuario = sequelize.define('Usuario', {
  nombre: DataTypes.STRING,
  correo electrónico: DataTypes.STRING
});

esperar sequelize.sync();
esperar Usuario.create({ nombre: 'Bob', correo electrónico: 'bob@ejemplo.com'
});
```

NOTA

Sequelize funciona con PostgreSQL, MySQL y SQLite utilizando el mismo interfaz.

SQLite: Ligero e integrado

SQLite es un motor SQL basado en archivos, ideal para crear prototipos o aplicaciones con Baja concurrencia. No requiere un servidor: los datos se almacenan en `archivo.sqlite`.

Instalación

Siga las instrucciones para su plataforma:

Plataforma	Instrucciones de instalación
macOS	<code>brew install sqlite</code>
Ubuntu/Debian	<code>sudo apt install sqlite3</code>
Ventanas	https://oreilly.com/catalog/errata/csp/tcsec

Puede crear e inspeccionar bases de datos utilizando la CLI:

```
base de datos de desarrollo sqlite3
```

```
> CREAR TABLA usuarios (nombre TEXTO, correo electrónico TEXTO);  
> .salir
```

Conectando con Sequelize

Puedes reutilizar Sequelize para SQLite:

```
const sequelize = new Sequelize({ dialecto:
  'sqlite', almacenamiento:
  './dev.db' });

const Tarea = sequelize.define('Tarea', { descripción:
  DataTypes.STRING, completado:
  DataTypes.BOOLEAN });

await sequelize.sync(); await
Task.create({ descripción: 'Escribir el capítulo del libro', completado:
falso });
```

Como alternativa, better-sqlite3 ofrece un controlador sincrónico y liviano sin sobrecarga de ORM.

Bases de datos locales versus bases de datos alojadas en la nube

Cuándo utilizar bases de datos locales:

- Prototipado o desarrollo en su propia máquina
- Sin internet ni necesidad de persistencia fuera de línea
- Aplicaciones integradas (por ejemplo, de escritorio o móviles)

Cuándo utilizar bases de datos alojadas en la nube:

- Colaboración en equipo y copias de seguridad
- Alta disponibilidad, escalabilidad, acceso remoto
- Servicios como MongoDB Atlas, Amazon RDS (PostgreSQL) o Supabase

Las bases de datos en la nube a menudo brindan acceso sencillo al panel de control, autenticación, escalabilidad y herramientas de respaldo que ahorran tiempo de configuración en entornos de producción.

Servicios en la nube

Para escalar su aplicación más allá del desarrollo local, cada una de estas bases de datos se puede implementar en servicios alojados en la nube que ofrecen infraestructura administrada, copias de seguridad y alta disponibilidad.

Para MongoDB, [MongoDB Atlas](#) Proporciona una solución en la nube totalmente administrada con implementación con un solo clic en AWS, Azure o GCP.

PostgreSQL cuenta con un amplio soporte a través de servicios como [Amazon RDS](#), [Google Cloud SQL](#) y [Supabase](#). permitiéndole liberarse de la aplicación de parches, escalado y seguridad.

SQLite no suele usarse en entornos de producción en la nube, pero para casos de uso livianos y compatibles con el borde, puede explorar [Cloudflare D1](#) o [Turso](#).

Conectar tu aplicación Node a estos servicios suele implicar actualizar la URL de tu base de datos con una cadena de conexión segura y ajustar la configuración de tu ORM o controlador. Muchos proveedores de nube también ofrecen monitorización integrada, registro de consultas e integración con VPC o gestores de secretos para mantener tus credenciales seguras.

Comparación de bases de datos para aplicaciones de nodos

A continuación se muestra una comparación de bases de datos comunes que se pueden utilizar con Aplicaciones de nodo:

Base de datos	Tipo	Fortalezas	Debilidades	Suita caso
MongoDB	NoSQL (Basado en documentos)	Esquema flexible, escalable, Compatible con JavaScript	Uniones complejas, aprendizaje curva	Datos de Dyna, Scala aplicaciones pesadas
PostgreSQL	SQL (Relacional)	Cumplimiento de ACID, consultas potentes, funciones robustas	Cambios de esquema complejos, aprendizaje más pronunciado curva	Con consulta estructura datos, análisis
SQLite	SQL (Incorporado)	Ligero, sin servidor Requerido, configuración sencilla	Escalabilidad limitada, escritor único	Pequeño proyecto proto CLI t
MySQL	SQL (Relacional)	Fácil configuración, amplio soporte	Funciones avanzadas limitadas	Datos de estructura web
Redis	NoSQL (clave-valor)	Extremadamente rápido, excelente para el almacenamiento en caché.	No persistente de forma predeterminada, limitado por memoria	Datos reales en caché, sesión

Elección de una capa de persistencia

Tenga en cuenta estas características al hacer la elección para su proyecto:

Característica	MongoDB PostgreSQL		SQLite
Flexibilidad del esquema	Alto	Medio (rígido)	Medio (rígido)
Hospedaje	Local + Nube (Atlas)	Local + Nube (RDS, Supabase)	Solo local
Poder de consulta	Medio (JSON)	Alto (uniones, índices)	Bajo a Medio
Caso de uso	Aplicaciones con mucho contenido JSON	Lógica relacional, analítica	Prototipado, Herramientas CLI

Cada base de datos tiene sus ventajas y desventajas. Este libro ofrece ejemplos prácticos de las tres, que encontrará en diferentes proyectos a lo largo de los capítulos.

Si bien las bases de datos son esenciales para la persistencia de datos a largo plazo y las consultas estructuradas, muchas aplicaciones modernas de Node también dependen de herramientas de soporte de alta velocidad para mejorar la capacidad de respuesta y la escalabilidad. Dos herramientas comunes son Redis y RabbitMQ, que no se utilizan para el almacenamiento principal, sino para datos temporales, procesamiento en segundo plano y comunicación asíncrona. Estas herramientas complementan la base de datos principal al permitir un almacenamiento en caché rápido, colas de tareas eficientes y mensajería en tiempo real.

Redis: Velocidad en memoria para almacenamiento en caché y colas. Redis es un almacén de datos en memoria que se utiliza a menudo para almacenamiento en caché, almacenamiento de sesiones y colas de mensajes ligeras. Destaca por su velocidad y...

simplicidad, lo que lo convierte en una excelente opción para partes críticas para el rendimiento de su Aplicación de nodo, como almacenar datos temporales o administrar colas de trabajos.

Redis admite múltiples tipos de datos, incluidas cadenas, listas, conjuntos y hashes. Muchos frameworks de Node se integran bien con Redis para almacenamiento en caché, Pub/Sub y trabajos en segundo plano.

Instalación

Instale Redis usando el administrador de paquetes de su plataforma:

Plataforma	Instrucciones de instalación
macOS (Homebrew)	<code>brew install redis</code>
Ubuntu/Debian	<code>sudo apt install redis</code>
Ventanas	https://oreil.ly/ZdMWj

Para verificar que Redis esté funcionando, inicie el servidor Redis y conéctese usando la CLI:

```
servidor redis
```

Luego en otra terminal:

```
redis-cli
> SET prueba "hola"
> Prueba GET
```

macOS

Utilice Homebrew para instalar y ejecutar Redis:

```
brew instalar redis  
brew servicios iniciar redis
```

Ahora puedes conectarte a través de redis-cli.

Ubuntu/Debian

Instalar e iniciar el servidor Redis:

```
sudo apt update  
sudo apt install redis sudo  
systemctl start redis
```

Prueba con:

```
ping de redis-cli
```

Debería responder con PONG.

Ventanas

Redis no es compatible oficialmente con Windows, pero puedes ejecutarlo con el Subsistema de Windows para Linux (WSL) o Docker. Para Docker:

```
docker run -p 6379:6379 redis
```

Luego, conéctese usando un cliente Redis o redis-cli.

Conectarse desde Node Para conectarse

desde una aplicación Node, utilice el paquete ioredis o redis .

```
npm instalar ioredis
```

Ejemplo de uso:

```
importar Redis desde 'ioredis';

constante redis = nueva Redis();

await redis.set('saludo', '¡Hola, Redis!'); const valor = await
redis.get('saludo'); console.log(valor);
```

Redis es ideal para:

- Almacenamiento en caché de respuestas de API o consultas de base de datos
- Almacenamiento de datos de sesión para usuarios conectados
- Implementación de funciones en tiempo real mediante Pub/Sub
- Gestión de colas de tareas ligeras

Usar Redis como cola: Puedes usar listas de Redis

para implementar una cola básica. Para agregar un trabajo:

```
await redis.lpush( 'trabajos',

JSON.stringify({ tipo:
'correo electrónico',
para: 'alice@example.com', }) );
```

Para procesar trabajos:

```
mientras (verdadero)
{ const trabajo = await redis.rpop('trabajos'); si
(trabajo)
{ const datos = JSON.parse(trabajo);
console.log('Procesando:', datos);
}
}
```

NOTA

Para aplicaciones de producción, considere usar una biblioteca como Bull o marcos de trabajo basados en Redis que agregan características como reintentos y límites de velocidad.

Redis funciona bien para colas simples y la gestión de trabajos temporales, pero cuando su aplicación requiere funciones de mensajería más robustas, como garantías de entrega, enrutamiento o distribución de la carga de trabajo, un agente de mensajes dedicado es la mejor opción. RabbitMQ cumple esa función, ofreciendo un enfoque más estructurado para la gestión de tareas en segundo plano y la comunicación entre servicios.

RabbitMQ: Mensajería basada en colas para aplicaciones de Node. RabbitMQ

es un

agente de mensajes que ayuda a escalar su aplicación al delegar tareas a trabajadores en segundo plano. En **el capítulo 11**, aprenderá a integrar RabbitMQ en sus aplicaciones de Node. En lugar de realizar operaciones que consumen mucho tiempo en el ciclo de solicitud principal, puede publicar mensajes en una cola y dejar que otros servicios los procesen. Esto mejora el rendimiento y desacopla las diferentes partes de su sistema.

Instalación

RabbitMQ requiere el entorno de ejecución Erlang, que se incluye automáticamente en la mayoría de las plataformas cuando instala RabbitMQ.

Plataforma	Instrucciones de instalación
macOS (Homebrew)	brew install rabbitmq
Ubuntu/Debian	sudo apt install rabbitmq-server
Ventanas	https://oreilly.com/catalog/lin/hh/

Para comprobar si RabbitMQ se está ejecutando, utilice:

```
estado de rabbitmqctl
```

macOS

Instalar RabbitMQ usando Homebrew:

```
instalar rabbitmq
```

Servicios de elaboración de cerveza inician RabbitMQ

Para habilitar la interfaz de usuario basada en web:

Los complementos de rabbitmq habilitan la administración de rabbitmq

Visita <http://host:15672> e iniciar sesión con invitado / invitado.

Ubuntu/Debian

Instalar RabbitMQ con:

```
sudo apt update
sudo apt install rabbitmq-server
sudo systemctl start rabbitmq-server
```

A continuación, habilite el complemento de administración:

`sudo rabbitmq-plugins habilita la administración de rabbitmq`

El panel de control estará disponible en `http://host local :15672`.

Ventanas

1. `Instale el entorno de ejecución de Erlang.`
2. `Descargue el instalador de RabbitMQ.`
3. Después de la instalación, abra una terminal y ejecute:

Los complementos de RabbitMQ habilitan RabbitMQ_Management
inicio de `rabbitmq-service.bat`

Luego puede acceder a la interfaz de administración en `http:// host local :15672`.

Conectando desde el nodo

Utilice el paquete `amqplib` para enviar y recibir mensajes:

```
npm instalar amqplib
```

Ejemplo: enviar un pedido a una cola

```
importar amqp desde 'amqplib';

const conn = await amqp.connect('amqp://localhost');
const canal = await conn.createChannel();
const cola = 'pedidos';

esperar canal.assertQueue(cola);
canal.sendToQueue(cola, Buffer.from('café'));
```

Ejemplo: consumir desde una cola

```
importar amqp desde 'amqplib';
```

```
const conn = await amqp.connect('amqp://localhost'); const canal =  
await conn.createChannel(); const cola = 'pedidos';  
  
esperar canal.assertQueue(cola);  
canal.consume(cola, msg => { if (msg)  
  
    { console.log('Recibido:', msg.content.toString()); canal.ack(msg);  
  
    } } );
```

CONSEJO

Utilice RabbitMQ para pasar mensajes entre microservicios, descargar trabajo de su servidor principal o implementar colas de tareas.

Próximos pasos

A lo largo de este libro, utilizará cada una de estas bases de datos en proyectos reales: desde un administrador de tareas con SQLite hasta una API REST completa respaldada por MongoDB y un sistema de autenticación con PostgreSQL. También construirá sistemas que aprovechen Redis para el almacenamiento en caché y los datos temporales, así como RabbitMQ para administrar trabajos en segundo plano y la comunicación entre servicios.

Asegúrate de haber instalado al menos un motor y controlador de base de datos antes de continuar con el siguiente capítulo. Si ejecutas proyectos Docker), consulta los servicios en archivos yml (incluimos preconfigurados de docker-compose para cada base de datos y cola para empezar rápidamente.

NOTA

Algunos capítulos de este libro requieren una base de datos o un sistema de colas. Configurar al menos uno ahora le ayudará a seguirlo sin problemas más adelante.

Resumen

En este apéndice aprendiste:

- Cómo elegir y configurar bases de datos como MongoDB, PostgreSQL y SQLite en proyectos Node
- Cuándo utilizar herramientas de soporte como Redis y RabbitMQ para el almacenamiento en caché y el procesamiento en segundo plano
- Las compensaciones entre los servicios alojados localmente y en la nube para el desarrollo y la implementación

Apéndice D. Trabajar con ejemplos de código y contenerizar proyectos

Cada capítulo de este libro incluye ejemplos de código de Node funcionales para reforzar los conceptos que estás aprendiendo. Estos ejemplos están organizados, probados y mantenidos en un repositorio público de GitHub. Este apéndice te muestra cómo acceder al código, ejecutarlo localmente y, opcionalmente, usar Docker para simplificar la configuración, especialmente si trabajas en entornos aislados o en varias máquinas.

Acceder al repositorio de GitHub El repositorio oficial de

GitHub Los ejemplos de código de este libro están estructurados por capítulo. Dentro de cada carpeta de capítulo, encontrará subdirectorios para cada proyecto o sección temática, como:

```
oreilly-node-proyectos-código/ |— capítulo_1/
| |— 1_1/ | |— ❶
| |— 1_2/ | |— ❷
| |— capítulo_2/
| ...
|_ ...
```

❶ Primera sección del **Capítulo 1**

❷ Segunda sección del **Capítulo 1**

Esta estructura facilita la navegación y la búsqueda del código correspondiente a cada capítulo. Además, podrás trabajar con los capítulos.

En segmentos más fáciles de digerir. Cada carpeta de proyecto contiene su propio archivo `package.json`, lo que permite instalar dependencias y ejecutar scripts sin afectar a otros proyectos.

Para empezar, puedes clonar el repositorio o bifurcarlo a tu propia cuenta de GitHub. Esto te permite experimentar con el código, hacer cambios y guardar tu progreso.

Para clonar el repositorio (recomendado para la mayoría de los usuarios), navegue en su línea de comando a un directorio donde desee almacenar el código en su computadora y ejecute el siguiente comando:

```
clon de git https://github.com/JonathanWexler/oreilly-node-projects-code.git
```

A continuación, navegue a cualquier subdirectorio de capítulo o proyecto y siga las instrucciones del archivo `README` correspondiente a esa sección. Por ejemplo, para ejecutar el primer proyecto del Capítulo 1, introduzca los siguientes comandos en la línea de comandos:

```
cd oreilly-node-projects-code/capítulo_1/1_1 npm install  
npm start
```

Quizás prefieras bifurcar el repositorio a tu propia cuenta de GitHub. Esto te permite guardar los cambios y publicar tus propias modificaciones a medida que trabajas en los proyectos de capítulos. Para ello, ve a GitHub y haz clic en Bifurcar. Luego, haz clic en el nombre de tu cuenta de GitHub.

Esto creará una copia del repositorio en tu cuenta. Después, puedes clonar el repositorio bifurcado en tu equipo local con el siguiente comando:

```
clon git https://github.com/su-nombre-de-usuario/oreilly-node-projects-code.git
```

Después de la clonación, navegue a cualquier capítulo o subdirectorio del proyecto y siga las instrucciones del archivo README para esa sección.

CONSEJO

Cada carpeta es autónoma e incluye su propio packagejson, por lo que no es necesario instalar dependencias globales ni ejecutar ningún script fuera del alcance del proyecto.

Ahora puede seguir el libro, ejecutando cada proyecto a medida que avanza. Si encuentra algún problema, no dude en abrir una incidencia en el repositorio de GitHub. La siguiente sección explica cómo ejecutar los ejemplos de código con Docker, lo que simplifica la configuración y garantiza la coherencia en todos los entornos.

Ejecutar proyectos con Docker Node es fácil de instalar,

pero administrar múltiples proyectos, entornos o dependencias a nivel de sistema operativo aún puede ser una molestia, especialmente entre equipos o en máquinas nuevas.

Docker soluciona este problema al permitir ejecutar cada proyecto dentro de un contenedor aislado, utilizando un entorno definido. Esto garantiza que:

- Todos ejecutan la misma versión de Node, independientemente de la configuración local.
- No contaminas tu entorno global con diferentes cadenas de herramientas o archivos de configuración.
- El tiempo de configuración se reduce: simplemente clone el repositorio y ejecute el contenedor Docker.

Puedes ejecutar cualquier proyecto de este libro usando Docker. Cada capítulo La carpeta incluye un archivo docker-compose.yml que inicia todos los proyectos en Contenedores aislados con configuración mínima.

NOTA

Para reducir la repetición, algunos usuarios podrían preferir un docker.yml de nivel Componga raíz que haga referencia a todos los capítulos. Sin embargo, este libro proporciona uno por capítulo para simplificar.

Para comenzar, necesitará instalar Docker. Siga las instrucciones para su sistema operativo:

Plataforma	Instrucciones de instalación
macOS (Intel/Apple Silicon)	https://oreil.ly/twX1A
Windows 10/11 (Pro, Home, WSL2)	https://oreil.ly/TSk16
Ubuntu/Debian Linux	https://oreil.ly/1Rznz

Después de la instalación, verifique que Docker funcione ejecutando lo siguiente comandos en su línea de comandos:

```
docker --versión
versión de Docker Compose
```

CONSEJO

En Windows, Docker Desktop requiere WSL2 (se le solicitará durante el proceso) instalar).

Con Docker instalado, ahora puedes ejecutar los ejemplos de código en este libro usando contenedores a través del repositorio que clonó en el Sección anterior. Cada carpeta de capítulo contiene su propio archivo Componga ·docker-yml . Por ejemplo, para ejecutar el primer proyecto del **Capítulo 1**, y navegue hasta la carpeta del capítulo en su línea de comandos y ejecute `cd capítulo_1` y luego `docker compose`.

Este comando creará e iniciará el contenedor para ese proyecto.

También puede ejecutar `docker compose up -d` para ejecutarlo en modo separado (en segundo plano). Verá una salida que indica que el contenedor está en ejecución, junto con todos los registros de la aplicación.

NOTA

Si encuentra algún problema, asegúrese de que Docker se esté ejecutando y de que tener los permisos correctos para ejecutar comandos de Docker. Si necesita...

Para reconstruir el contenedor después de realizar cambios, puede ejecutar Docker. componer compilación seguido de `docker compose up --build`.

Cada proyecto se compilará y ejecutará en su propio contenedor. Puedes abrir el aplicaciones en su navegador que utilizan el puerto indicado en `docker-compose.yml`— por ejemplo, `http://localhost:3001`, etc. Cuando `://` `:3002`,

Ya terminaste, puedes detener los contenedores con Docker Compose.

hacia abajo. Esto eliminará los contenedores pero conservará las imágenes, por lo que Puede iniciarlos nuevamente más tarde sin reconstruirlos.

El siguiente es un ejemplo de un Dockerfile para un proyecto Node.

Este Dockerfile define los pasos para crear una imagen Docker para

Una aplicación de Node. Comienza con una imagen base oficial de Node 20.

Garantizando un tiempo de ejecución consistente y actualizado. El directorio de trabajo

La instrucción establece el directorio de trabajo dentro del contenedor en

`/usr/src/app`, donde se ejecutarán todos los comandos subsiguientes.

El comando `COPY . .` copia los archivos del proyecto desde su máquina local

En el contenedor. A continuación, npm install instala las dependencias.
definido en packagejson. Finalmente, el contenedor está configurado para ejecutarse.
npm se inicia de forma predeterminada cuando se inicia, lo que inicia su
aplicación basada en el script definido en su packagejson. Esto
La configuración proporciona un entorno limpio y aislado para ejecutar cualquier nodo.
proyecto.

DESDE el nodo:20
WORKDIR /usr/src/app
COPIAR . .
EJECUTAR npm install
CMD ["npm", "inicio"]

CONSEJO

Puede optar por agregar un archivo dockerignore para excluir archivos innecesarios
De la imagen. Es similar a gitignore, pero para Docker. Por ejemplo,
Es posible que desees ignorar node_modules, npm-debug.log y otros
archivos locales que no son necesarios en el contenedor.

También puede utilizar comandos para crear y ejecutar un contenedor Docker para
Un proyecto específico. Por ejemplo, el comando docker build -t
capítulo_1-1 . crea una imagen de Docker desde el directorio actual
utilizando el Dockerfile y lo etiqueta con el nombre capítulo_1-1.
El comando docker run -p 3000:3000 capítulo_1-1 inicia
Un contenedor de esa imagen y asigna el puerto 3000 dentro del contenedor
al puerto 3000 en su máquina local, lo que hace que la aplicación sea accesible en
<http://localhost:3000> . Este enfoque es útil para ejecutar un proyecto.
a la vez que tiene control total sobre su red y su entorno de ejecución.

Observarás que el repositorio de GitHub incluye un archivo docker-
Componga .yml en cada carpeta de capítulo. Este archivo define cómo ejecutar
todos los proyectos de ese capítulo como contenedores separados. Puede usar
Docker Compose para administrar múltiples contenedores fácilmente.

Por ejemplo, el **capítulo 1** de docker-compose.yml se ve así:

```
versión: "3.9" servicios:
  capítulo1_1:
    compilación: ./
    capítulo_1/1_1 puertos: -

    "3000:3000" capítulo1_2:
    compilación: ./
    capítulo_1/1_2 puertos: - "3001:3000"
```

De esta manera, cada capítulo estará disponible en su propio puerto local y Docker se encargará del aislamiento del entorno, la gestión de dependencias y las versiones consistentes de Node.

Beneficios de contenerizar proyectos de nodos

El uso de Docker tiene varias ventajas:

Consistencia

Todos ejecutan el mismo código en el mismo entorno.

Sencillez

No es necesario instalar manualmente Node, npm o cualquier dependencia global directamente en su computadora.

Aislamiento

Cada proyecto se ejecuta independientemente con su propio sistema de archivos y dependencias.

Portabilidad

Ejecute el mismo contenedor en CI/CD, desarrollo o incluso plataformas en la nube como AWS o Vercel.

Incluso si eres nuevo en Docker, los ejemplos de este libro están diseñados para facilitarte los primeros pasos. Una vez que configures un capítulo con un contenedor Docker, el resto seguirá el mismo patrón.

Próximos pasos:

A medida que avance en este libro, siéntase libre de clonar, bifurcar o contenerizar los ejemplos para que se adapten a su flujo de trabajo de desarrollo preferido. Ya sea que codifique localmente o ejecute contenedores, los ejemplos están diseñados para funcionar de forma aislada y son fáciles de comenzar. Descubrirá que tener un entorno consistente, especialmente al cambiar entre capítulos, le ahorrará tiempo y evitará muchos de los problemas de configuración típicos que experimentan los desarrolladores.

Para un uso más avanzado de Docker, como montaje de volumen, anulaciones específicas del entorno o compilaciones de producción, consulte la [documentación oficial de Docker y Node](#).

Resumen

En este apéndice aprendiste:

- Cómo clonar o bifurcar el repositorio oficial de GitHub y ejecutar ejemplos de código localmente para cada capítulo
- Cómo contener proyectos usando Docker y docker-compose para entornos consistentes y aislados
- Los beneficios de usar contenedores para simplificar la configuración, administrar dependencias y garantizar flujos de trabajo de desarrollo repetibles

Apéndice E. Configuración de cuentas de desarrollador y credenciales de API

Muchos de los proyectos de este libro dependen de servicios externos como OpenAI, Google Cloud, GitHub y MongoDB Atlas. Para usar estos servicios en su propio entorno de desarrollo, deberá crear cuentas, generar claves API o tokens, y configurar sus aplicaciones para que los usen de forma segura.

Este apéndice te guía por todo el proceso: desde el registro y la búsqueda de credenciales hasta su almacenamiento seguro en archivos de entorno y la comprensión de los límites de velocidad. Durante el proceso, aprenderás la diferencia entre las claves API y los tokens OAuth, cómo evitar la exposición de datos confidenciales y cómo las cuotas de servicio pueden afectar a tus aplicaciones.

Estas herramientas son esenciales en el desarrollo moderno de Node, y configurarlas con antelación ayudará a evitar obstáculos a medida que avanza. Las secciones a continuación están organizadas en el orden en que se utiliza cada servicio en el libro, para que pueda consultarlas según sea necesario durante los capítulos correspondientes. Pero primero, veamos los conceptos básicos del trabajo con claves API y tokens.

Trabajar con claves API El desarrollo

moderno con frecuencia implica la integración con servicios de terceros (como bases de datos, proveedores de pago, modelos de IA o plataformas de mapeo) a través de API (interfaces de programación de aplicaciones).

Para interactuar de forma segura con estos servicios, normalmente es necesario crear una cuenta y autenticar sus solicitudes mediante tokens API.

Un token de API es una cadena única de caracteres que actúa como una contraseña para el acceso programático. Cuando tu aplicación envía solicitudes a un servicio externo, incluye el token de API para verificar su identidad y garantizar que la solicitud esté autorizada. Los tokens suelen estar vinculados a cuentas o proyectos específicos y pueden incluir permisos para restringir las acciones que se pueden realizar.

Los tokens de API se consideran confidenciales y deben almacenarse de forma segura, generalmente en archivos `env` que no están comprometidos con el control de versiones. Cada usuario o aplicación generalmente recibe un token único, cuyo alcance puede limitar el acceso (por ejemplo, solo lectura o escritura). Muchas plataformas permiten regenerar o revocar tokens si se ven comprometidos o si es necesario modificar el acceso.

La creación de cuentas en plataformas externas (como OpenAI, Google Cloud, GitHub o MongoDB Atlas) es necesaria por varias razones. En primer lugar, la autenticación es fundamental: las API deben saber qué usuario o aplicación realiza una solicitud para evitar abusos y garantizar la rendición de cuentas. En segundo lugar, los tokens de API permiten a estos servicios rastrear el uso, aplicar cuotas y aplicar la facturación según el volumen de solicitudes. En tercer lugar, permiten una gestión precisa del acceso, como restringir acciones a usuarios o recursos específicos. Por último, este sistema mejora la seguridad al permitir aislar, supervisar y controlar el acceso a funciones o datos sensibles.

Las claves API son cadenas largas que se utilizan como contraseñas para identificar tu aplicación ante un servicio. Los tokens OAuth son más seguros y específicos del usuario: se utilizan cuando tu aplicación actúa en nombre de un usuario (por ejemplo, al acceder a su perfil de GitHub). Muchas API admiten ambos métodos, pero este libro utiliza claves API por defecto siempre que sea posible para simplificar.

CONSEJO

Nunca incorpore claves API en su código. En su lugar, utilice un archivo env en cada... proyecto. Asegúrese de instalar dotenv para cada proyecto para el que pretende almacenar claves privadas.

Almacenar claves en un archivo env no solo las mantiene fuera de su base de código pero también facilita la gestión de diferentes entornos, como desarrollo y producción.

NOTA

Añade siempre env a tu gitignore para evitar filtrar secretos.

Las siguientes secciones lo guiarán a través de la creación de cuentas y API. Generación de claves para los servicios que puede utilizar en este libro, comenzando con GitHub, que es esencial para el control de versiones y la colaboración.

GitHub

GitHub permite a los desarrolladores colaborar en el código en tiempo real, Mantener un historial detallado de los cambios del proyecto y gestionar Contribuciones de múltiples usuarios. También se integra con la automatización. herramientas como GitHub Actions para CI/CD, y le permite alojar documentación, sitios web estáticos y wikis directamente desde su repositorios. Para las personas, GitHub sirve como un portafolio de programación. Proyectos y una plataforma para contribuir al software de código abierto.

El Apéndice D proporciona los pasos para clonar el código de este libro desde su Repositorio público de GitHub. Puedes hacerlo sin una cuenta de GitHub. Sin embargo, si desea tener un mejor control sobre su propio progreso A lo largo del libro y los cambios que realice a lo largo del camino,

Benefíciate de tener una cuenta de GitHub y la línea de comandos de GitHub

herramientas.

Para crear una cuenta, siga estos pasos:

1. Visita <https://github.com>.
2. Haga clic en el enlace "Registrarse" en la esquina superior derecha.
3. Ingrese una dirección de correo electrónico válida y haga clic en Continuar.
4. Crea un nombre de usuario y una contraseña segura. Alternativamente, Es posible que se le solicite que configure un código de acceso con su navegador.
5. Verifique su dirección de correo electrónico utilizando el código enviado a su bandeja de entrada.
6. Elija el plan gratuito para comenzar.
7. Complete las preferencias de incorporación solicitadas.

Una vez creada tu cuenta, puedes bifurcar y clonar otras correctamente. repositorios, como el código de este libro. También puede marcar con una estrella y Mire los proyectos que le interesan para recibir notificaciones de actualizaciones y cambios. Lo más importante es que puedes crear tus propios repositorios para almacenar código. y proyectos personales.

Para usar GitHub desde la línea de comandos, deberá configurar Claves SSH o tokens de acceso personal (PAT). Visita GitHub. [SSH página de configuración de claves](#) y haga clic en "Nueva clave SSH". Genere una clave en su computadora local ejecutando `ssh-keygen -t ed25519 -C "your_email@gmail.com"`. Luego, agregue el contenido de `~/.ssh/ed25519.pub` a su perfil de GitHub.

Alternativamente, para acceder a repositorios privados o enviar a GitHub usando HTTPS, visite [la página de configuración de tokens de acceso personal](#) de GitHub y haga clic "Generar nuevo token (clásico)". Seleccione ámbitos como repositorio, flujo de trabajo, y leer:usuario, dependiendo de las características que necesite su proyecto, y guarde el token de forma segura; no podrá volver a verlo.

MongoDB Atlas: Crear

una cuenta de MongoDB Atlas te da acceso a una versión totalmente administrada y alojada en la nube de MongoDB, una de las bases de datos NoSQL más populares. Esto es especialmente útil para desarrolladores que crean aplicaciones web modernas, ya que permite almacenar documentos tipo JSON sin necesidad de instalar ni administrar la infraestructura de la base de datos. Atlas ofrece funciones como copias de seguridad automáticas, clústeres globales, paneles de monitorización y seguridad integrada. Al usar un clúster gratuito, puedes desarrollar y probar aplicaciones con funcionalidades de base de datos reales y escalarlas fácilmente posteriormente si es necesario.

NOTA

También puede optar por instalar MongoDB localmente en su máquina, pero se recomienda utilizar Atlas por su simplicidad y facilidad de uso.

Consulte la [documentación de MongoDB](#) o siga estos pasos para crear una cuenta gratuita de MongoDB Atlas:

1. Visite la [página web de la base de datos Atlas](#) y haga clic en "Comenzar gratis".
2. Regístrese usando su correo electrónico o una cuenta de GitHub/Google.
3. Después de verificar su correo electrónico, inicie sesión en el panel de control.
4. Haga clic en "Crear un clúster", elija el nivel gratuito M0, seleccione un región y créela.
5. Una vez creado el clúster, vaya a la pestaña "Acceso a la base de datos" y haga clic en "Agregar nuevo usuario de base de datos".
6. Establezca un nombre de usuario y una contraseña y elija "Leer y escribir en cualquier base de datos".
7. Vaya a la pestaña "Acceso a la red" y haga clic en "Agregar dirección IP".

8. Seleccione "Permitir acceso desde cualquier lugar" (0.0.0.0/0). Puede restringir esta opción más adelante para mayor seguridad.
9. Cuando esté listo, haga clic en "Conectar" y "Conectar su aplicación".
10. Copie la cadena de conexión proporcionada (por ejemplo, `mongodb+srv://<usuario>:<contraseña>@cluster...`).
11. Pegue esta cadena en el archivo `env` para su aplicación (por ejemplo, `MONGODB_URI=`).

CONSEJO

Puede usar la interfaz gráfica de MongoDB Compass para visualizar y administrar su base de datos. Descárguela del [sitio web de MongoDB](#).

Con su cadena de conexión en su lugar, ahora puede usar MongoDB Atlas en sus aplicaciones Node sin la necesidad de instalar MongoDB localmente.

API de OpenAI:

Crear una cuenta de API de OpenAI te da acceso a potentes modelos de lenguaje y generación de código como GPT-4 y Codex. Estas herramientas se pueden usar para crear aplicaciones inteligentes, como chatbots, asistentes de código, generadores de contenido y más. La API te permite enviar mensajes de texto y recibir respuestas generadas dinámicamente, lo que la hace ideal para mejorar las interfaces de usuario con capacidades de IA. Una cuenta gratuita ofrece créditos de uso limitados para que puedas empezar a experimentar de inmediato.

NOTA

Es posible que se le solicite que proporcione una tarjeta de crédito para verificación, pero no se le cobrará a menos que exceda los límites del nivel gratuito.

Visita el [sitio web de OpenAI](#) y sigue estos pasos para crear uno gratis

Cuenta API de OpenAI:

1. En el sitio web de OpenAI, regístrate usando su correo electrónico o una cuenta de Google/Microsoft.
2. Después de verificar su correo electrónico y número de teléfono, será llevado al panel de OpenAI.
3. Vaya a la [página de claves API](#) y haga clic en “Crear nuevo secreto” llave.”
4. Copie la clave generada y guárdela en un lugar seguro. Esta clave no volverá a aparecer.
5. Agregue la clave API al archivo env de su proyecto (por ejemplo, OPENAI_API_KEY=).

CONSEJO

Puede supervisar su uso, cuotas y facturación en la [página Uso](#).

Con su clave API configurada, puede comenzar a llamar a los modelos de OpenAI desde su aplicación usando solicitudes HTTP o bibliotecas de cliente como el paquete oficial Node de openai .

NOTA

La API de OpenAI no es de código abierto y requiere conexión a internet para funcionar. Deberá administrar su clave API de forma segura para evitar usos no autorizados.

API de Google Gemini. Crear

una cuenta de la API de Google Gemini a través de Google Cloud Console te da acceso a la familia de modelos de IA multimodales de Google (anteriormente Bard), capaces de razonar con texto, imágenes, código y más. Estos modelos se pueden usar en aplicaciones avanzadas como asistentes inteligentes, herramientas de interpretación de imágenes y plataformas para desarrolladores con IA. La API de Gemini forma parte de Google AI Studio y se integra con la plataforma de IA Vertex para flujos de trabajo empresa

NOTA

Para acceder a Gemini es necesario habilitar la facturación en un proyecto de Google Cloud, pero también puedes usar un nivel gratuito con un uso limitado.

Visita [la página de claves API](#) de Google AI Studio o siga estos pasos para crear una cuenta de API de Google Gemini y comenzar:

1. Vaya a la [consola de Google Cloud](#) e inicia sesión con tu cuenta de Google.
2. Haga clic en “Crear proyecto” para crear un nuevo proyecto o seleccionar uno existente.
3. Navegue hasta la [biblioteca API](#) y busque “Gemini API”.
4. Haga clic en la API y habilítela para el proyecto seleccionado.
5. Vaya a la [página de Credenciales](#) y haga clic en “Crear credenciales”.

6. Elija una clave API (para uso básico) o credenciales OAuth 2.0 (para flujos autenticados por el usuario).
7. Copie la clave generada y guárdela en su archivo env (por ejemplo, GEMINI_API_KEY=).
8. Opcionalmente, visite la [página de claves API](#) para generar una clave directamente a través de AI Studio.

CONSEJO

Puedes probar las indicaciones en el navegador usando el [área de juegos Gemini](#).

Una vez que su clave esté en su lugar, puede comenzar a llamar a los modelos Gemini en Node como se describe en el libro o en la [documentación de la API de Gemini](#).

LÍMITES DE TARIFA Y CUOTAS

Al trabajar con API, especialmente con servicios de terceros como OpenAI, Google Gemini o MongoDB Atlas, es fundamental comprender cómo los límites de velocidad y las cuotas afectan la estabilidad y la fiabilidad de la aplicación. Estos límites son impuestos por los proveedores para garantizar un uso justo de la infraestructura compartida y evitar abusos o sobrecargas del sistema.

Los límites de velocidad definen la cantidad de solicitudes que se pueden realizar en un periodo corto, generalmente por segundo, minuto u hora. Superar el límite de velocidad suele generar una respuesta HTTP 429 "Demasiadas solicitudes". Algunas API también incluyen un encabezado "Retry-After", que indica cuánto tiempo se debe esperar antes de reintentar.

Las cuotas definen su capacidad total de uso durante un período más largo, generalmente por día o por ciclo de facturación. Esto puede incluir:

- Número total de llamadas a la API
- Número de tokens generados
- Ancho de banda utilizado
- Solicitudes por modelo o punto final

Si excede su cuota, sus solicitudes pueden fallar hasta que la cuota se restablezca, es posible que se limite (reduzca la velocidad) automáticamente o podría incurrir en cargos por exceso si tiene un plan pago.

CONSEJO

Diseñe siempre sus aplicaciones con lógica de reintento, retroceso exponencial y degradación gradual en caso de limitación de velocidad. Por ejemplo, ponga en cola las solicitudes no críticas o reduzca la frecuencia en condiciones de uso intensivo.

Puede supervisar los límites de velocidad y las cuotas en el panel de control de su proveedor:

- **OpenAI**
- **Google Cloud** (busque en “IAM y administración” > “Cuotas”)

Resumen El

desarrollo moderno con Node.js suele implicar la conexión a servicios de terceros mediante claves API, tokens y credenciales seguras. En este apéndice, aprendiste a configurar cuentas con proveedores como GitHub, MongoDB Atlas, OpenAI y Google Cloud, así como a generar, almacenar y administrar las credenciales necesarias para acceder a sus API.

Siguiendo las mejores prácticas, como usar archivos `.env`, evitar secretos codificados y supervisar los límites de velocidad, podrá integrar servicios externos en sus aplicaciones de forma segura. Estos pasos de configuración optimizarán su experiencia en los proyectos de este libro y le prepararán para trabajar con API en entornos reales.

Índice

Símbolos

Variable \$ (signo de dólar), [análisis con herramientas compatibles con HTML](#)

(Obtener API), [Obtener programación](#)

(HTML

- obtener el contenido HTML del sitio web, [obtener programación](#)

(extraer páginas web y usar datos en una aplicación)

- obtener contenido HTML de una página externa), [Obtener programación](#)

(función de texto, que convierte contenido HTML en texto sin formato), [Obtener Programación](#)

Plugin @fastify/formbody: [comience a programar con un diseño de API](#)

Plugin @fastify/session: [Cómo guardar y proteger cuentas de usuario](#)

A

Clase de cuenta

- agregar un método de autenticación para [guardar y proteger usuarios](#)
[Cuentas](#)

- añadiendo el método genStrategy a, [Guardar y proteger usuarios](#)
[Cuentas](#)

- añadiendo el método passportAuthenticate a, [Guardar y proteger](#)
[Cuentas de usuario](#)

- agregar los métodos serializeUser y deserializeUser a, **Guardar y proteger cuentas de usuario**
- creando una clase JavaScript que extiende el modelo Sequelize.Account, **Guardar y proteger cuentas de usuario**
- Función setPassword, **Guardar y proteger cuentas de usuario**

Información de la cuenta, **Guardar y proteger cuentas de usuario**

Modelo de secuenciación de cuentas

- Aplicación de nodo totalmente configurable para usarse para registrarse y Autenticar nuevas cuentas, **Guardar y proteger usuarios Cuentas**
- configuración en Account.js, **guardar y proteger cuentas de usuario**

expresiones de gratitud

- acuse de recibo de mensajes para un procesamiento de cola confiable, **integrando un sistema robusto Sistema de mensajería**

Hashing adaptativo, **guardado y protección de cuentas de usuario**

Protocolo de cola de mensajes avanzado (AMQP), **que integra un sistema robusto Sistema de mensajería**

Conceptos avanzados de Node, **Dominando el oficio**

función agregada, definición, **construcción de un agregador**

IA (inteligencia artificial), **procesador de lenguaje natural, sentimiento Análisis**

- (ver también aprendizaje automático)

API de IA y LLM, **comience a programar**

Asistente de aprendizaje impulsado por IA, construcción, **Construcción de un asistente de aprendizaje impulsado por IA Asistente de aprendizaje con la API Gemini de Google - Resumen**

- personalizar el asistente, [Personalizar el Asistente de IA para Asistencia para el aprendizaje](#) : personalización del asistente de IA para el aprendizaje [Asistencia](#)
 - guiar la IA para enseñar programación, [personalizar la IA Asistente de asistencia para el aprendizaje](#)
- API Gemini de Google, utilizada para LLM, [Get Planning](#)
- preparación para entrevistas técnicas, [Su Aviso](#)
- ingeniería rápida, elaboración de instrucciones de IA efectivas, [Personalización del Asistente de IA para la asistencia en el aprendizaje](#)
- configuración de la base de datos y autenticación de usuarios, [Configuración de su Autenticación de base de datos y usuarios: configuración de la autenticación de base de datos y usuarios](#)
 - respuestas sensibles al contexto, [configuración de su base de datos y Autenticación de usuario](#)
 - Consultar al asistente de IA con perfiles de aprendizaje de usuario, [Configuración Su base de datos y autenticación de usuarios](#)
 - Sistema de registro e inicio de sesión de usuarios, [Configuración de su Autenticación de base de datos y usuario: configuración de su Autenticación de bases de datos y usuarios](#)
- configuración del servidor Fastify para, [Configuración del servidor Fastify-Configuración del servidor Fastify](#)
- arquitectura del sistema para la fase 1 y la fase 2, [Obtener planificación](#)
- Prueba de la solicitud de API de IA a Gemini, [Get Programming](#)

Validador de esquemas AJV, [uso de Fastify en este libro](#)

Amazon SQS: [Integración de un sistema de mensajería robusto](#)

AMQP (Protocolo de cola de mensajes avanzado), [que integra un protocolo robusto Sistema de mensajería](#)

Función `amqp.connect`: [Integración de un sistema de mensajería robusto](#)

Paquete `amqplib`, instalación, [integración de un sistema de mensajería robusto](#)

Claves API, trabajar con, [trabajar con claves API](#)

API, [Convertirse en desarrollador de Node, Web Scraper](#)

- acerca de, [API de la biblioteca](#)
- autenticación a, [Autenticación de la aplicación](#)
- autenticación, uso de JWT para, [uso de JWT para API](#)
[Autenticación: uso de JWT para la autenticación de API](#)
- agregador de contenido que integra feeds RSS y [contenido](#)
[Feed de agregación](#)
- crear, usar Fastify para un tiempo de respuesta rápido, [usar Fastify en Este libro](#)
- Proyecto de API de biblioteca, [Su resumen rápido](#)
 - Diseño de API de programación, [Obtenga programación con una API](#)
[Programación de diseño con API](#)
 - rutas y acciones, añadir a la aplicación, [Añadir rutas y](#)
[Acciones para su aplicación: cómo agregar rutas y acciones a su aplicación](#)
- límites de velocidad y cuotas para [la API de Google Gemini](#)

`API_URL`, [Obtener programación](#)

objeto de aplicación (Fastify), [Obtener programación](#)

variable de aplicación, [Obtener programación](#)

Función `app.get`, [Trabajando con Fastify](#)

Función app.listen, **Obtener programación con un diseño de API, Agregar un marco para su servicio de correo, Obtener programación**

Método app.post, **Creación de un formulario de inicio de sesión**

Función appendFileAsync: **traducción de la entrada del usuario a CSV**

Función de derivación de claves de Argon2, **Guardar y proteger cuentas de usuario**

Paquete asciichart: **Conexión de una base de datos y visualización**

- métodos, agregando a la visualización del análisis de sentimientos, **conectando una base de datos y visualización**

Funciones async/await, **Traducción de la entrada del usuario a CSV, Análisis Sentimiento**

- agregar a la aplicación de análisis de sentimientos para entradas de diario, **conectar una base de datos y visualización**
- ventajas para su uso en codificación, **guardado y protección de datos de usuario Cuentas**
- para el sistema de publicación/suscripción de Redis, **agregar un servidor Redis**
- Clientes de publicación/suscripción de Redis, **agregar un servidor Redis**

funciones asincrónicas

- Conexión a MongoDB con, **Guardar contraseñas con MongoDB**
- usar promisify para encapsular, **traducir la entrada del usuario a CSV**
- envoltura en promesa, **traducción de la entrada del usuario a CSV**

Programación asincrónica: **Cómo convertirse en desarrollador de Node**

autenticación

- autenticación de aplicaciones, **autenticación de aplicaciones**
 - (ver también la aplicación Node de autenticación de inicio de sesión)

- autenticar un usuario que inicia sesión, **guardar y proteger el usuario**

Cuentas

- definir variables de la página de autenticación en index.js, **crear un**

Formulario de inicio de sesión

- definir la estrategia para las cuentas de usuario, **guardar y proteger las cuentas de usuario**

Cuentas

- a la conexión de base de datos SQLite, **Conexión de una base de datos a su**

Aplicación

- uso de JWT para la autenticación de API, **uso de JWT para API**

Autenticación: uso de JWT para la autenticación de API

Paquete axios, **Obtenga programación, Obtenga programación**

B

Función hash bcrypt, **Guardar y proteger cuentas de usuario**

Paquete de hash bcrypt, **Cree un administrador de contraseñas local seguro,**

Configuración de su base de datos y autenticación de usuarios : instalación

y uso, **creación de un administrador de línea de comandos local**

- probar funciones en index.js, **crear una línea de comandos local**

Gerente

bloque (blockchain), **Obtener planificación**

- Minar un bloque, **codificar la blockchain**

Clase de bloque, **Codificación de la cadena de bloques**

- definiendo y **codificando la cadena de bloques**

Clase Blockchain, **Codificación de la Blockchain, Codificación de la Blockchain**

- pasos clave en el proceso de minería, **Codificación de la cadena de bloques**

Mercado blockchain para la distribución de música, **sello discográfico**

Resumen del mercado de blockchain

- sobre blockchain, **comience a planificar**
- agregar puntos finales /register-node y /sync-peers, **Obtener Programación**
- función de transmisión, **Obtener programación**
- funciones broadcastSelf y registerNode en el nodo de mercado, **Obtenga programación**
- codificando la cadena de bloques, **Codificando la cadena de bloques-Codificando la Cadena de bloques**
 - creando la función broadcastBlockchain, **codificando la Cadena de bloques**
 - creando la función calculateHash, **codificando la blockchain**
 - Definición de la clase Block, **Codificación de la Blockchain**
 - Definición de la clase Blockchain, **Codificación de la Blockchain**
- importar el paquete fastify y configurar el servidor web, **Obtener Programación**
- inicializando nueva instancia de MarketplaceNode, **obtener programación**
- integración de transacciones
 - definir/comprar ruta, **Codificando la Blockchain**
 - definir ruta de pago, **Codificación de la Blockchain**
 - definiendo la ruta /songs, **Codificando la Blockchain**
 - definir el método mineBlock, **Codificación de la cadena de bloques**
- nuevo nodo de mercado, **Get Programming**

- Clases de nodos que representan componentes fundamentales, [Obtener Planificación](#)
- planificación de la aplicación, [Su Aviso](#)
- Ejecución de un ejemplo del mundo real, [Ejecución del ejemplo del mundo real](#)
- iniciar servidores de nodos, salida de línea de comandos, [obtener programación](#)

blockchain, definición, [obtener planificación](#)

middleware de análisis corporal, [Get Programming](#)

Modelo de libro, definición y sincronización con la base de datos SQLite en la aplicación API de biblioteca, [conexión de una base de datos a su aplicación](#)

Objeto ORM de libro, acceso, [conexión de una base de datos a su aplicación](#)

Biblioteca CSS de Bootstrap: [creación de un formulario de inicio de sesión](#)

Transmisión en cadenas de bloques, [Obtener programación](#)

Método Buffer.from: [Integración de un sistema de mensajería robusto](#)

Función bufferBytes.toString: [Cómo guardar y proteger cuentas de usuario](#)

buffers, [Dominando el oficio](#)

```
do
```

devoluciones de llamada, [Dominando el oficio](#)

- El infierno de las devoluciones de llamadas, [traducción de la entrada del usuario a CSV](#)
- Fastify proporciona API de biblioteca, [obtenga programación con una API Disposición](#)
- función llamada cuando ocurre un evento, [Agregar un servidor Redis](#)
- procesamiento por el hilo principal del bucle de eventos, [Obtener planificación](#)

Plantillas de correo de campaña, [Implementación de un píxel de marketing para correo electrónico](#)
[Compromiso](#)

middleware de manejo de errores de captura general, [Obtenga programación con una API](#)
[Disposición](#)

canales

- Método `channel.assertQueue`, [Integración de una mensajería robusta](#)

[Sistema](#)

- creación en el servidor RabbitMQ, [integración de una mensajería robusta](#)

[Sistema](#)

Paquete cheerio, [análisis con herramientas compatibles con HTML](#)

cheerio, uso para analizar contenido HTML, [análisis con HTML-Friendly](#)

[Herramientas](#)

El motor JavaScript V8 de Chrome: [comprensión de Node](#)

Clases (Nodo) en blockchain, [Obtener planificación](#)

clientes (Redis), creación, [adición de un servidor Redis](#)

bases de datos alojadas en la nube

- expandiéndose a [los servicios en la nube](#)
- bases de datos locales versus [bases de datos alojadas en la nube](#)

Agrupamiento, uso para la escalabilidad de nodos, [Dominando el oficio](#)

ejemplos de código, trabajar con, [trabajar con los ejemplos de código y](#)
[Proyectos de contenerización](#)

- acceder al repositorio de GitHub, [Acceder al repositorio de GitHub](#)

aplicación de pedidos de café

- sistema de colas para

- colas en JavaScript), [Obtener programación](#)

Aplicación de pedidos de café, sistema de colas para, [administrador de pedidos de café](#)

Resumen

- agregar un servidor Redis, [Agregar un servidor Redis](#) – [Agregar un servidor Redis](#)
[Servidor](#)
- agregar una cola interna para gestionar solicitudes de pedidos, [Obtener planificación](#)
- agregar ruta para imitar la solicitud retrasada, [Obtener programación](#)
- agregar rutas para colocar orden en la cola, [Obtener programación](#)
- configurar la aplicación, [Obtener Programación](#)
- instalando Fastify, [empieza a programar](#)
- Integrar un sistema de mensajería robusto, [Integrar un sistema de mensajería robusto](#)
[Sistema de mensajería](#) : Integración de un sistema de mensajería robusto
- middleware que analiza las solicitudes entrantes, agregando, [Obtener](#)
[Programación](#)
- punto final de conteo de pedidos de cola, agregando, [Obtener programación](#)
- Servidor Redis para gestionar solicitudes de pedidos, [Obtener planificación](#)
- actualización con el servidor de cola de mensajes RabbitMQ, [Obtener planificación](#)

colecciones, [conceptos básicos: esquemas, tablas, colecciones y](#)

Documentos

archivos de valores separados por comas (ver archivos CSV)

interfaz de línea de comandos (CLI)

- solicitudes de línea de comandos para la entrada del usuario, [traducción de la entrada del usuario a](#)
[CSV](#)

- crear un mensaje para la computadora traduciendo la entrada del usuario a CSV,
Traducir la entrada del usuario a CSV
- crear un administrador de contraseñas de línea de comandos local, **crear un**
Administrador de línea de comandos local : creación de una línea de comandos local
Gerente

Indicador de computadora para crear formato tabular, o CSV, de datos (proyecto de ejemplo),
Su indicador de programación

- trabajar con paquetes externos, **trabajar con paquetes externos**
Paquetes: Trabajar con paquetes externos

Función de plantilla de confirmación de correo, **Conexión a una base de datos**

consenso (blockchain), **Obtener planificación**

Función console.log: **lectura y análisis de un feed**

Función console.table: **lectura y análisis de un feed**

Contenerización de proyectos, **Ejecución de proyectos con Docker: Próximos pasos**

- beneficios de, **Beneficios de contenerizar proyectos de nodos**
- ejecutar proyectos con Docker, **ejecutar proyectos con Docker-**
Ejecución de proyectos con Docker

Agregador de contenido que integra feeds RSS y API, **Contenido**

Resumen de la fuente de agregación

- agregar elementos personalizados al agregador, **Agregar elementos personalizados a**
Su agregador: cómo agregar artículos personalizados a su agregador
- plan de aplicación, **Obtener planificación**
- construyendo el agregador, **Construyendo un Agregador-Construyendo un**
Agregador

- Obtener API que realiza una solicitud a la URL de la fuente RSS de Bon Appétit, **Obtener Planificación**
- inicializando la aplicación, **Obtener planificación**
- analizar y leer desde una fuente RSS, **Leer y analizar un Lectura y análisis de feeds**

Inyección de contexto (ingeniería de indicaciones), **personalización del asistente de IA para la asistencia al aprendizaje**

Respuestas sensibles al contexto (asistente de IA), **configuración de su base de datos y autenticación de usuarios**

Cookies, **guardar y proteger cuentas de usuario**

Función createTransport (nodemailer), **Obtener programación**

Acciones CRUD (crear, leer, actualizar, eliminar), **agregar rutas y Acciones para su aplicación**

- Aplicación API de biblioteca conectada a la base de datos, **Conexión a una base de datos a tu aplicación**

Biblioteca de cifrado, **Guardar y proteger cuentas de usuario**

Algoritmo hash crypto.pbkdf2: **Cómo guardar y proteger cuentas de usuario**

Función crypto.pbkdf2Sync, **Guardar y proteger cuentas de usuario, Guardar y proteger cuentas de usuario**

Rompecabezas criptográfico (blockchain), **Obtener planificación**

CSS

- Biblioteca CSS de Bootstrap, **creación de un formulario de inicio de sesión**
- usar para diseñar la interfaz de usuario de la aplicación web del restaurante, **mejorando la interfaz de usuario- Mejorando la interfaz de usuario**

Archivos CSV, **Obtener planificación**

- método saveToCSV en la clase Persona (ejemplo), [Traducción de Usuario](#)
[Entrada a CSV](#)

- traducir la entrada del usuario a CSV, [Traducir la entrada del usuario a CSV-](#)
[Traducir la entrada del usuario a CSV](#)

Paquete csv-writer, instalación y uso, [trabajo con archivos externos](#)

[Paquetes](#)

utilidad cURL

- Comandos que prueban rutas en la API de la biblioteca de la aplicación conectada a la base de datos, [Conexión de una base de datos a su aplicación](#)
- ejecutándose contra el servidor API de la biblioteca, [agregando rutas y acciones a Tu aplicación](#)
- usar para publicar pedidos de bebidas, [obtener programación](#)

D

módulos de datos, conversión de datos de restaurantes, [adición de rutas y Datos](#)

bases de datos, [Convertirse en un desarrollador de Node](#)

- conectar una base de datos a una aplicación API de biblioteca, [conectar una Base de datos a su aplicación: Cómo conectar una base de datos a su aplicación](#)
 - definir Libro en books.js, [Conectar una base de datos a su Aplicación](#)
 - configurar la configuración de la base de datos SQLite, [conectar una Base de datos para su aplicación](#)
 - actualización de rutas con el modelo Libro, [Conexión de una base de datos a Tu aplicación](#)

- conectar la base de datos a la aplicación de marketing por correo electrónico, [conectar una Base de datos: Conexión de una base de datos](#)
- consultas de base de datos para la aplicación de análisis de sentimientos, [Conexión de una Base de datos y visualización](#)
- ORM para mapear objetos JavaScript a bases de datos SQL, [conectando un Base de datos para su aplicación](#)
- Secuestrar modelos que permitan la interacción con, [Conectando un Base de datos y visualización](#)
- configuración de la base de datos para el asistente de aprendizaje de IA, [configuración de su Autenticación de base de datos y usuarios: configuración de la autenticación de base de datos y usuarios](#)
- autenticación de usuarios para la base de datos del asistente de aprendizaje de IA, [Configuración Mejore su base de datos y la autenticación de usuarios: configuración de su Autenticación de bases de datos y usuarios](#)
- trabajar con proyectos de Node, [trabajar con bases de datos en Node Proyectos-Próximos pasos](#)
 - ventajas de usar bases de datos, [Por qué son importantes las bases de datos](#)
 - elegir una base de datos para un proyecto, [elegir una persistencia capa](#)
 - comparación de bases de datos, [Comparación de bases de datos para Node Aplicaciones](#)
 - expansión a bases de datos alojadas en la nube, [servicios en la nube](#)
 - bases de datos alojadas localmente versus en la nube, [local versus nube-Bases de datos alojadas](#)
 - MongoDB, [MongoDB: NoSQL para esquemas flexibles](#)
 - PostgreSQL, [PostgreSQL: Poder y estructura de SQL](#)

- esquemas, tablas, colecciones y documentos, [Core Conceptos: Esquemas, Tablas, Colecciones y Documentos](#)
- SQLite, [SQLite: Ligero e integrado](#)

almacén de datos, utilizando Redis como, [agregando un servidor Redis](#)

Clase DataTypes (sequelize), [Conexión a una base de datos y Visualización](#)

Debian/Ubuntu Linux

- Instalación de Node, [Instalación de Linux](#)
- instalación de VS Code, [instalación de Linux](#)

Método DELETE (HTTP), [Trabajar con Fastify, Obtener planificación](#)

- flujo de datos en la solicitud DELETE, [Obtener planificación](#)

Deserialización, [guardado y protección de cuentas de usuario](#)

Función deserializeUser: [Guardar y proteger cuentas de usuario](#)

Método deserializeUser (Cuenta), [Guardar y proteger el usuario Cuentas](#)

Tarea de desestructuración, [Obtener programación](#)

cuentas de desarrollador

- Claves API, trabajar con, [Trabajar con claves API](#)
- Cuenta API de Google Gemini, [API de Google Gemini](#)
- Atlas de MongoDB, [Atlas de MongoDB](#)
- Cuenta API OpenAI, [API OpenAI](#)
- límites de velocidad y cuotas para API, [API de Google Gemini](#)
- configurar una cuenta de GitHub, [GitHub](#)

Desarrollador, Node, convertirse, [Convertirse en un desarrollador de Node](#)

desarrollo

- conceptos avanzados de Node, [Dominando el oficio](#)
- masterización para Node, [Dominando el Oficio](#)

herramientas de desarrollo, configuración, [configuración de sus herramientas de desarrollo](#)

[Resumen](#)

- VS Code, [Configuración de sus herramientas de desarrollo](#)

Extensiones de VS Code, [extensiones de VS Code recomendadas](#)

directorios

- servicio de email marketing con base de datos, [Conexión de una base de datos](#)
 - Estructura del directorio del proyecto, [Obtener programación](#)
 - Estructura de directorio recomendada, [Directorio recomendado](#)
- [Estructura](#)
- directorio de código fuente, [Obtener programación](#)

Docker, [próximos pasos](#)

- ejecutar proyectos con, [Ejecutar proyectos con Docker-Ejecutar](#)
- [Proyectos con Docker](#)

Dockerfiles, [Ejecución de proyectos con Docker](#)

Gestor de bases de datos orientado a documentos (MongoDB), [Ahorro](#)
[Contraseñas con MongoDB](#)

documentos, [conceptos básicos: esquemas, tablas, colecciones y](#)
[Documentos](#)

paquete dotenv, [Obtener programación](#)

servicio de marketing por correo electrónico, [resumen de marketing](#) por correo

- agregar un marco para, [Agregar un marco para su correo](#)
Servicio: Cómo añadir [un marco para su servicio de correo](#)
- conectar una base de datos a la aplicación, [Conectar una base de datos-](#)
[Conectar una base de datos](#)
- partes cruciales de la aplicación, [Obtener planificación](#)
- píxel de marketing para la interacción por correo electrónico, implementación,
[Implementación de un píxel de marketing para la interacción por correo electrónico](#)
[Implementación de un píxel de marketing para la interacción por correo electrónico](#)
- planificación de la aplicación, [Su Aviso](#)
- programando tu mailer, [Obtener Programación-Obtener Programación](#)
- configurar el correo electrónico usando Google, [Get Programming](#)
- Programador de tareas, implementación, [Integración de un Programador de tareas-](#)
[Integración de un programador de tareas](#)

Plantillas de JavaScript incrustado (EJS), uso con Fastify para crear la interfaz de usuario de aplicaciones web, [Creación de su interfaz de usuario - Creación de su interfaz de usuario](#)

puntos finales (API), [Obtener planificación](#)

Archivo .env, almacenamiento de claves API, [trabajo con claves API](#)

variables de entorno

- para asistente de aprendizaje impulsado por IA, [Get Programming](#)
- iniciar el servidor Fastify con, [Configuración del servidor Fastify](#)

manejo de errores

- para el proceso de generación de correo electrónico, [Obtenga programación](#)

- en la ruta GET del enrutador API de la biblioteca, [Agregar rutas y acciones a Tu aplicación](#)

errores

- esperando pronta respuesta, [Analizando Sentimiento](#)
- middleware de manejo de errores de captura general, [Obtenga programación con un Diseño de API](#)

Módulos ES (ESM), [comprensión de package.json y scripts](#), patrones de sintaxis de nodos modernos

Sintaxis predeterminada de exportación de ES6, [Agregar rutas y datos](#)

Sintaxis del módulo ES6 (importación/exportación), [Obtener programación](#)

Escucha de eventos para el cliente Redis, [Agregar un servidor Redis](#)

bucle de eventos, [Dominando el oficio](#)

- bloqueo, [conseguir planificación](#)
- desafíos a los que se enfrentan las colas, [Coffee Order Manager](#)
- Aprovechamiento de Fastify, [Obtener planificación](#)
- manejo de múltiples tareas simultáneamente, [Uso de Fastify en este libro](#)

Arquitecturas basadas en eventos, compatibilidad con el sistema de mensajería de publicación/suscripción Redis, [adición de un servidor Redis](#)

exportaciones, sintaxis predeterminada de exportación de ES6, [agregar rutas y datos](#)

Expresar

- comparación con Fastify, [Uso de Fastify en este libro](#)

Clase ExtractJwt: [uso de JWT para la autenticación de API](#)

Fastify, [Cómo usar Fastify en este libro](#), [Cómo empezar a planificar](#), [Cómo empezar a programar con un diseño de API](#)

- agregar soporte de análisis codificado en JSON y URL a la aplicación, [Obtener Programación con un diseño de API](#)
- agregar configuraciones de proyecto para la aplicación de pedidos de café, [Integrar una Sistema de mensajería robusto](#)
- comparación con los marcos Express, Koa y Hapi, [utilizando Fastify en este libro](#)
- en el servicio de marketing por correo electrónico, [Get Planning](#)
- Se espera la carpeta de vistas para las plantillas, [Creación de un formulario de inicio de sesión](#)
- Explorando la plantilla de inicio, [Uso de Fastify en este libro](#)
- manejo de solicitudes JSON, [programación con diseño de API](#)
- instalar como marco de aplicación web para la aplicación de pedidos de café, [Obtenga programación](#)
- crear una instancia de la aplicación Fastify y comenzar a escuchar solicitudes, [obtener Programación con un diseño de API](#)
- arquitectura de bajo consumo, [uso de Fastify en este libro](#)
- sistema modular de complementos, [Uso de Fastify en este libro](#)
- analizar funciones de middleware, [aprender a programar con una API Disposición](#)
- configuración del servidor Fastify para el asistente de aprendizaje impulsado por IA
 - iniciar el servidor con variables de entorno, [configurar el Servidor Fastify](#)
- configuración del servidor Fastify para el asistente de aprendizaje impulsado por IA, [Configuración del servidor Fastify: Configuración del servidor Fastify](#)

- Manejo de consultas de IA, [Configuración del servidor Fastify](#)
- SSR, creación de archivos HTML con Handlebars, [Get Programming](#)
- uso para crear un servidor de correo, [agregar un marco para su servidor de correo Servicio](#)
- trabajando con, proyecto de servidor web de restaurante, [trabajando con Fastify - Cómo trabajar con Fastify](#)
- creación de interfaz de usuario mediante plantillas EJS, [Creación de su interfaz de usuario Tu interfaz de usuario](#)

Autenticación de inicio de sesión de Fastify

- Implementación exitosa con Passport.js, [Guardado y protección de cuentas de usuario](#)

Obtener API, [Obtener planificación](#)

Incitación de pocos disparos, [Personalización del Asistente de IA para el aprendizaje Asistencia](#)

archivo, comprobando su existencia antes de añadirlo, [Traduciendo Usuario Entrada a CSV](#)

sistemas de archivos, módulo fs para interacción con, [Obtener programación](#)

Opción force: true para Book.sync, [Conexión de una base de datos a su aplicación](#)

Solicitud de formato (estructuración de salida), [Personalización del Asistente de IA para asistencia en el aprendizaje](#)

formularios

- formulario de inicio de sesión, creación, [creación de un formulario de inicio de sesión – creación de un formulario de inicio de sesión Forma](#)

marcos (Node)

- comparación entre Fastify, Express, Koa y Hapi, **utilizando Fastify en este libro**
- Fastify y otros, **empieza a planificar**

Módulo fs (sistema de archivos), **Obtener programación**

GRAMO

API de Gemini: **creación de un asistente de aprendizaje impulsado por IA con API Gemini de Google**

- Conectando a, **Obtener Programación**
- cuenta de desarrollador con **API de Google Gemini**
- enviar indicaciones de usuario a, **Obtener programación**

GEMINI_API_KEY, **Obtener programación**

Bloque génesis en blockchain, **Codificación de la blockchain**

Función genStrategy: **Guardar y proteger cuentas de usuario**

Método GET (HTTP), **Trabajar con Fastify, Obtener planificación**

- flujo de datos en solicitud GET, **Get Planning**
- Ruta de seguimiento de campañas GET para el marketing por correo electrónico, **Implementación de una Píxel de marketing para la interacción por correo electrónico**
- Ruta de solicitud GET para libros en booksRouter personalizado, **agregando Rutas y acciones para su aplicación**
- Obtener ruta para la ruta /order-count en la aplicación de pedidos de café, **Obtener Programación**

Introducción a una aplicación Node, **Aplicación práctica: obtener Programación**

- mensaje de la computadora para crear un formato tabular, o CSV, de datos .

Inmediato

- herramientas y aplicaciones utilizadas, prerequisites para, [Práctica](#)
[Aplicación, creación de un servidor web de nodo, creación de un servidor local seguro](#)
[Administrador de contraseñas, fuente de agregación de contenido, API de biblioteca,](#)
[Análisis de sentimientos del procesador de lenguaje natural, marketing](#)
[Mailer, Web Scraper, Autenticación de aplicaciones, Pedido de café](#)
[Gerente, Mercado Blockchain de Sellos Musicales, Construyendo una IA-](#)
[Asistente de aprendizaje potenciado con la API Gemini de Google](#)

Repositorio de GitHub para este libro, [Acceso al repositorio de GitHub](#)

GitHub, configuración de una cuenta con [GitHub](#)

Aplicación API de Gmail, [empieza a programar](#)

Cuentas de Google y Google Cloud Console, [Aprende a programar](#)

Google, iniciar sesión en el correo electrónico a través de, [Obtener programación](#)

Gemini de Google (ver API de Gemini)

gráficos

- añadiendo al análisis de sentimientos de las entradas del diario, [conectando un Base de datos y visualización](#)
- definir configuraciones para gráficos de análisis de sentimientos, [conectar una base de datos y una visualización](#)

Áreas de crecimiento para los ingenieros de nodos, [Áreas de crecimiento para los ingenieros de nodos](#)

H

Biblioteca de manillares

- crear plantillas con, [Obtener programación](#)
- configurar la aplicación de autenticación de inicio de sesión para usar, [obtener programación](#)

- instalación de soporte para, [Obtener programación](#)
- expresiones sintácticas para usar en sus plantillas, [Obtener Programación](#)

Marco Hapi, comparación con los marcos Fastify, Express y Koa, [uso de Fastify en este libro](#)

hashes

- calcular y devolver el valor hash del bloque, [codificar el Cadena de bloques](#)
- generado a partir de la contraseña proporcionada y la sal en comparación con la almacenada hash para usuario, [guardar y proteger cuentas de usuario](#)

Algoritmos hash para contraseñas, [Guardar y proteger contraseñas de usuario Cuentas](#)

Resúmenes de hash, [cómo guardar y proteger cuentas de usuario](#)

paquetes hash, [cree un administrador de contraseñas local seguro](#)

Hashing de contraseñas, [Guardar y proteger cuentas de usuario](#), [Guardar y proteger Protección de cuentas de usuario](#), [guardado y protección de cuentas de usuario](#)

- definir función para en Account.js, [guardar y proteger usuarios Cuentas](#)
- Configuración de sal, [Guardar y proteger cuentas de usuario](#)

navegador sin cabeza, raspado de páginas web con, [raspado de páginas web con un navegador sin cabeza - raspado de páginas web con un navegador sin cabeza Navegador](#)

cadenas hexadecimales

- convertir un conjunto de bytes a, [guardar y proteger cuentas de usuario](#)

Homebrew, [Instalación](#)

- Instalación de Node con, [Instalación en Mac](#)

HTML, [empieza a programar](#)

- Contenido estándar para el formulario de inicio de sesión, [Creación de un formulario de inicio de sesión](#)
- crear un módulo que contenga plantillas HTML de correo electrónico, [agregar un Marco para su servicio de correo](#)
- EJS mostrando contenido dentro, [Construyendo su UI](#)
- tipos de entrada para cumplir con las expectativas del contenido de entrada, [creación de un Formulario de inicio de sesión](#)
- contenido de página de destino para aplicación web de restaurante, [Edificio](#)
- [Tu interfaz de usuario](#)
- análisis utilizando herramientas compatibles con HTML, [análisis con herramientas compatibles con HTML](#)
- [Herramientas: Análisis con herramientas compatibles con HTML](#)
- Estructura para otras páginas de aplicaciones web de restaurantes, [Edificio](#)
- [Tu interfaz de usuario](#)
- plantillas para, construir con Handlebars, [empezar a programar](#)
- usar para mejorar el correo electrónico, [empezar a programar](#)

Métodos HTTP, [Trabajar con Fastify](#), [Obtener planificación](#)

Módulo http, [Obtener planificación](#)

Servidor HTTP, configuración con Fastify, [Get Programming](#)

HTTP, métodos JavaScript para acceder al contenido, [Obtener planificación](#)

I

Operaciones de E/S, [Dominando el oficio](#)

Expresiones de función invocadas inmediatamente (IIFE), [Análisis Sentimiento](#)

Sintaxis del módulo de importación/exportación (ES6), [Obtener programación](#)

importaciones

- Sintaxis de importación de módulos ES6, [Trabajar con Fastify](#)
- importar solo los módulos y funciones necesarios en lugar de todo el contenido biblioteca, [obtener programación](#)

Archivo index.js, [Obtener programación](#)

- recopilar la entrada del usuario dentro de startApp (ejemplo), [traducir la entrada del usuario](#)
[Entrada a CSV](#)
- definir la clase Persona en (ejemplo), [Traducir la entrada del usuario a CSV](#)
- archivo final para la solicitud de la computadora que traduce la entrada del usuario a CSV,
[Traducir la entrada del usuario a CSV](#)
- probar funciones bcrypt en, [Construyendo una línea de comandos local](#)
[Gerente](#)

inferencia, [obtener programación](#)

instancias (Nodo), [Dominando el oficio](#)

Entrevistas (técnicas), preparándose para, [Su mensaje](#)

yoredis

- Conexión de Redis a Node.js, [Conexión desde Node](#)

Yo

JavaScript

- Nodo construido sobre el motor JavaScript V8 de Chrome, [comprensión](#)
[Nodo](#)
- proyectos abiertos de OpenJS Foundation, [trabajando con Fastify](#)

- ORM para mapear objetos JavaScript a bases de datos SQL, [conectando un Base de datos para su aplicación](#)

JavaScript, colas en, [obtener programación](#)

Aplicación JavaShipped con cuello de botella de solicitud (ejemplo), [Obtener Planificación](#)

Biblioteca jQuery, [análisis con herramientas compatibles con HTML](#)

JSON

- API basadas en JSON, [Obtener planificación](#)
- Análisis con Fastify, [Obtenga programación con un diseño de API](#)

Tokens web JSON (JWT)

- uso para autenticación de API, [uso de JWT para autenticación de API- Uso de JWT para la autenticación de API](#)
- uso para el registro y autenticación de usuarios para el asistente de IA, [Configuración de su base de datos y autenticación de usuarios: configuración Su base de datos y autenticación de usuarios](#)
 - middleware de autenticación, [configuración de su base de datos y Autenticación de usuario](#)
 - punto final de inicio de sesión del usuario, [configuración de su base de datos y usuario Autenticación](#)
 - punto final de registro de usuario, [configuración de su base de datos y Autenticación de usuario](#)

Paquete jsonwebtoken, [Configuración de su base de datos y usuario Autenticación](#)

JWT (ver Tokens web JSON)

Kafka: [Integrando un Sistema de Mensajería Robusto](#)

Marco Koa, comparación con los marcos Fastify, Express y Hapi, [uso de Fastify en este libro](#)

Yo

modelos de lenguaje grandes (ver LLM)

Perfiles de aprendizaje, [configuración de su base de datos y autenticación de usuarios](#)

- utilizando el asistente de consulta de IA, [configurando su base de datos y Autenticación de usuario](#)

libro mayor (blockchain), [obtener planificación](#)

lematización, [Obtener planificación](#)

bibliotecas, importando solo los módulos y archivos necesarios en lugar de la biblioteca completa, [Obtener programación](#)

Proyecto de API de biblioteca, [su resumen rápido](#)

- agregar rutas y acciones a la aplicación, [Agregar rutas y acciones A su aplicación: cómo agregar rutas y acciones a su aplicación](#)
 - Ruta POST, [Cómo añadir rutas y acciones a tu aplicación](#)
 - Rutas RESTful, [Cómo añadir rutas y acciones a tu aplicación](#)
 - Diseño de la estructura del directorio de enrutamiento, [adición de rutas y Acciones para su aplicación](#)
 - probar otras rutas que no sean GET, [agregar rutas y acciones a Tu aplicación](#)
- conectar una base de datos a la aplicación, [Conectar una base de datos a Su aplicación: Cómo conectar una base de datos a su aplicación](#)

- Comandos cURL que prueban rutas en la aplicación conectada a la base de datos, [Conexión de una base de datos a su aplicación](#)
- definir el modelo de libro en books.js, [conectar una base de datos a Tu aplicación](#)
- Estructura del directorio del proyecto con base de datos, [Conexión de una Base de datos para su aplicación](#)
- configurar la configuración de la base de datos SQLite, [conectar una Base de datos para su aplicación](#)
- actualización de rutas con el modelo Libro, [Conexión de una base de datos a Tu aplicación](#)
- planificar la aplicación, [Obtener planificación](#)
- Diseño de API de programación, [Obtenga programación con un diseño de API- Obtenga programación con un diseño de API](#)

Proyecto de API de biblioteca, agregar rutas y acciones a la aplicación, ruta de solicitud GET, [agregar rutas y acciones a su aplicación](#)

Biblioteca Libuv, [¿Qué está pasando bajo el capó?](#)

Linux

- instalación de MongoDB en [Linux](#)
- Instalación de Node, [Instalación de Linux](#)
- instalación de VS Code, [instalación de Linux](#)

función de escucha, [Obtener programación](#)

LLM (modelos de lenguaje extensos), [creación de un aprendizaje impulsado por IA Asistente con la API Gemini de Google](#)

- API de IA y, [Aprende a programar](#)

Balanceadores de carga para instancias de Node, [Dominando el oficio](#)

bases de datos locales versus bases de datos alojadas en la nube, bases de datos [locales versus bases de datos alojadas en la nube](#)

Bases de datos alojadas

localhost, [Trabajando con Fastify](#)

Clase LocalStrategy: [Cómo guardar y proteger cuentas de usuario](#)

- Método genStrategy que devuelve una nueva instancia de, [Guardando y Protección de cuentas de usuario](#)
- opciones para [guardar y proteger cuentas de usuario](#)

explotación florestal

- console.log y otros tipos de registro, [lectura y análisis de un Alimento](#)
- mensaje de registro cuando se realiza y procesa un pedido de café, [Obtenga programación](#)
- Biblioteca Pino incluida con Fastify, [Cómo usar Fastify en este libro](#)

Aplicación de nodo de autenticación de inicio de sesión, [Resumen](#) de planificación

- creación de plantillas de páginas HTML con Handlebars, [Get Programación](#)
- creación de un formulario de inicio de sesión, [Creación de un formulario de inicio de sesión – Creación de un formulario de inicio de sesión](#)
- planificación de la aplicación, [Su Aviso](#)
- guardar y proteger cuentas de usuario, [Guardar y proteger cuentas de usuario Cuentas](#)
- iniciar y probar su aplicación, [empezar a programar](#)
- uso de JWT para la autenticación de API, [uso de JWT para API Autenticación](#)

Estrategia de inicio de sesión, [guardar y proteger cuentas de usuario](#)

bucles

- bucle for de la ruta /slow-order, agregando a la ruta /process-order, [Obtenga programación](#)
- iteración de bucle largo para imitar la tarea de bloqueo, [Obtener programación](#)

METRO

aprendizaje automático (ML), [sentimiento del procesador de lenguaje natural](#)

Análisis-Resumen

- modelos, [análisis de sentimientos del procesador de lenguaje natural](#)
- procesador de lenguaje natural con análisis de sentimientos, [Su Resumen del tema](#)
 - analizar el sentimiento, [Analizar el sentimiento-Analizar Sentimiento](#)
 - limpieza del texto de entrada para el análisis de PNL, [obtener planificación](#)
 - conectar una base de datos y una visualización, [conectar una Base de datos y visualización: Conexión de una base de datos y Visualización](#)
 - Pasos de PNL para la entrada de texto en la aplicación Node, [comience a programar con Paquetes de procesamiento de cadenas: Programación con cadenas Procesamiento de paquetes](#)
 - planificar la aplicación, [Obtener planificación](#)

macOS

- Homebrew, [Instalación](#)
- Instalación de Node, [Instalación en Mac](#)
- Instalación de VS Code, [Instalación en Mac](#)

servidores de correo, [obtener programación](#)

correos, [comience a planificar](#)

- paquete nodemailer, [Obtener programación](#)

- configurar su correo con Google, [Get Programming](#)

Manifiestos (proyecto), [Inicialización de un proyecto de nodo \(usando npm\)](#)

correo de marketing (ver servicio de marketing por correo electrónico)

Píxel de marketing para la interacción por correo electrónico, [Implementación de un píxel de marketing](#)

[Píxel para la interacción por correo electrónico](#)

Nodos de mercado (blockchain), [Codificación de la blockchain](#)

- agregar nuevo nodo a la red, [empezar a programar](#)

- Función de transmisión para MarketplaceNode, [Obtener programación](#)

- funciones broadcastSelf y registerNode para MarketplaceNode,
[Obtenga programación](#)

- definir la clase MarketplaceNode, [Obtener programación](#)

- inicializando nueva instancia de MarketplaceNode, [obtener programación](#)

contraseña maestra, [Obtener planificación](#)

dominio del desarrollo de Node, [Dominando el oficio](#)

tokens máximos (API de IA), [obtener programación](#)

Medium, [tu mensaje](#)

memoria, guardar datos temporalmente, [crear una línea de comandos local](#)
[Gerente](#)

mensajería

- integrando un sistema de mensajería robusto con la aplicación de pedidos de café, [Integración de un sistema de mensajería robusto](#) : [Integración de un sistema de mensajería robusto](#)
[Sistema de mensajería](#)

- Sistema de mensajería de publicación/suscripción de Redis, [Cómo agregar un servidor Redis](#)

Sistemas de mensajería, opciones a considerar, [Integrando un Sistema Robusto](#)
[Sistema de mensajería](#)

MFA (autenticación multifactor), [autenticación de aplicaciones](#)

middleware

- agregar a la aplicación de pedidos de café, [obtener programación](#)
- middleware de análisis corporal para request.body, [Programación Get](#)

Funciones de middleware, [Obtenga programación con un diseño de API](#)

Minando un bloque, [Codificando la Blockchain](#), [Codificando la Blockchain](#)

Tipos de consulta de modelo, información sobre, [Conexión de una base de datos a](#)
[Tu aplicación](#)

modelos (ML), [análisis de sentimientos del procesador de lenguaje natural](#)

Sistema modular de complementos (Fastify), [Uso de Fastify en este libro](#)

módulos

- Sintaxis de importación/exportación de ES6, [Obtener programación](#), [Trabajar con](#)
[Fastify](#)
- Módulos ES modernos, [comprensión de package.json y scripts](#)

MongoClient, se utiliza para configurar la conexión al servidor MongoDB local.

[Guardar contraseñas con MongoDB](#)

MongoDB

- comparación con otras bases de datos para aplicaciones Node, [Base de datos Comparación para aplicaciones de nodo](#)
- conectando con Mangosta, [Conectando con Mangosta](#)
- instalación de [MongoDB: NoSQL para esquemas flexibles](#)
- instalación en Linux, [instalación en Linux](#)
- instalación en Windows, [instalación de Windows](#)
- guardar contraseñas en, [Guardar contraseñas con MongoDB-Guardar Contraseñas con MongoDB](#)
 - creando una función asíncrona para conectarse a MongoDB, [guardando Contraseñas con MongoDB](#)
 - Función principal para inicializar la base de datos, [Guardar contraseñas con MongoDB](#)
- fortalezas y desventajas, [Elección de una capa de persistencia](#)

Cuentas de MongoDB Atlas, [MongoDB Atlas](#)

Paquete mongodb: [Cómo guardar contraseñas con MongoDB](#)

Mangosta, [Conectando con Mangosta](#)

autenticación multifactor (MFA), [autenticación de aplicaciones](#)

MySQL

- comparación con otras bases de datos para aplicaciones Node, [Base de datos Comparación para aplicaciones de nodo](#)
- Sequelize trabajando con, [Conectando con Sequelize](#)

norte

procesamiento del lenguaje natural (ver PNL)

Paquete natural, [Obtenga programación con paquetes de procesamiento de cadenas](#)

Clase natural.PorterStemmer, [Análisis de sentimiento](#)

Función natural.PorterStemmer.stem, [Obtenga programación con paquetes de procesamiento de cadenas](#)

Clase natural.SentimentAnalyzer, [análisis de sentimientos](#)

Clase natural.WordTokenizer: [Obtenga programación con paquetes de procesamiento de cadenas](#)

Red de nodos (blockchain), [Codificación de la Blockchain](#)

redes neuronales, [Aprende a programar](#)

PNL (procesamiento del lenguaje natural), [procesador de lenguaje natural](#)
[Análisis de sentimientos](#)

- limpieza del texto de entrada para el análisis de PNL, [obtener planificación](#)
- corrección ortográfica en una cadena de texto, [Obtener Programación con Cadenas](#)
[Paquetes de procesamiento: Programación con procesamiento de cadenas](#)
[Paquetes](#)
- Función derivada para la aplicación Node, [Programación con cadenas](#)
[Procesamiento de paquetes](#)
- pasos en una cadena de texto, [Obtener planificación](#)
- Eliminación de palabras vacías, ejecución para la aplicación Node, [obtener programación con paquetes de procesamiento de cadenas](#)

Nodo

- ventajas y características únicas de, [Por qué Node se destaca](#)
- Convertirse en desarrollador, [Convertirse en desarrollador de Node](#)
- áreas de crecimiento para ingenieros, [áreas de crecimiento para ingenieros de nodos](#)

- Cómo funciona, ¿Qué está pasando bajo el capó?
- instalando, [instalando Node](#)
- patrones de sintaxis modernos, [Patrones de sintaxis de nodo modernos-Modern Patrones de sintaxis de nodos](#)
- package.json y scripts, comprensión, [Comprensión package.json y scripts: comprensión de package.json y scripts](#)
- comprensión, [Nodo de comprensión](#)
- Verificación de la configuración, [Uso de Fastify en este libro](#)

paquete node-fetch, [Obtener planificación](#)

Paquete node-schedule: [Integración de un programador de tareas](#)

paquete nodemailer, [obtener programación](#)

Paquete nodemon, [Obtenga programación con un diseño de API](#)

Puntuaciones de sentimiento normalizadas, [Conexión de una base de datos y Visualización](#)

Bases de datos NoSQL, [Conexión de una base de datos a su aplicación, Core Conceptos: Esquemas, Tablas, Colecciones y Documentos](#)

- Gestión de MongoDB, [Guardar contraseñas con MongoDB](#)

npm (Administrador de paquetes de nodos), [instalación en Linux](#)

- paquetes hash en el registro, [crear una contraseña local segura Gerente](#)
- inicializar proyectos con, [Inicializar un proyecto de nodo \(usando npm\)](#)
- Yarn versus, [Inicialización de un proyecto de Node \(usando npm\)](#)

Comandos npm, atajos y banderas de CLI, [trabajar con Fastify](#)

Comando npm init, [Trabajando con Fastify](#)

Comando npm install fastify, [Trabajando con Fastify](#)

Oh

Mapeador relacional de objetos (ORM) entre objetos JavaScript y bases de datos SQL,
[Conexión de una base de datos a su aplicación](#)

Cuentas API OpenAI, [API OpenAI](#)

ChatGPT de OpenAI: [creación de un asistente de aprendizaje impulsado por IA con API Gemini de Google](#)

Fundación OpenJS, [trabajando con Fastify](#)

ORM (mapeador relacional de objetos) entre objetos JavaScript y bases de datos SQL,
[Conexión de una base de datos a su aplicación](#)

PAG

Archivo package.json, [Introducción a la programación](#), [Trabajando con Fastify](#)

- documentación para, [Obtener programación](#)
- partes clave de, [Obtener programación](#)
- Comprensión de package.json y scripts, [Comprensión package.json y scripts: comprensión de package.json y scripts](#)

paquetes

- paquetes externos npm, trabajando con el indicador de la computadora
Proyecto, [Trabajar con paquetes externos](#) - [Trabajar con paquetes externos Paquetes](#)
- Control de versiones de paquetes npm, [Trabajando con Fastify](#)

Clase Parser, [lectura y análisis de un feed](#)

objetos del analizador, [lectura y análisis de un feed](#)

Función `parser.parseURL`: [lectura y análisis de un feed](#)

Paquete de pasaportes, [Guardar y proteger cuentas de usuario](#), [Guardar y proteger Protección de cuentas de usuario](#)

Paquete `Passport-Local`, [Cómo guardar y proteger cuentas de usuario](#), [Cómo guardar y proteger cuentas de usuario](#), [Cómo guardar y proteger cuentas de usuario](#)

`Passport.js`, [guardar y proteger cuentas de usuario](#)

- funciones para serializar y deserializar la información de la cuenta de usuario, [Guardar y proteger cuentas de usuario](#)
- hacer que la clase de `Cuenta` sea accesible para `el Usuario`, [Guardar y Proteger Cuentas](#)
- uso de sesiones en `Fastify`, [guardar y proteger cuentas de usuario](#)
- uso con sesiones en `Fastify`, [guardar y proteger usuarios Cuentas](#)

Método de autenticación de pasaporte, [Guardar y proteger cuentas de usuario](#), [Guardar y proteger cuentas de usuario](#)

- actualizado para autenticar el retorno, [guardar y proteger al usuario Cuentas](#)

administrador de contraseñas (seguro, local), creación, [crear un administrador de contraseñas local seguro](#)

Administrador [de contraseñas](#) - Resumen

- creación de un administrador de línea de comandos local, [creación de un administrador de línea de comandos local](#)

Administrador [de línea de comandos](#) : creación de una línea de comandos local
[Gerente](#)

- agregar importaciones de módulos y base de datos simulada a `index.js`, [creación de un Administrador de línea de comandos local](#)

- agregar las funciones viewPasswords y promptManageNewPassword, [crear un administrador de línea de comandos local](#)
- comparar la contraseña en hash con la contraseña en texto sin formato, [Creación de un administrador de línea de comandos local](#)
- funciones que solicitan al usuario que escriba la contraseña, [creando una Administrador de línea de comandos local](#)
- Función saveNewPassword, [creación de un comando local Gerente de línea](#)
- Función showMenu, [Creación de una línea de comandos local Gerente](#)
- planificar la aplicación, [Obtener planificación](#)
- guardar contraseñas en MongoDB, [guardar contraseñas con MongoDB: Cómo guardar contraseñas con MongoDB](#)
 - Función principal para inicializar la base de datos, [Guardar contraseñas con MongoDB](#)

contraseñas

- agregar entrada de contraseña condicional en el formulario de inicio de sesión, [creación de un inicio de sesión Forma](#)
- definir la función de hash de contraseña, [guardar y proteger el usuario Cuentas](#)
- Guardar para el formulario de registro de cuenta, [Crear un formulario de inicio de sesión](#)
- configuración de cuentas de usuario, [guardar y proteger cuentas de usuario](#)

Pino (biblioteca de registro), [uso de Fastify en este libro](#)

Correos electrónicos de texto sin formato, cómo mejorarlos con HTML, [aprender a programar](#)

complementos (Fastify), [Uso de Fastify en este libro](#)

Algoritmo de derivación de Porter: Programación con procesamiento de cadenas
Paquetes

Clase PorterStemmer, Análisis de Sentimientos

Método POST (HTTP), Trabajar con Fastify, Obtener planificación

- agregar ruta POST al enrutador API de la biblioteca, agregar rutas y Acciones para su aplicación
- agregar una ruta POST al servidor de correo, agregar un marco para su Servicio de correo
- creación de rutas POST /account y /auth, creación de un formulario de inicio de sesión
- flujo de datos en solicitud POST, Obtener planificación
- método utilizado para formularios, Creación de un formulario de inicio de sesión
- Solicitudes POST en la aplicación de pedidos de café, Get Programming
- Rutas POST que escuchan solicitudes en los puntos finales /account y /auth, guardando y protegiendo cuentas de usuario
- controlador de ruta para la ruta /slow-order, Obtener programación

PostgreSQL

- comparación con SQLite, Conexión de una base de datos a su aplicación
- comparación con otras bases de datos para aplicaciones Node, Base de datos Comparación para aplicaciones de nodo
- conectando con Sequelize, Conectando con Sequelize
- instalación de PostgreSQL: Poder y estructura de SQL
- fortalezas y desventajas, Elección de una capa de persistencia

Función de impresión, definición, construcción de un agregador

proyectos

- Construyendo mi primer proyecto desde cero, [Resumen de aplicación práctica](#)
 - solicitud de la computadora para crear un formato tabular, o CSV, de datos, [Su mensaje](#)
 - planificar la aplicación, [Obtener planificación](#)
 - programando la aplicación, [Obtener programación-Obtener programación](#)
 - traducir la entrada del usuario a CSV, [traducir la entrada del usuario a CSV: traducción de la entrada del usuario a CSV](#)
 - trabajar con paquetes externos, [trabajar con paquetes externos](#)
Paquetes: Trabajar [con paquetes externos](#)
- inicialización usando npm, [inicialización de un proyecto de nodo \(usando npm\)- Inicialización de un proyecto de nodo \(usando npm\)](#)
- Estructura de directorio recomendada, [Directorio recomendado Estructura](#)

Promesas

- función asíncrona envuelta en, [Traduciendo la entrada del usuario a CSV](#)
- Enrutamiento basado en promesas de Fastify, [Trabajar con Fastify](#)
- devuelto por la función await, [analizando el sentimiento](#)
- envolviendo lógica, habilitando la llamada de función usando async-await, [Guardar y proteger cuentas de usuario](#)

Función promisify, uso para encapsular funciones asíncronas, [traducción Entrada del usuario a CSV](#)

Ingeniería rápida, elaboración de instrucciones de IA efectivas, [personalización de la Asistente de IA para la asistencia en el aprendizaje](#)

Módulo de aviso, [Análisis de sentimientos](#)

paquete rápido, instalación y trabajo con, **trabajo con aplicaciones externas Paquetes**

Paquete prompt-sync, instalación, **creación de una línea de comandos local Gerente**

Función prompt.get, **Análisis de sentimiento**

Función promptNewPassword: **creación de una línea de comandos local Gerente**

Función promptOldPassword: **creación de una línea de comandos local Gerente**

indicaciones

- agregar una solicitud de entrada a la aplicación de análisis de sentimientos, **Analizando Sentimiento**
- agregar un método de solicitud a la aplicación de análisis de sentimientos, **conectar una Base de datos y visualización**

Prueba de trabajo (blockchain), **Codificación de la blockchain**

Método de publicación, **Agregar un servidor Redis**

Sistema de mensajería de publicación/suscripción (pub/sub), **Agregar un Redis Servidor**

- crear clientes para publicar y suscribirse, **agregar un Redis Servidor**

Titiritero (navegador sin cabeza), raspando páginas web con, **Raspando**

Páginas web con un navegador sin interfaz gráfica: extracción de páginas web con un navegador sin interfaz gráfica **Navegador sin cabeza**

Método PUT (HTTP), **Trabajando con Fastify**

- flujo de datos en solicitud PUT, **Obtener planificación**

Python, uso en aprendizaje automático, [procesador de lenguaje natural](#)
[Análisis de sentimientos](#)

Q

Instancias de cola, métodos utilizados con, [Programación Get](#)

paquete npm de cola, [Obtener programación](#)

colas, [Gerente de pedidos de café](#)

- (ver también RabbitMQ)
- cola interna para gestionar solicitudes de pedidos de café, [Obtener planificación](#)
- en JavaScript, [Aprende a programar](#)

Arquitectura de cola de mensajes de RabbitMQ, [Obtener planificación](#)

cuotas, [API de Google Gemini](#)

R

RabbitMQ

- instalación para su uso en proyectos Node, [RabbitMQ: basado en cola](#)
[Mensajería para aplicaciones de nodo](#)
- Integración con la aplicación de pedidos de café, [Integración de un sistema robusto](#)
[Sistema de mensajería](#) : Integración de un sistema de mensajería robusto
 - agregar configuraciones de proyecto Fastify y RabbitMQ,
[Integración de un sistema de mensajería robusto](#)
 - ventajas de RabbitMQ, [integrando una mensajería robusta](#)
[Sistema](#)
 - ventajas de configurar servicios con Fastify, [Integrando un](#)
[Sistema de mensajería robusto](#)

- Conexión al servidor RabbitMQ en index.js, [Integración de un sistema de mensajería robusto](#)
- características y capacidades de RabbitMQ, [integrando un robusto Sistema de mensajería](#)
- número de puerto para RabbitMQ, [integrando una mensajería robusta Sistema](#)
- servidor de cola de mensajes entre servicios, [Obtener planificación](#)

límites de velocidad, [API de Google Gemini](#)

Redis

- comparación con otras bases de datos para aplicaciones Node, [Base de datos Comparación para aplicaciones de nodo](#)
- instalación para su uso en proyectos Node, [Redis: velocidad en memoria para Almacenamiento en caché y colas](#)
- limitaciones de [agregar un servidor Redis](#)
- más información sobre [cómo agregar un servidor Redis](#)

Paquete Redis, instalación, [agregar un servidor Redis](#)

Servidor Redis, agregar a la aplicación de pedidos de café, [obtener planificación](#), [agregar un Servidor Redis](#) : cómo agregar [un servidor Redis](#) :

implementar Redis como patrón de mensajería de publicación/suscripción entre la aplicación y los datos de pedidos de bebidas, [cómo agregar un servidor Redis](#)

bases de datos relacionales, [conectar una base de datos a su aplicación](#)

objetos de respuesta), [Obtenga programación con un diseño de API](#)

Solicitar objetos, [Obtener planificación](#)

objetos de solicitud, [obtener programación con un diseño de API](#)

ciclo de solicitud-respuesta

- en la aplicación Fastify, [Trabajar con Fastify](#)
- HTTP, implementación de Fastify, [obtener planificación](#)

objeto request.body, [Obtener programación](#)

solicitudes por segundo (RPS), gran número manejado por Fastify, [utilizando Fastify en este libro](#)

Objetos de respuesta, [Obtener planificación](#)

Proyecto de servidor web para restaurante: [resumen de sus instrucciones](#)

API RESTful, rutas, [cómo agregar rutas y acciones a su aplicación](#)

Indicación de roles, [personalización del Asistente de IA para la asistencia al aprendizaje](#)

enrutamiento

- agregar rutas y acciones a la aplicación API de la biblioteca, [Agregar rutas y Acciones para su aplicación: cómo agregar rutas y acciones a su aplicación](#)
 - Ruta POST, [Cómo añadir rutas y acciones a tu aplicación](#)
 - Rutas RESTful, [Cómo añadir rutas y acciones a tu aplicación](#)
 - Diseño de la estructura del directorio de enrutamiento, [adición de rutas y Acciones para su aplicación](#)
 - probar otras rutas que no sean GET, [agregar rutas y acciones a Tu aplicación](#)
- agregar rutas al servidor web del restaurante, [Agregar rutas y datos](#)
- mercado de blockchain
 - definir/comprar ruta, [Codificando la Blockchain](#)
 - definir ruta de pago, [Codificación de la Blockchain](#)

- definir rutas /register-node y /sync-peers, **Obtener Programación**
- definiendo la ruta /songs, **Codificando la Blockchain**
- definir rutas API para autenticación, **uso de JWT para API Autenticación**
- definir rutas API para el servidor de correo, **agregar un marco para su Servicio de correo**
- diseñar rutas /cuenta y /auth para el formulario de registro, **crear un Formulario de inicio de sesión**
- Ruta de seguimiento de campañas GET para el marketing por correo electrónico, **Implementación de una Píxel de marketing para la interacción por correo electrónico**
- Ruta POST que escucha solicitudes en el punto final /cuenta, **guardando y protegiendo cuentas de usuario**
- Ruta POST escuchando solicitudes en el punto final /auth, **guardando y Protección de cuentas de usuario**
- ruta para imitar la solicitud retrasada en la aplicación de pedidos de café, **Obtener Programación**
- rutas para realizar pedidos en tu cola, agregar a la aplicación de pedidos de café, **Obtenga programación**
- actualización de rutas de Fastify para renderizar archivos EJS, **creación de su interfaz de usuario**
- actualización de rutas con el modelo de libro en la aplicación API de biblioteca, **Conexión una base de datos para su aplicación**
- ruta de verificación para verificar la dirección de correo electrónico en la aplicación de marketing, **Conectar una base de datos**

enrutamiento, agregar rutas y acciones a la aplicación API de la biblioteca, obtener ruta para libros en libros personalizados Enrutador, **agregar rutas y acciones a su**

Aplicación

Fuentes RSS, [fuente de agregación de contenido](#)

- Recetas de Bon Appetit en el navegador, [Obtener planificación](#)
- lectura y análisis de XML desde, [Lectura y análisis de un feed-](#)
[Lectura y análisis de un feed](#)

Paquete rss-parser: [lectura y análisis de un feed](#)

S

sal, [Construyendo un administrador de línea de comandos local](#)

- sal criptográfica personalizada para cada hash generado, [Guardar y](#)
[Protección de cuentas de usuario](#)

escalabilidad

- Arquitectura asincrónica y sin bloqueos de Fastify, [Uso de Fastify](#)
[en este libro](#)
- utilizando agrupamiento, [dominando el oficio](#)

Programadores, [comiencen a planificar](#)

- programador de tareas para servicio de marketing por correo electrónico, [Integración de una tarea](#)
[Programador: Integración de un programador de tareas](#)

Validación y serialización basadas en esquemas, utilizando AJV, [Uso de Fastify en](#)
[este libro](#)

esquemas, [conceptos básicos: esquemas, tablas, colecciones y](#)
[Documentos](#)

raspado de páginas web y uso de datos en una aplicación, [Web Scraper-](#)
[Resumen](#)

- acceder y extraer datos, [obtener planificación](#)

- Filtrado de datos extraídos, [Obtener planificación](#)
- Salida HTML del sitio, [Obtener programación](#)
- HTML, descripción general, [aprender a programar](#)
- identificar elementos HTML que vale la pena extraer, [aprender a programar](#)
- análisis con herramientas compatibles con HTML, [análisis con herramientas compatibles con HTML](#)
[Herramientas: Análisis con herramientas compatibles con HTML](#)
- planificar la aplicación, [Obtener planificación](#)
- raspado usando un navegador sin cabeza, [raspado de páginas web con un Navegador sin cabeza](#) : Rastreo de páginas web con un navegador sin cabeza
[Navegador](#)

scripts, [Comprensión de package.json y Scripts-Comprensión de package.json y Scripts](#)

Función de derivación de clave de propósito general de script, [Guardar y proteger Cuentas de usuario](#)

seguridad

- autenticación de aplicaciones, [autenticación de aplicaciones](#)
 - (ver también la aplicación Node de autenticación de inicio de sesión)
- configurar su correo con Google, [Get Programming](#)

Función sendMail, [Obtener programación](#)

- parámetros para pasar HTML personalizado y direcciones de correo electrónico, [agregando Un marco para su servicio de correo](#)

Aplicación de análisis de sentimientos, [procesador de lenguaje natural Sentiment Análisis](#), [Análisis del Sentimiento-Análisis del Sentimiento](#)

- añadir solicitud de entrada, [análisis de sentimientos](#)

- analizar el sentimiento con tokens, [Analizando el sentimiento](#)
- analizar la entrada de texto de una solicitud, [analizar el sentimiento](#)
- conectar una base de datos y visualizar las entradas del diario,

[Conexión de una base de datos y una visualización: Conexión de una base de datos y una visualización](#)

[Base de datos y visualización](#)

- Creación de la clase SentimentJournal, [Conexión de una base de datos y visualización](#)
- Gráfico para puntuaciones de sentimiento, [Conexión de una base de datos y visualización](#)
- solicitud de entrada de texto, [conexión a una base de datos y Visualización](#)

Clase SentimentAnalyzer, [Análisis de sentimientos](#)

Secuela

- Conexión con SQLite, [Conexión con Sequelize](#)
- Conexión de PostgreSQL a, [Conexión con Sequelize](#)

API de sequelize, información sobre, [Conexión de una base de datos a su](#)

[Aplicación](#)

Clase Sequelize: [Conexión de una base de datos a su aplicación](#)

- Modelo de libro, [Conexión de una base de datos a su aplicación](#)
- Creación de un modelo de cliente potencial para una aplicación de marketing por correo electrónico, [Conexión de un Base de datos](#)
- Función findOne, [Guardar y proteger cuentas de usuario](#)
- modelo para la conexión de base de datos en la aplicación de análisis de sentimientos, [Conexión de una base de datos y una visualización](#)

– campos updatedAt y createdAt, [Conexión de una base de datos a su](#)

[Aplicación](#)

Paquete sequelize, [Conexión de una base de datos a su aplicación](#), [Guardar y Protección de cuentas de usuario](#), [guardado y protección de cuentas de usuario](#)

Sequelize.Model, clase de cuenta de JavaScript que extiende, [guarda y protege cuentas de usuario](#)

serialización, [guardar y proteger cuentas de usuario](#)

– por Fastify, [Uso de Fastify en este libro](#)

Función serializeUser: [Guardar y proteger cuentas de usuario](#)

Método serializeUser (Cuenta), [Guardar y proteger cuentas de usuario](#)

Renderizado del lado del servidor (SSR), [Obtener programación](#)

– configuración mediante plantillas EJS con Fastify, [creación de su interfaz de usuario](#)

sesiones, [guardar y proteger cuentas de usuario](#)

– incorporando autenticación basada en tokens con, [utilizando JWT para Autenticación de API](#)

– uso en Node, [guardar y proteger cuentas de usuario](#)

– usar Passport.js con Fastify, [guardar y proteger usuarios Cuentas](#)

Configuración (Node y VS Code), verificación, [uso de Fastify en este libro](#)

Algoritmo hash SHA-256, [Codificación de la cadena de bloques](#)

Resumen del hash sha512: [Cómo guardar y proteger cuentas de usuario](#)

Contratos inteligentes (blockchain), [Obtener planificación](#)

Servidores de correo SMTP, crea el tuyo propio, [empieza a programar](#)

Clase SpellChecker, [Obtenga programación con procesamiento de cadenas](#)

[Paquetes](#)

- salida de línea de comandos para ortografías corregidas, [Obtener programación con paquetes de procesamiento de cadenas](#)

Corrección ortográfica, [empezar a planificar](#), [empezar a programar con cadenas](#)

[Paquetes de procesamiento: Programación con procesamiento de cadenas](#)

[Paquetes, conexión de una base de datos y visualización](#)

- agregar función de corrección ortográfica al proyecto Node, [Obtener Programación con paquetes de procesamiento de cadenas](#)
- resultado de la función de corrección ortográfica, asignación a la función de tokenización, [Programación con paquetes de procesamiento de cadenas](#)

Bases de datos SQL, [Conexión de una base de datos a su aplicación](#), [Core](#)

[Conceptos: Esquemas, Tablas, Colecciones y Documentos](#)

- ORM entre objetos JavaScript y, [Conexión a una base de datos a tu aplicación](#)

SQLite, [SQLite: Ligero e integrado](#)

- comparación con PostgreSQL, [Conexión de una base de datos a su aplicación](#)
- comparación con otras bases de datos para aplicaciones Node, [Base de datos Comparación para aplicaciones de nodo](#)
- conectar una base de datos a una aplicación de marketing por correo electrónico, [conectar una Base de datos: Conexión de una base de datos](#)
- conectando con Sequelize, [Conectando con Sequelize](#)
- instalación local, [Instalación](#)
- instalar el paquete npm más reciente, [Conectar una base de datos a Tu aplicación](#)

- configuración de bases de datos y autenticación de usuarios para el aprendizaje de IA
Asistente, [Configuración de su base de datos y autenticación de usuarios-](#)
[Configuración de su base de datos y autenticación de usuarios](#)
- configurar la configuración de la base de datos, [conectar una base de datos a](#)
[Tu aplicación](#)
- configurar las configuraciones de la base de datos para la aplicación de autenticación de inicio de sesión,
[Guardar y proteger cuentas de usuario](#)
- fortalezas y desventajas, [Elección de una capa de persistencia](#)

Paquete sqlite3, [Guardar y proteger cuentas de usuario](#), [Guardar y proteger](#)
[Protección de cuentas de usuario](#)

SSR (renderizado del lado del servidor), [obtener programación](#)

Función asincrónica startApp (ejemplo), [Traducción de la entrada del usuario a](#)
[CSV](#)

Plantilla de inicio (Fastify), exploración, [uso de Fastify en este libro](#)

Plugin estático (Fastify), [mejorando la interfaz de usuario](#)

derivación y lematización, [empezar a planificar](#)

- agregar función de derivación al proyecto Node, [Obtener programación](#)
[con paquetes de procesamiento de cadenas](#)

eliminación de palabras vacías

- realizando una aplicación Node, [obtenga programación con cadenas](#)
[Procesamiento de paquetes](#)

Paquete de palabras vacías, [Obtenga programación con procesamiento de cadenas](#)
[Paquetes](#)

arroyos, [Dominando el oficio](#)

Paquetes de procesamiento de cadenas, [Obtenga programación con procesamiento de cadenas](#)
[Paquetes: Programación con procesamiento de cadenas](#)

Método de suscripción, [Agregar un servidor Redis](#)

Sincronización del modelo de libro con la base de datos, [Conexión de un](#)
[Base de datos para su aplicación](#)

T

Tablas, [conceptos básicos: esquemas, tablas, colecciones y](#)
[Documentos](#)

Programador de tareas para el servicio de marketing por correo electrónico, [Integración de una tarea](#)
[Programador: Integración de un programador de tareas](#)

Temperatura (API de IA), [Obtener programación](#)

Plantillas, Handlebars, [Obtener Programación](#)

– (ver también biblioteca Handlebars)

Motores de plantillas, uso con Node y Fastify, [creación de su interfaz de usuario](#)

– Soporte de Fastify para [mejorar la interfaz de usuario](#)

texto

– limpieza del texto de entrada para el análisis de PNL, [obtener planificación](#)

– descifrando el significado del uso de modelos ML, [Obtener planificación](#)

esta (palabra clave), [Guardar y proteger cuentas de usuario](#)

Temporizadores, [Dominando el oficio](#)

marcas de tiempo

– incluso para cada entrada de contacto, [Trabajar con paquetes externos](#)

Autenticación basada en tokens, [uso de JWT para la autenticación de API](#)

- (ver también Tokens web JSON)

Tokenizar texto, [Obtener planificación](#), [Obtener programación con cadenas](#)

[Procesando paquetes, obtener programación](#)

- agregar función de tokenización al proyecto Node, [Obtener programación con paquetes de procesamiento de cadenas](#)
- analizar el sentimiento con tokens, [Analizando el sentimiento](#)
- asignar el resultado de la función de ortografía correcta a la tokenización
Función, [Obtener programación con paquetes de procesamiento de cadenas](#)

tokens (API), [Trabajar con claves API](#)

píxel de seguimiento (ver píxel de marketing para interacción por correo electrónico)

Clase de transacción, [Codificación de la cadena de bloques](#)

transacciones (blockchain), [Obtener planificación](#)

Redes neuronales basadas en transformadores (LLM), [Get Programming](#)

transportador (correo), configurando en index.js, [Obtener Programación](#)

Función transporter.sendMail, [Obtener programación](#)

U

Interfaces de usuario (UI)

- creación de interfaz de usuario para la aplicación del servidor web del restaurante, [Building Your UI - Construyendo su UI](#)
- mejorar la interfaz de usuario de la aplicación web del restaurante, [embellecer la interfaz de usuario- Mejorando la interfaz de usuario](#)

URI (Identificadores uniformes de recursos), [obtener planificación](#)

Codificación de URL

- agregar un complemento de análisis a la aplicación Fastify para **programar con un diseño de API**
- solicitudes entrantes con cargas útiles codificadas en URL, **Obtener Programación**

URL

- definir para leer desde el agregador de contenido, **Construir un Agregador**
- URL de origen HTML de la página web, **análisis con HTML compatible**
Herramientas

cuentas de usuario

- aplicación y API dedicadas para **obtener planificación**
 - (ver también la aplicación Node de autenticación de inicio de sesión)
- guardar y proteger, **Guardar y proteger cuentas de usuario**

Entrada del usuario, traducción a CSV, **Traducción de la entrada del usuario a CSV- Traducir la entrada del usuario a CSV**

nombres de usuario

- registro de cuentas de usuario, **guardar y proteger cuentas de usuario**
Cuentas
- guardar desde el formulario de registro de cuenta, **crear un formulario de inicio de sesión**

V

Motor JavaScript V8, **comprensión de Node**

validación

- validación de entrada para correo electrónico y números de teléfono, **trabajando con Paquetes externos**

- Regla de secuestro de isEmail para el servicio de marketing por correo electrónico, [Conexión de un Base de datos](#)
- Validación basada en esquemas con AJV, [uso de Fastify en este libro](#)

variables

- definir la página de autenticación en index.js, [crear un inicio de sesión Forma](#)
- definir formVars para que coincidan con las variables de la página, [crear un inicio de sesión Forma](#)
- nombres precedidos por un guión bajo (_), [Obtener programación con un Diseño de API](#)

Versiones, control de versiones de paquetes npm, [trabajar con Fastify](#)

complemento de visualización (Fastify), [creación de su interfaz de usuario](#)

Carpeta de vistas, Plantillas de Handlebars, [Creación de un formulario de inicio de sesión](#)

visualizaciones

- definir configuraciones para gráficos de análisis de sentimientos, [conectar una base de datos y una visualización](#)
- Gráfico en la aplicación de análisis de sentimientos, [Conexión a una base de datos y visualización](#)

VS Code (Visual Studio Code): [configuración de sus herramientas de desarrollo](#)

- instalación, [Instalación de VS Code](#)
- extensiones recomendadas, [extensiones recomendadas de VS Code](#)
- Verificación de la configuración, [Uso de Fastify en este libro](#)

O

navegadores web

- cookies enviadas al navegador del cliente para la sesión del usuario, [Guardar y Protección de cuentas de usuario](#)
- representación de la página menu.ejs en, [Construyendo su UI](#)
- raspado de contenido de páginas con navegador sin cabeza, [raspado web](#)
[Páginas con un navegador sin interfaz gráfica: páginas web con un navegador sin interfaz gráfica](#)
[Navegador sin cabeza](#)
- ver la respuesta de su servidor web en, [Trabajar con Fastify](#)

raspado web, [raspador web](#)

- (ver también raspado de páginas web y uso de datos en una aplicación)

Servidor web, construcción, [Construcción de un servidor web de nodo: resumen](#)

- acerca de los servidores web, [Obtener planificación](#)
- agregar rutas y datos, [Agregar rutas y datos-Agregar rutas y datos](#)
- bloqueando el bucle de eventos, [obtener planificación](#)
- construcción del esqueleto de la aplicación web, [construcción de la aplicación](#)
[Esqueleto](#)
- Creación de la interfaz de usuario de la aplicación web del restaurante, [Creación de su propia interfaz de usuario](#)
[Tu interfaz de usuario](#)
- mejorar la interfaz de usuario de la aplicación web del restaurante, [embellecer la interfaz de usuario-](#)
[Mejorando la interfaz de usuario](#)
- planificar la aplicación, [Obtener planificación](#)
- usando Fastify, [Trabajando con Fastify-Trabajando con Fastify](#)
- solicitudes web, procesamiento en la aplicación web impulsada por Fastify,
[Obtener planificación](#)

Ventanas

- instalación de MongoDB en [Windows](#)
- Instalación de Node, [Instalación de Windows](#)
- instalación de VS Code, [instalación de Windows](#)

Clase WordTokenizer: [Programación con procesamiento de cadenas](#)
[Paquetes](#)

Indógena

XML

- acceder al feed RSS de Bon Appétit, [Obtener planificación](#)
- lectura y análisis de fuentes RSS, [Lectura y análisis de una](#)
[Lectura y análisis de feeds](#)
- uso mediante feeds RSS, [Obtener planificación](#)

Interfaz XMLHttpRequest, [Obtener planificación](#)

Y

Yarn versus npm: [Inicialización de un proyecto de Node \(usando npm\)](#)

Z

ZeroMQ: [Integrando un Sistema de Mensajería Robusto](#)

Acerca del autor

Jonathan Wexler, autor del notable libro *Get Programming with Node.js*, aporta su amplia experiencia en ingeniería de software y pasión por la enseñanza hasta su último trabajo, *Proyectos Node.js*. Su enfoque de la escritura, profundamente arraigado en la experiencia práctica y en una comprensión intuitiva de las tecnologías web, particularmente Node.js, transmite sutilmente su profunda experiencia. Wexler refleja elementos de sus experiencias enseñando en un campamento de codificación y desarrollando aplicaciones empresariales en grandes empresas tecnológicas para ayudar a descomponer conceptos técnicos complejos en proyectos atractivos y manejables. Esto le ha valido reconocimiento y críticas positivas, lo que hace que su La orientación que se ofrece en este nuevo libro es un recurso invaluable para los desarrolladores que buscan para mejorar sus habilidades a través de aplicaciones en el mundo real.

Colofón

El animal de la portada de Nodejs Projects es la jirafa angoleña, también conocida como jirafa de Namibia o jirafa ahumada, una de varias especies o subespecies de jirafa nativas de Angola y Namibia.

La jirafa angoleña se identifica fácilmente por las manchas grandes, irregulares y a menudo muy angulosas que cubren todo su cuerpo, contrastando marcadamente con el color de fondo más claro de su pelaje. Este patrón distintivo sirve como excelente camuflaje en su hábitat de sabana.

Como todas las jirafas, las jirafas angoleñas son ramoneadoras, utilizando sus largos cuellos para alcanzar las hojas en lo alto de los árboles, una fuente de alimento prácticamente inaccesible para otros herbívoros. Debido a su impresionante altura, requieren un sistema cardiovascular especializado —con una presión arterial notablemente alta y una red de válvulas unidireccionales— para bombear sangre a la cabeza contra la fuerza de la gravedad.

Con aproximadamente 13.000 ejemplares en libertad, las jirafas angoleñas enfrentan desafíos en su hábitat natural, pero, debido a la tendencia al alza de su población, actualmente no están amenazadas, según la Unión Internacional para la Conservación de la Naturaleza. Muchos de los animales que aparecen en las portadas de O'Reilly están en peligro de extinción; todos son importantes para el mundo.

La ilustración de la portada es de Monica Kamsvaag, basada en un grabado antiguo. El diseño de la serie es de Edie Freedman, Ellie Volckhausen y Karen Montgomery. Las fuentes de la portada son Gilroy Semibold y Guardian Sans. La fuente del texto es Adobe Minion Pro; la fuente del encabezado es Adobe Myriad Condensed; y la fuente del código es Ubuntu Mono de Dalton Maag.